

操作系统的发展史

Unix

Minix

Linux

操作系统的发展

Minix没有火起来的原因

Linux介绍

Linux内核&发行版

Linux内核版本

Linux发行版本

类Unix系统目录结构

Linux目录

用户目录

命令行基本操作

命令使用方法

查看帮助文档

help

man(manual)

tab键自动补全

history游览历史

命令行中的ctrl组合键

Linux命令

权限管理

列出目录的内容: ls

显示inode的内容: stat

文件访问权限

修改文件权限: chmod

修改文件所有者: chown

修改文件所属组: chgrp

文件内容查看

基本显示: cat、tac

显示行号: nl

分屏显示: more、less

取首尾n行: head、tail

文件管理

输出重定向: >

管道: |

清屏: clear

切换工作目录: cd

显示当前路径: pwd

创建目录: mkdir

删除文件: rm

建立链接文件: ln

文本搜索: grep

查找文件: find

拷贝文件: cp

移动文件: mv

归档管理: tar

文件压缩解压: gzip、bzip2

文件压缩解压: zip、unzip

查看命令位置: which

用户和用户组管理

查看当前用户: whoami

查看登录用户: who

退出登录账户：exit
添加用户账号：useradd
修改用户：usermod
设置用户密码：passwd
删除用户：userdel
切换用户：su
以root身份执行指令：sudo
添加、删除组账号：groupadd、groupdel
用户组管理：groupmod

系统管理

查看当前日历：cal
显示或设置时间：date
查看网络状态：netstat
查看进程信息：ps
以树状图显示进程关系：pstree
动态显示进程：top
终止进程：kill
关机重启：reboot、shutdown、init
检测磁盘空间：df
检测目录所占磁盘空间：du
查看或配置网卡信息：ifconfig
测试远程主机连通性：ping

Linux 磁盘管理

df
du
fdisk
磁盘格式化
磁盘检验
磁盘挂载与卸载

Linux管道命令

wc - 统计字数
cut - 列选取命令
grep - 行选取命令
sort - 排序
uniq - 去重
tee - 同时输出多个文件
tr - 替换指定的字符
join - 文件按行连接
paste-将多个文件对应行链接在一起
split - 文件切割
xargs - 参数代换

linux命令练习

常用命令

系统命令

用户和组

权限

帮助文档

VIM

查找

打包与压缩

正则表达式

输入输出重定向

进程控制

其他命令

远程文件复制：scp

locate查找

which命令

echo

设置或显示环境变量：export

- 修改主机名&ip地址

- mount挂载

- ssh免密登陆

- 批量添加用户

- 与用户账号有关的系统文件

- 实操

- crontab的使用

特殊权限

- 三种特殊的权限

- SetUID(suid)

- setGID (sgid)

- sticky(sbit)粘滞位

- chattr权限

- lsattr：查看特殊权限

- chattr：设置特殊权限

- 掩码umask

- umask的作用

- umask是如何改变新文件的权限

- 永久修改umask

linux软件安装

- Ubuntu软件安装与卸载

- 更新Ubuntu软件下载地址

- Ubuntu软件操作的相关命令

- yum安装命令

- 更新国内yum源

- yum常用命令

- yum在线安装MySQL5.7

- rpm

- Linux的基本配置

- 安装JDK

- 制作本地YUM源

Shell编程

- 入门

- 运行Shell脚本

- 编写一个快捷创建shell脚本的命令

- echo命令

- 显示换行

- 显示不换行

- printf 命令

- Shell 注释

- Shell变量

- 定义变量

- 使用变量

- 只读变量

- 删除变量

- 变量类型

- Shell 函数

- 文件包含

- shell数据类型

- 字符串

- 数组

- 定义数组

- 读取数组

- 获取数组的长度

- Shell传递参数

- Shell基本运算符

- 算术运算符

- 关系运算符

- 布尔运算符

- 逻辑运算符
- 字符串运算符
- 文件测试运算符

test命令

- 数值测试
- 字符串测试
- 文件测试

Shell 流程控制

- if else判断语句
- for循环
- while循环
- 无限循环
- until 循环
- case
- 跳出循环

Shell输入/输出重定向

- 输出重定向
- 输入重定向
- 重定向深入讲解
- Here Document
- /dev/null 文件

实例

- 杨辉三角：
- sum()&max():
- 99乘法表：

文本编辑命令

cut命令

sed命令

- 元字符集
- a|i:在指定行位置添加行
- d:删除指定行
- c:替换指定行
- p:仅显示指定行
- s:字符串替换
- y:单字符替换
- hHgG 模式空间&保持空间
- 多次指定 -e 选项进行多点编辑
- 选项 -i 直接修改文件内容

awk命令

基本用法

- F 指定分割字符
- v 设置变量
- f 指定awk脚本

AWK工作原理

- 运算符
- 输出顺序号 NR, 匹配文本行号
- 指定输出分割符
- 忽略大小写
- 正则字符串匹配
- 一些实例

awk编程

- 条件语句IF&ELSE
- 循环语句For&While
- awk数组
- AWK 用户自定义函数
- AWK 内置函数
- 算数函数
- 字符串函数

- 时间函数
- 位操作函数
- 其他函数

vim全套笔记

VIM快速复习

- 什么是 vim?
- 按:冒号即可进入last line mode
- 从command mode进入Insert mode
- 按ESC键退回command mode
- 多行编辑, vim支持, vi不支持

vim练习

vi/vim的三种模式

- 命令模式
- 输入模式
- 底行模式

vim基础操作

- 进入输入模式(Insert mode)
- 进入替换模式(Replace mode)
- 命令模式下常用命令
 - 移动光标
 - 删除操作
 - 撤销&复原&重复
 - 复制&粘贴
 - 合成行
 - 搜索
 - 替换
- 底行命令模式的常用操作
- 可视模式

Linux系统启动过程

- 加载内核
- 启动初始化进程init
- 确定运行级别
- 加载开机启动程序
- 用户登录
- 图形模式与文字模式的切换方式
- login shell
- Linux关机
- 计算机启动的流程
 - 第一阶段: BIOS
 - 第二阶段: 主引加粗样式导记录
 - 第三阶段: 硬盘启动
 - 第四阶段: 操作系统
- 单用户模式修改Centos系统root密码
- 救援模式修改Ubuntu系统root密码

虚拟机

- 三种网络模式
- 常见问题
 - 修改静态地址后发现无法ping外网
 - 虚拟机克隆后eth0消失

操作系统的发展史

Unix

- 1965年之前, 电脑只有军事或者学院的研究机构碰的起, 当时大型主机至多能提供30台终端 (30个键盘、显示器)的连接。

- 1965年左右由贝尔实验室、麻省理工学院 以及 通用电气共同发起了Multics项目，**想让大型主机支持300台终端**
- 1969年前后这个项目进度缓慢，资金短缺，贝尔实验室退出了研究
- 1969年从这个项目中退出的Ken Thompson当时在实验室无聊时，为了让一台空闲的电脑上能够运行“星际旅行”游行，在8月份左右趁着其妻子探亲的时间，用了1个月的时间编写出了 Unix操作系统的原型
- 1970年，美国贝尔实验室的 Ken Thompson，以 BCPL语言 为基础，设计出很简单且很接近硬件的 B语言（取BCPL的首字母），并且他用B语言写了第一个UNIX操作系统。
因为B语言的跨平台性较差，为了能够在其他的电脑上也能够运行这个非常棒的Unix操作系统，Dennis Ritchie和Ken Thompson 从B语言的基础上准备研究一个更好的语言
- 1972年，美国贝尔实验室的 Dennis Ritchie在B语言的基础上最终设计出了一种新的语言，他取了BCPL的第二个字母作为这种语言的名字，这就是C语言
- 1973年初，C语言的主体完成。Thompson和Ritchie迫不及待地开始用它完全重写了现在大名鼎鼎的Unix操作系统

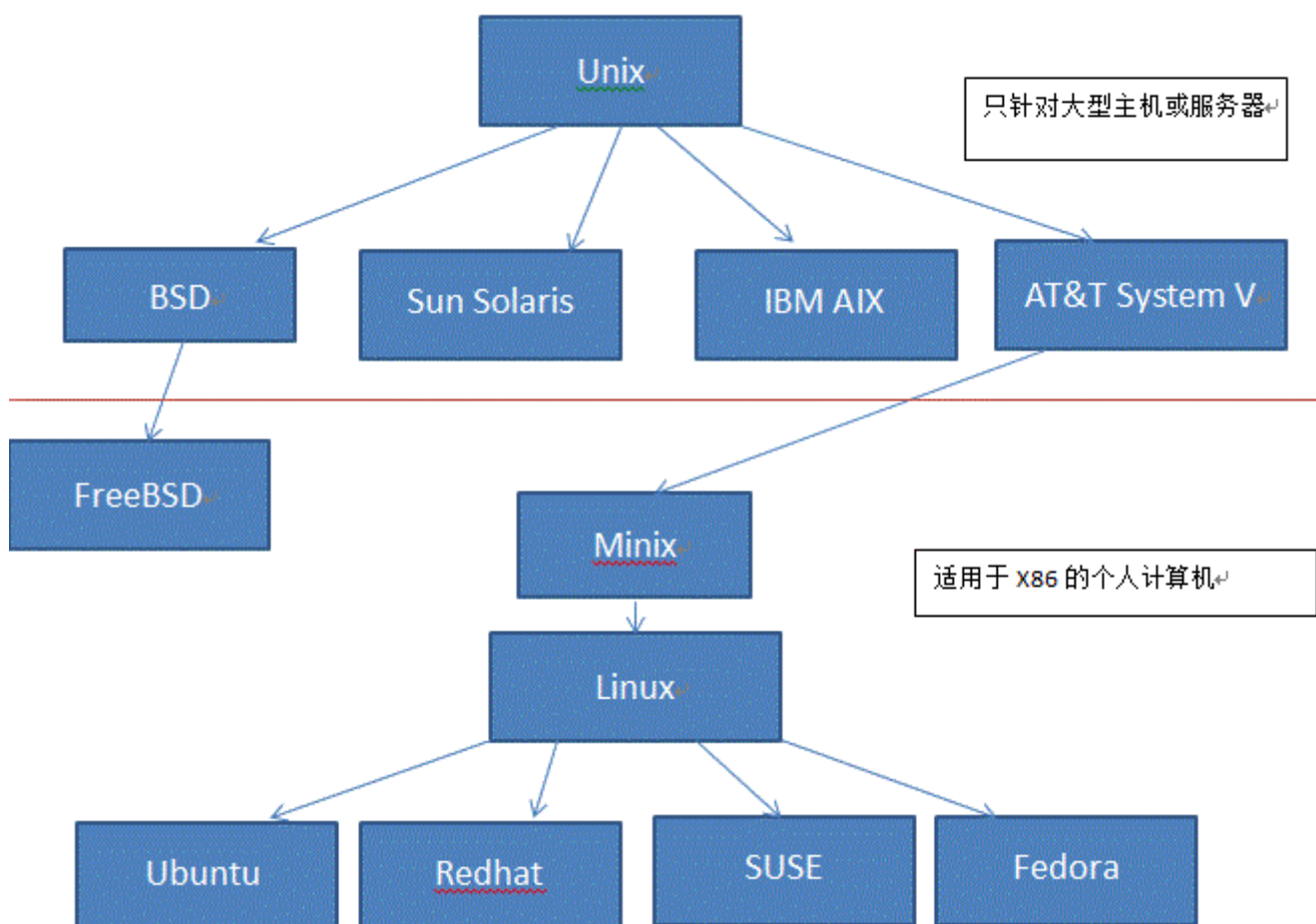
Minix

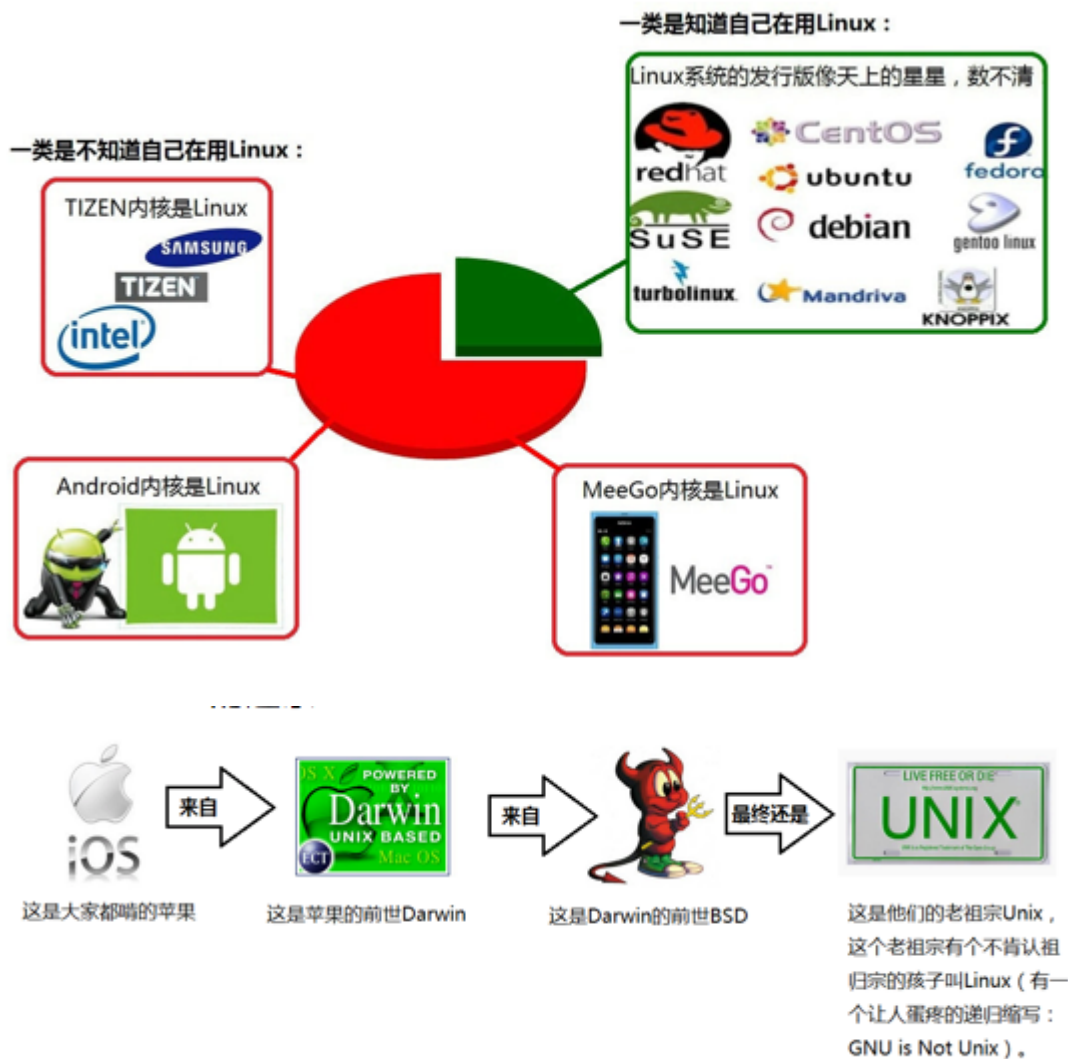
因为AT&T(通用电气)的政策改变，在Version 7 Unix推出之后，发布新的使用条款，将UNIX源代码私有化，在大学中不再能使用UNIX源代码。Andrew S. Tanenbaum(塔能鲍姆)教授为了能在课堂上教授学生操作系统运作的实务细节，决定在不使用任何AT&T的源代码前提下，自行开发与UNIX兼容的操作系统，以避免版权上的争议。他以小型UNIX (mini-UNIX) 之意，将它称为MINIX。

Linux

因为Minix只是教学使用，因此功能并不强，因此Torvalds利用GNU的bash当做开发环境，gcc当做编译工具，编写了Linux内核-v0.02，但是一开始Linux并不能兼容Unix，即Unix上跑的应用程序不能在Linux上跑，即应用程序与内核之间的接口不一致，因为Unix是遵循POSIX规范的，因此Torvalds修改了Linux，并遵循POSIX (Portable Operating System Interface，他规范了应用程序与内核的接口规范)；一开始Linux只适用于386，后来经过全世界的网友的帮助，最终能够兼容多种硬件

操作系统的发展





Minix没有火起来的原因

Minix的创始人说，MINIX 3没有统治世界是源于他在1992年犯下的一个错误，当时他认为BSD必然会一统天下，因为它是一个更稳定和更成熟的系统，其它操作系统难以与之竞争。因此他的MINIX的重心集中在教育上。四名BSD开发者已经成立了一家公司销售BSD系统，他们甚至还有一个有趣的电话号码1-800-ITS-UNIX。然而他们正因为这个电话号码而惹火上身。美国电话电报公司因电话号码而提起诉讼。官司打了三年才解决。在此期间，BSD陷于停滞，而Linux则借此一飞冲天。他的错误在于没有意识官司竟然持续了如此长的时间，以及BSD会因此受到削弱。如果美国电话电报公司没有起诉，Linux永远不会流行起来，BSD将统治世界。

Linux介绍

Linux内核&发行版

Linux内核版本

内核(kernel)是系统的核心，是运行程序和管理像磁盘和打印机等硬件设备的核心程序，它提供了一个在裸设备与应用程序间的抽象层。

Linux内核版本又分为稳定版和开发版，两种版本是相互关联，相互循环：

- 稳定版：具有工业级强度，可以广泛地应用和部署。新的稳定版相对于较旧的只是修正一些bug或加入一些新的驱动程序。
- 开发版：由于要试验各种解决方案，所以变化很快。

内核源码网址：<http://www.kernel.org> 所有来自全世界的对Linux源码的修改最终都会汇总到这个网站，由Linus领导的开源社区对其进行甄别和修改最终决定是否进入到Linux主线内核源码中。

Linux发行版本

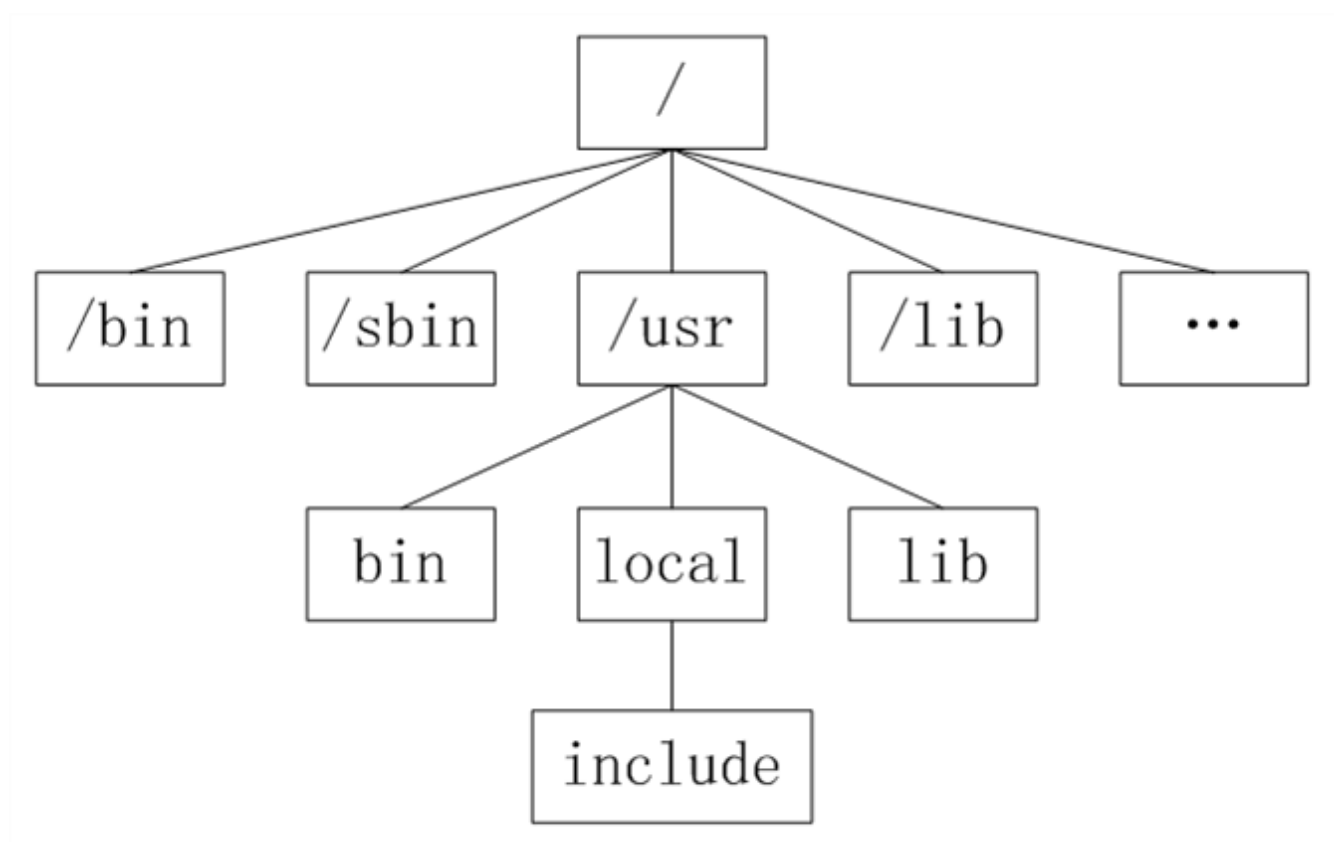
Linux发行版 (也被叫做 GNU/Linux 发行版) 通常包含了包括桌面环境、办公套件、媒体播放器、数据库等应用软件。

目前市面上较知名的发行版有：Ubuntu、RedHat、CentOS、Debian、Fedora、SuSE、OpenSUSE、Arch Linux、SolusOS 等。

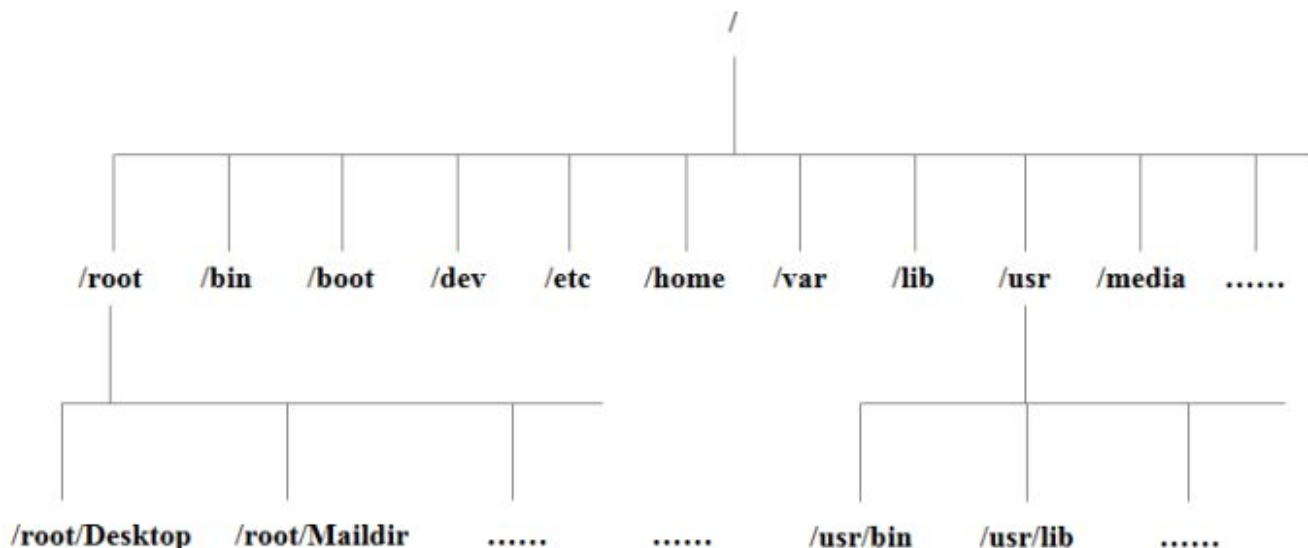


类Unix系统目录结构

Unix没有盘符这个概念，只有一个根目录/，所有文件都在它下面



Linux目录



- /: 根目录，一般根目录下只存放目录，在Linux下有且只有一个根目录。所有的东西都是从这里开始。当你在终端里输入“/home”，你其实是在告诉电脑，先从/（根目录）开始，再进入到home目录。
- /bin: /usr/bin: 可执行二进制文件的目录，如常用的命令ls、tar、mv、cat等。
- /boot: 放置linux系统启动时用到的一些文件，如Linux的内核文件: /boot/vmlinuz，系统引导管理器: /boot/grub。
- /dev: 存放linux系统下的设备文件，访问该目录下某个文件，相当于访问某个设备，常用的是挂载光驱 mount /dev/cdrom /mnt。
- /etc: 系统配置文件存放的目录，不建议在此目录下存放可执行文件，重要的配置文件有 /etc/inittab、/etc/fstab、/etc/init.d、/etc/X11、/etc/sysconfig、/etc/xinetd.d。
- /home: 系统默认的用户家目录，新增用户账号时，用户的家目录都存放在此目录下，~表示当前用户的家目录，~edu 表示用户 edu 的家目录。
- /lib: /usr/lib: /usr/local/lib: 系统使用的函数库的目录，程序在执行过程中，需要调用一些额外的参数时需要函数库的协助。
- /lost+fount: 系统异常产生错误时，会将一些遗失的片段放置于此目录下。
- /mnt: /media: 光盘默认挂载点，通常光盘挂载于 /mnt/cdrom 下，也不一定，可以选择任意位置进行挂载。
- /opt: 给主机额外安装软件所摆放的目录。
- /proc: 此目录的数据都在内存中，如系统核心，外部设备，网络状态，由于数据都存放于内存中，所以不占用磁盘空间，比较重要的目录有 /proc/cpuinfo、/proc/interrupts、/proc/dma、/proc/ioports、/proc/net/* 等。
- /root: 系统管理员root的家目录。
- /sbin: /usr/sbin: /usr/local/sbin: 放置系统管理员使用的可执行命令，如fdisk、shutdown、mount 等。与 /bin 不同的是，这几个目录是给系统管理员 root使用的命令，一般用户只能“查看”而不能设置和使用。
- /tmp: 一般用户或正在执行的程序临时存放文件的目录，任何人都可以访问，重要数据不可放置在此目录下。
- /srv: 服务启动之后需要访问的数据目录，如 www 服务需要访问的网页数据存放在 /srv/www 内。
- /usr: 应用程序存放目录，/usr/bin 存放应用程序，/usr/share 存放共享数据，/usr/lib 存放不能直接运行的，却是许多程序运行所必需的一些函数库文件。/usr/local: 存放软件升级包。/usr/share/doc: 系统说明文件存放目录。/usr/share/man: 程序说明文件存放目录。
- /var: 放置系统执行过程中经常变化的文件，如随时更改的日志文件 /var/log，/var/log/message: 所有的登录文件存放目录，/var/spool/mail: 邮件存放的目录，/var/run: 程序或服务启动后，其PID存放在该目录下。

用户目录

位于/home/user，称之为用户工作目录或家目录,表示方式：

```
1 | /home/user
2 | ~
```

从/目录开始描述的路径为**绝对路径**，如：

```
1 | cd /home
2 | ls /usr
```

从当前位置开始描述的路径为**相对路径**，如：

```
1 | cd ../../
2 | ls abc/def
```

每个目录下都有**.和..**

. 表示当前目录

.. 表示上一级目录，即父目录

根目录下的.和..都表示当前目录

文件的颜色	含义
蓝色	目录
绿色	可执行文件
红色	压缩文件
浅蓝色	链接文件
灰色	其他文件

命令行基本操作

命令使用方法

Linux命令格式：

```
1 | command [-options] [parameter1] ...
```

command: 命令名； [-options]: 选项,用来对命令进行控制，也可以省略， []代表可选 parameter1 ...: 传给命令的参数：可以是零个一个或多个

查看帮助文档

help

一般是linux命令自带的帮助信息

如：ls --help

man(manual)

man是linux提供的一个手册，包含了绝大部分的命令、函数使用说明
该手册分成很多章节（section），使用man时可以指定不同的章节来浏览。

例：man ls ; man 2 printf

man中各个section意义如下：

- 1. Standard commands（标准命令）
- 2. System calls（系统调用，如open,write）
- 3. Library functions（库函数，如printf,fopen）
- 4. Special devices（设备文件的说明，/dev下各种设备）
- 5. File formats（文件格式，如passwd）
- 6. Games and toys（游戏和娱乐）
- 7. Miscellaneous（杂项、惯例与协定等，例如Linux档案系统、网络协定、ASCII 码；environ全局变量）
- 8. Administrative Commands（管理员命令，如ifconfig）

man是按照手册的章节号的顺序进行搜索的。

man设置了如下的功能键：

功能键	功能
空格键	显示手册页的下一屏
Enter键	一次滚动手册页的一行
b	回滚一屏
f	前滚一屏
q	退出man命令
h	列出所有功能键
/word	搜索word字符串

```
wencong@sz:~$ man 1 ls
LS(1)                                User Commands                                LS(1)
NAME
  ls - list directory contents
SYNOPSIS
  ls [OPTION]... [FILE]...
DESCRIPTION
  List information about the FILES (the current directory by default).
  Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.
  Mandatory arguments to long options are mandatory for short options too.
  -a, --all
    do not ignore entries starting with .
  -A, --almost-all
    do not list implied . and ..
```

注意：实际上，我们不用指定第几个章节也用查看，如，man ls

tab键自动补全

在敲出命令的前几个字母的同时，按下tab键，系统会自动帮我们补全命令

history游览历史

当系统执行过一些命令后，可按上下键翻看以前的命令，history将执行过的命令列举出来

history保留了最近执行的命令记录，默认可以保留1000。

历史清单从0开始编号到最大值。

常见用法：

```
1 history N      显示最近N条命令
2 history -C      清除所有的历史记录
3 history -W xxx.txt 保存历史记录到文本xxx.txt
```

命令行中的ctrl组合键

- Ctrl+c 结束正在运行的程序
 - Ctrl+d 结束输入或退出shell
 - Ctrl+s 暂停屏幕输出【锁住终端】
 - Ctrl+q 恢复屏幕输出【解锁终端】
 - Ctrl+l 清屏，【是字母L的小写】等同于Clear
-
- 当前光标到行首：ctrl+a
 - 当前光标到行尾：ctrl+e
 - 删除当前光标到行首：ctrl+u
 - 删除当前光标到行尾：ctrl+k
-
- Ctrl+y 在光标处粘贴剪切的内容
 - Ctrl+r 查找历史命令【输入关键字，就能调出以前执行过的命令】
 - Ctrl+t 调换光标所在处与其之前字符位置，并把光标移到下个字符
 - Ctrl+x+u 撤销操作
 - Ctrl+z 转入后台运行

Linux命令

权限管理

列出目录的内容：ls

Linux文件或者目录名称最长可以有265个字符，“.”代表当前目录，“..”代表上一级目录，以“.”开头的文件为隐藏文件，需要用 -a 参数才能显示。

ls常用参数：

参数	含义
-a	显示指定目录下所有子目录与文件，包括隐藏文件
-l	以列表方式显示文件的详细信息
-h	配合 -l 以人性化的方式显示文件大小

```
1 [root@VM_0_9_centos ~]# ll -h
2 total 24K
3 -rw-r--r-- 1 root root 1.6K Dec  1 2016 CentOS7-Base-163.repo.1
4 -rw-r--r-- 1 root root 6.0K Nov 12 2015 mysql-community-release-el7-5.noarch.rpm
5 -rw-r--r-- 1 root root  90 Nov 23 10:26 passwd
6 drwxr-xr-t 2 root root 4.0K Nov 22 21:15 test
7 -rw-r--r-- 1 root root 276 Nov 24 10:01 user
8 lrwxrwxrwx 1 root root  14 Nov  6 19:18 web -> /var/www/html/
```

列出的信息的含义：



ls支持通配符：

通配符	含义
*	文件代表文件名中所有字符
ls te*	查找以te开头的文件
ls *html	查找结尾为html的文件
?	代表文件名中任意一个字符
ls ?.c	只找第一个字符任意，后缀为.c的文件
ls a.?	只找只有3个字符，前2字符为a.，最后一个字符任意的文件
[]	[“和”]将字符组括起来，表示可以匹配字符组中的任意一个。“-”用于表示字符范围。
[abc]	匹配a、b、c中的任意一个
[a-f]	匹配从a到f范围内的的任意一个字符
ls [a-f]*	找到从a到f范围内的的任意一个字符开头的文件
ls a-f	查找文件名为a-f的文件,当“-”处于方括号之外失去通配符的作用
\	如果要使通配符作为普通字符使用，可以在其前面加上转义字符。“?”和“*”处于方括号内时不用使用转义字符就失去通配符的作用。
ls *a	查找文件名为*a的文件

显示inode的内容: stat

```
1 | stat [文件或目录]
```

查看 testfile 文件的inode内容内容，可以用以下命令：

```
1 [root@VM_0_9_centos ~]# stat mysql-community-release-el7-5.noarch.rpm
2   File: 'mysql-community-release-el7-5.noarch.rpm'
3   Size: 6140          Blocks: 16          IO Block: 4096   regular file
4 Device: fd01h/64769d  Inode: 394398      Links: 1
5 Access: (0644/-rw-r--r--)  Uid: (    0/   root)   Gid: (    0/   root)
6 Access: 2019-11-23 13:30:42.000000000 +0800
7 Modify: 2015-11-12 15:58:42.000000000 +0800
8 Change: 2019-11-23 13:30:42.400300171 +0800
9  Birth: -
10
11 python@ubuntu:~/txt$ stat find我的Mr.Right.txt
12   文件: 'find我的Mr.Right.txt'
13   大小: 64768          块: 128          IO 块: 4096   普通文件
14 设备: 801h/2049d      Inode: 655501      硬链接: 1
15 权限: (0664/-rw-rw-r--)  Uid: ( 1000/  python)  Gid: ( 1000/  python)
16 最近访问: 2019-11-22 22:43:17.565105155 +0800
17 最近更改: 2019-11-22 22:42:42.000000000 +0800
18 最近改动: 2019-11-22 22:43:08.797116491 +0800
19 创建时间: -
```

文件访问权限

用户能够控制一个给定的文件或目录的访问程度，一个文件或目录可能有读、写及执行权限：

- 读权限 (r) ：对于文件，具有读取文件内容的权限；对于目录，具有浏览目录的权限。
- 写权限 (w) ：对于文件，具有修改文件内容的权限；对于目录，具有删除、移动目录内文件的权限。
- 可执行权限 (x) ：对于文件，具有执行文件的权限；对于目录，该用户具有进入目录的权限。

通常，Unix/Linux系统只允许文件的属主(所有者)或超级用户改变文件的读写权限。

示例：

```
wencong@sz:~/code$ ls -lh
总用量 8.0K
-rw-rw-r-- 1 wencong wencong 407  3月 17 19:53 demo.c
-rw-rw-r-- 1 wencong wencong 344  3月 17 19:53 tags
```

所有者

用户组

其他用户拥有的权限：可读

用户组拥有的权限：可读，可写

所有者拥有的权限：可读，可写

文件类型：普通文件

第1个字母代表文件的类型：

- “d” 代表文件夹
- “-” 代表普通文件
- “c” 代表硬件字符设备

- “b” 代表硬件块设备
- “s”表示管道文件
- “l” 代表软链接文件。

后9个字母分别代表三组权限：文件所有者、用户组、其他用户拥有的权限。

修改文件权限：chmod

chmod 修改文件权限有两种使用格式：字母法与数字法。

字母法：chmod u/g/o/a +/-/= rwx 文件

[u/g/o/a]	含义
u	user 表示该文件的所有者
g	group 表示与该文件的所有者属于同一组(group)者，即用户组
o	other 表示其他以外的人
a	all 表示这三者皆是

[+/-=]	含义
+	增加权限
-	撤销权限
=	设定权限

rwx	含义
r	read 表示可读取，对于一个目录，如果没有r权限，那么就意味着不能通过ls查看这个目录的内容。
w	write 表示可写入，对于一个目录，如果没有w权限，那么就意味着不能在目录下创建新的文件。
x	excute 表示可执行，对于一个目录，如果没有x权限，那么就意味着不能通过cd进入这个目录。

数字法：“rwx” 这些权限也可以用数字来代替

字母	说明
r	读取权限，数字代号为 "4"
w	写入权限，数字代号为 "2"
x	执行权限，数字代号为 "1"
-	不具任何权限，数字代号为 "0"

如执行：chmod u=rwx,g=rx,o=r filename 就等同于：chmod u=7,g=5,o=4 filename

chmod 751 file：

- 文件所有者：读、写、执行权限
- 同组用户：读、执行的权限
- 其它用户：执行的权限

chmod 777 file：所有用户拥有读、写、执行权限

注意：如果想递归所有目录加上相同权限，需要加上参数“-R”。如：chmod 777 test/ -R 递归 test 目录下所有文件加 777 权限

修改文件所有者：chown

```
1 python@ubuntu:~/test$ ll h.txt
2 -rw----- 1 python python 4 11月 22 22:35 h.txt
3 python@ubuntu:~/test$ chown mike h.txt
4 chown: 正在更改'h.txt' 的所有者：不允许的操作
5 python@ubuntu:~/test$ sudo chown mike h.txt
6 python@ubuntu:~/test$ ll h.txt
7 -rw----- 1 mike python 4 11月 22 22:35 h.txt
```

修改文件所属组：chgrp

```
1 python@ubuntu:~/test$ ll h.txt
2 -rw----- 1 mike python 4 11月 22 22:35 h.txt
3 python@ubuntu:~/test$ sudo chgrp mike h.txt
4 python@ubuntu:~/test$ ll h.txt
5 -rw----- 1 mike mike 4 11月 22 22:35 h.txt
```

文件内容查看

Linux系统中使用以下命令来查看文件的内容：

- cat 由第一行开始显示文件内容
- tac 从最后一行开始显示
- nl 显示的时候，顺道输出行号
- more 一页一页的显示文件内容
- less与more 类似，但可以往前翻页
- head 只看头几行
- tail 只看尾巴几行

基本显示：cat、tac

语法：

```
1 cat [-AbETv] 文件...
```

选项与参数：

- -A：相当于 -vET 的整合选项，可列出一些特殊字符而不是空白而已；
- -v：列出一些看不出来的特殊字符
- -E：将结尾的断行字节 \$ 显示出来；
- -T：将 [tab] 按键以 ^I 显示出来；
- -b：列出行号，空白行不标行号
- -n：列出行号，连同空白行也会有行号


```

1 [root@VM_0_9_centos ~]# cat -b /etc/issue
2     1  \S
3     2  Kernel \r on an \m
4 [root@VM_0_9_centos ~]# cat -n /etc/issue
5     1  \S
6     2  Kernel \r on an \m
7     3
8 [root@VM_0_9_centos ~]#

```

tac与cat命令刚好相反，文件内容从最后一行开始显示，可以看出 tac 是 cat 的倒着写！如：

```

1 [root@VM_0_9_centos ~]# tac /etc/issue
2 Kernel \r on an \m
3 \S

```

显示行号：nl

语法：

```

1 nl [-bnw] 文件

```

选项与参数：

- -b：指定行号指定的方式，主要有两种：
 - b a：表示不论是否为空行，也同样列出行号(类似 cat -n)；
 - b t：如果有空行，空的那一行不要列出行号(默认值)；
- -n：列出行号表示的方法，主要有三种：
 - n ln：行号在荧幕的最左方显示；
 - n rn：行号在自己栏位的最右方显示，且不加 0；
 - n rz：行号在自己栏位的最右方显示，且加 0；
- -w：行号栏位的占用的位数。

```

1 [root@www ~]# nl /etc/issue
2     1  CentOS release 6.4 (Final)
3     2  Kernel \r on an \m

```

分屏显示：more、less

```

1 [root@www ~]# more /etc/man_db.config
2 #
3 # Generated automatically from man.conf.in by the
4 # configure script.
5 #
6 # man.conf from man-1.6d
7 ....(中间省略)....
8 --More--(28%)  <== 光标在这里等待命令

```

more运行时可以输入的命令有：

- 空白键(space)：代表向下翻一页；
- Enter：代表向下翻『一行』；
- /字符串：代表在这个显示的内容当中，向下搜寻『字符串』这个关键字；
- :f：立刻显示出档名以及目前显示的行数；
- q：代表立刻离开 more，不再显示该文件内容。

- b 或 [ctrl]-b：代表往回翻页，不过这动作只对文件有用，对管线无用。

```
1 [root@www ~]# less /etc/man.config
2 #
3 # Generated automatically from man.conf.in by the
4 # configure script.
5 #
6 # man.conf from man-1.6d
7 ....(中间省略)....
8 :    <== 这里可以等待你输入命令！
```

less运行时可以输入的命令有：

- 空白键：向下翻动一页；
- [pagedown]：向下翻动一页；
- [pageup]：向上翻动一页；
- /字符串：向下搜寻『字符串』的功能；
- ?字符串：向上搜寻『字符串』的功能；
- n：重复前一个搜寻(与/或?有关！)
- N：反向的重复前一个搜寻(与/或?有关！)
- q：离开 less 这个程序；

取首尾n行：head、tail

head取出文件前面几行

语法：

```
1 head [-n number] 文件
```

选项与参数：

- -n：后面接数字，代表显示几行的意思

```
1 [root@www ~]# head /etc/man.config
```

默认的情况中，显示前面 10 行！若要显示前 20 行，就得要这样：

```
1 [root@www ~]# head -n 20 /etc/man.config
```

tail取出文件后面几行

语法：

```
1 tail [-n number] 文件
```

选项与参数：

- -n：后面接数字，代表显示几行的意思
- -f：表示持续侦测后面所接的档名，要等到按下[ctrl]-c才会结束tail的侦测

```
1 [root@www ~]# tail /etc/man.config
2 # 默认的情况中，显示最后的十行！若要显示最后的 20 行，就得要这样：
3 [root@www ~]# tail -n 20 /etc/man.config
```

文件管理

输出重定向：>

可将本应显示在终端上的内容保存到指定文件中。

如：ls > test.txt (test.txt 如果不存在，则创建，存在则覆盖其内容)

注意：>输出重定向会覆盖原来的内容，>>输出重定向则会追加到文件的尾部。

管道：|

管道：一个命令的输出可以通过管道做为另一个命令的输入。

“|”的左右分为两端，从左端写入到右端。

```
1 python@ubuntu:/bin$ ll -h |more
2 总用量 13M
3 drwxr-xr-x  2 root root  4.0K 8月  4 2016 ./
4 drwxr-xr-x 26 root root  4.0K 7月 30 2016 ../
5 -rwxr-xr-x  1 root root 1014K 6月 24 2016 bash*
6 -rwxr-xr-x  1 root root   31K 5月 20 2015 bunzip2*
7 -rwxr-xr-x  1 root root  1.9M 8月 19 2015 busybox*
8 -rwxr-xr-x  1 root root   31K 5月 20 2015 bzip2*
9 lrwxrwxrwx  1 root root    6 5月 16 2016 bzip2 -> bzip2diff*
10 -rwxr-xr-x  1 root root   2.1K 5月 20 2015 bzip2diff*
11 lrwxrwxrwx  1 root root    6 5月 16 2016 bzip2grep -> bzip2grep*
12 --更多--
```

清屏：clear

clear作用为清除终端上的显示(类似于DOS的cls清屏功能)，快捷键：Ctrl + l (“l” 为字母)。

切换工作目录：cd

Linux所有的目录和文件名大小写敏感

cd后面可跟绝对路径，也可以跟相对路径。如果省略目录，则默认切换到当前用户的主目录。

命令	含义
cd	相当于cd ~
cd ~	切换到当前用户的主目录(/home/用户目录)
cd .	切换到当前目录
cd ..	切换到上级目录
cd -	进入上次所在的目录

显示当前路径：pwd

```
1 python@ubuntu:~$ pwd
2 /home/python
```

选项与参数：

- **-P**：显示出确实的路径，而非使用连结 (link) 路径。

```
1 [root@VM_0_9_centos ~]# cd /var/mail/
2 [root@VM_0_9_centos mail]# pwd
3 /var/mail
4 [root@VM_0_9_centos mail]# pwd -P
5 /var/spool/mail
```

创建目录：mkdir

mkdir可以创建一个新的目录。

注意：新建目录的名称不能与当前目录中已有的目录或文件同名，并且目录创建者必须对当前目录具有写权限。

语法：

```
1 mkdir [-mp] 目录名称
```

选项与参数：

- **-m**：指定被创建目录的权限，而不是根据默认权限 (umask) 设定
- **-p**：递归创建所需要的目录

实例：**-p**递归创建目录：

```
1 [root@www ~]# cd /tmp
2 [root@www tmp]# mkdir test    <==创建一名为 test 的新目录
3 [root@www tmp]# mkdir test1/test2/test3/test4
4 mkdir: cannot create directory `test1/test2/test3/test4':
5 No such file or directory    <== 没办法直接创建此目录啊！
6 [root@www tmp]# mkdir -p test1/test2/test3/test4
```

mkdir创建的目录权限默认根据umask得到，而**-m**参数可以指定被创建目录的权限：

```
1 [root@VM_0_9_centos ~]# mkdir t1
2 [root@VM_0_9_centos ~]# ll
3 drwxr-xr-x 2 root root 4096 Nov 22 18:54 t1
4 [root@VM_0_9_centos ~]# mkdir t2 -m 711
5 [root@VM_0_9_centos ~]# ll
6 drwxr-xr-x 2 root root 4096 Nov 22 18:54 t1
7 drwx--x--x 2 root root 4096 Nov 22 18:54 t2
```

删除文件：rm

可通过rm删除文件或目录。使用rm命令要小心，因为文件删除后不能恢复。为了防止文件误删，可以在rm后使用**-i**参数以逐个确认要删除的文件。

常用参数及含义如下表所示：

参数	含义
-i	以进行交互式方式执行
-f	强制删除，忽略不存在的文件，无需提示
-r	递归地删除目录下的内容，删除文件夹时必须加此参数

建立链接文件：ln

软链接：ln -s 源文件 链接文件

硬链接：ln 源文件 链接文件

软链接类似于Windows下的快捷方式，如果软链接文件和源文件不在同一个目录，源文件要使用绝对路径，不能使用相对路径。

硬链接只能链接普通文件不能链接目录。两个文件占用相同大小的硬盘空间，即使删除了源文件，链接文件还是存在，所以-s选项是更常见的形式。

文本搜索：grep

Linux系统中grep命令是一种强大的文本搜索工具，grep允许对文本文件进行模式查找。如果找到匹配模式，grep打印包含模式的所有行。

grep一般格式为：

```
1 | grep [-选项] '搜索内容串' 文件名
```

在grep命令中输入字符串参数时，最好引号或双引号括起来。例如：`grep 'a' 1.txt`。

在当前目录中，查找前缀有test字样的文件中包含 test 字符串的文件，并打印出该字符串的行。此时，可以使用如下命令：

```
1 | $ grep test test* #查找前缀有test的文件包含test字符串的文件
2 | testfile1:This a Linux testfile! #列出testfile1 文件中包含test字符的行
3 | testfile_2:This is a linux testfile! #列出testfile_2 文件中包含test字符的行
4 | testfile_2:Linux test #列出testfile_2 文件中包含test字符的行
```

以递归的方式查找符合条件的文件。例如，查找指定目录/etc/acpi 及其子目录（如果存在子目录的话）下所有文件中包含字符串"update"的文件，并打印出该字符串所在行的内容，使用的命令为：

```
1 | $ grep -r update /etc/acpi #以递归的方式查找“etc/acpi”
2 | #下包含“update”的文件
3 | /etc/acpi/ac.d/85-anacron.sh:# (Things like the slocate updatedb cause a lot of IO.)
4 | Rather than
5 | /etc/acpi/resume.d/85-anacron.sh:# (Things like the slocate updatedb cause a lot of
6 | IO.) Rather than
7 | /etc/acpi/events/thinkpad-cmos:action=/usr/sbin/thinkpad-keys--update
```

反向查找。前面各个例子是查找并打印出符合条件的行，通过"-v"参数可以打印出不符合条件行的内容。

查找文件名中包含 test 的文件中不包含test 的行，此时，使用的命令为：

```
1 | $ grep -v test* #查找文件名中包含test 的文件中不包含test 的行
2 | testfile1:helLinux!
3 | testfile1:Linis a free Unix-type operating system.
4 | testfile1:Lin
5 | testfile_1:HELLO LINUX!
6 | testfile_1:LINUX IS A FREE UNIX-TYPE OPTERATING SYSTEM.
7 | testfile_1:THIS IS A LINUX TESTFILE!
8 | testfile_2:HELLO LINUX!
9 | testfile_2:Linux is a free unix-type opterating system.
```

查找文件：find

常用用法：

命令	含义
find ./ -name test.sh	查找当前目录下所有名为test.sh的文件
find ./ -name '*.sh'	查找当前目录下所有后缀为.sh的文件
find ./ -name "[A-Z]*"	查找当前目录下所有以大写字母开头的文件
find /tmp -size 2M	查找在/tmp 目录下等于2M的文件
find /tmp -size +2M	查找在/tmp 目录下大于2M的文件
find /tmp -size -2M	查找在/tmp 目录下小于2M的文件
find ./ -size +4k -size -5M	查找当前目录下大于4k，小于5M的文件
find ./ -perm 0777	查找当前目录下权限为 777 的文件或目录

Linux find命令用来在指定目录下查找文件。任何位于参数之前的字符串都将被视为欲查找的目录名。如果使用该命令时，不设置任何参数，则find命令将在当前目录下查找子目录与文件。并且将查找到的子目录和文件全部进行显示。

语法：

```
1 | find path -option [ -print ] [ -exec -ok command ] {} \;
```

常用参数说明：

- -perm xxxx：权限为 xxxx的文件或目录
- -user：按照文件属主来查找文件。
- -size n：n单位,b:512位元组的区块,c:字数,c:kilo bytes,w:二个位元组
- -mount, -xdev：只检查和指定目录在同一个文件系统下的文件，避免列出其它文件系统下的文件
- -amin n：在过去 n 分钟内被读取过
- -anewer file：比文件 file 更晚被读取过的文件
- -atime n：在过去n天内被读取过的文件
- -cmin n：在过去 n 分钟内被修改过
- -cnewer file：比文件 file 更新的文件
- -ctime n：在过去n天内被修改过的文件
- -empty：空的文件
- -gid n or -group name：gid 是 n 或是 group 名称是 name
- -ipath p, -path p：路径名称符合 p 的文件，ipath 会忽略大小写
- -name name, -iname name：文件名称符合 name 的文件。iname 会忽略大小写
- -type 查找某一类型的文件：
 - b - 块设备文件
 - d - 目录
 - c - 字符设备文件
 - p - 管道文件

- l - 符号链接文件
- f - 普通文件
- -exec 命令名{} \ (注意: "}"和"\之间有空格)

find实例:

显示当前目录中大于20字节并以.c结尾的文件名

```
1 | find . -name "*.c" -size +20c
```

将当前目录及其子目录中所有普通文件列出

```
1 | find . -type f
```

将当前目录及其子目录下所有最近 20 天内更新过的文件列出

```
1 | find . -ctime -20
```

查找/var/log目录中更改时间在7日以前的普通文件，并在删除之前询问它们:

```
1 | find /var/log -type f -mtime +7 -ok rm {} \;
```

查找前目录中文件属主具有读、写权限，并且文件所属组的用户和其他用户具有读权限的文件:

```
1 | find . -type f -perm 644 -exec ls -l {} \;
```

查找系统中所有文件长度为0的普通文件，并列出它们的完整路径:

```
1 | find / -type f -size 0 -exec ls -l {} \;
```

从根目录查找类型为符号链接的文件，并将其删除:

```
1 | find / -type l -exec rm -rf {} \
```

从当前目录查找用户tom的所有文件并显示在屏幕上

```
1 | find . -user tom
```

在当前目录中查找所有文件以.doc结尾，且更改时间在3天以上的文件，找到后删除，并且给出删除提示

```
1 | find . -name *.doc -mtime +3 -ok rm {} \;
```

在当前目录下查找所有链接文件，并且以长格式显示文件的基本信息

```
1 | find . -type l -exec ls -l {} \;
```

在当前目录下查找文件名有一个小写字母、一个大写字母、两个数字组成，且扩展名为.doc的文件

```
1 | find . -name '[a-z][A-Z][0-9][0-9].doc'
```

拷贝文件：cp

cp命令的功能是将给出的文件或目录复制到另一个文件或目录中，相当于DOS下的copy命令。

常用选项说明：

选项	含义
-a	该选项通常在复制目录时使用，它保留链接、文件属性，并递归地复制目录，简单而言，保持文件原有属性。
-f	已经存在的目标文件而不提示
-i	交互式复制，在覆盖目标文件之前将给出提示要求用户确认
-r	若给出的源文件是目录文件，则cp将递归复制该目录下的所有子目录和文件，目标文件必须为一个目录名。
-v	显示拷贝进度
-l	创建硬链接(hard link)，而非复制文件本身
-s	复制成为符号链接 (symbolic link)，相当于批量创建快捷方式
-u	若 destination 比 source 旧才升级 destination ！

cp vim_configure/ code/ -ivr 把文件夹 vim_configure 拷贝到 code 目录里。

移动文件：mv

mv命令用来移动文件或目录，也可以给文件或目录重命名。

常用选项说明：

选项	含义
-f	禁止交互式操作，如有覆盖也不会给出提示
-i	确认交互方式操作，如果mv操作将导致对已存在的目标文件的覆盖，系统会询问是否重写，要求用户回答以避免误覆盖文件
-v	显示移动进度

归档管理：tar

此命令可以把一系列文件归档到一个大文件中，也可以把档案文件解开以恢复数据。

tar使用格式 tar [参数] 打包文件名 文件

tar命令参数很特殊，其参数前面可以使用“-”，也可以不使用。

常用参数：

参数	含义
-c	生成档案文件，创建打包文件
-v	列出归档解档的详细过程，显示进度
-f	指定档案文件名称，f后面一定是.tar文件，所以必须放选项最后
-t	列出档案中包含的文件
-x	解开档案文件

注意：除了f需要放在参数的最后，其它参数的顺序任意。

```
1 python@ubuntu:~/test$ tar -cvf test.tar 1.txt 2.txt 3.txt
2 1.txt
3 2.txt
4 3.txt
5 python@ubuntu:~/test$ ll
6 总用量 32
7 drwxrwxr-x  2 python python  4096 11月 21 14:02 ./
8 drwxr-xr-x 31 python python  4096 11月 21 13:34 ../
9 -rw-rw-r--  1 python python   51 1月 20 2017 1.txt
10 -rw-rw-r--  1 python python   55 1月 20 2017 2.txt
11 -rw-rw-r--  1 python python   51 1月 20 2017 3.txt
12 -rw-rw-r--  1 python python 10240 11月 21 14:02 test.tar
13 python@ubuntu:~/test$ rm -rf *.txt
14 python@ubuntu:~/test$ ll
15 总用量 20
16 drwxrwxr-x  2 python python  4096 11月 21 14:03 ./
17 drwxr-xr-x 31 python python  4096 11月 21 13:34 ../
18 -rw-rw-r--  1 python python 10240 11月 21 14:02 test.tar
19 python@ubuntu:~/test$ tar -xvf test.tar
20 1.txt
21 2.txt
22 3.txt
23 python@ubuntu:~/test$ ls *.txt
24 1.txt 2.txt 3.txt
```

文件压缩解压：gzip、bzip2

tar与gzip命令结合使用实现文件打包、压缩。tar只负责打包文件，但不压缩，用gzip压缩tar打包后的文件，其扩展名一般用xxxx.tar.gz。

gzip使用格式如下：

```
1 | gzip [选项] 被压缩文件
```

常用选项：

选项	含义
-d	解压文件
-r	压缩文件

```

1 python@ubuntu:~/test$ ll *.tar*
2 -rw-rw-r-- 1 python python 10240 11月 25 07:39 test.tar
3 python@ubuntu:~/test$ gzip -r test.tar test.tar.gz ==>或者:gzip test.tar
4 python@ubuntu:~/test$ ll *.tar*
5 -rw-rw-r-- 1 python python 139 11月 25 07:39 test.tar.gz
6 python@ubuntu:~/test$ gzip -d test.tar.gz
7 python@ubuntu:~/test$ ll *.tar*
8 -rw-rw-r-- 1 python python 10240 11月 25 07:39 test.tar

```

tar命令中-z选项可以调用gzip实现了一个压缩的功能，实行一个先打包后压缩的过程。

压缩用法: tar zcvf 压缩包名 文件1 文件2 ...

例如: tar zcvf test.tar.gz 1.c 2.c 3.c 4.c把 1.c 2.c 3.c 4.c 压缩成 test.tar.gz

```

1 python@ubuntu:~/test/code$ ls
2 1.c 2.c 3.c 4.c
3 python@ubuntu:~/test/code$ tar zcvf test.tar.gz 1.c 2.c 3.c 4.c
4
5 python@ubuntu:~/test/code$ ls
6 1.c 2.c 3.c 4.c test.tar.gz
7 python@ubuntu:~/test/code$ tar -zcvf new.tar.gz 1.c 2.c 3.c 4.c
8
9 python@ubuntu:~/test/code$ ls
10 1.c 2.c 3.c 4.c new.tar.gz test.tar.gz

```

解压用法: tar zxvf 压缩包名

例如:

```

1 python@ubuntu:~/test/code$ ls
2 new.tar.gz test.tar.gz
3 python@ubuntu:~/test/code$ tar zxvf new.tar.gz
4 1.c
5 2.c
6 3.c
7 4.c
8 python@ubuntu:~/test/code$ ls
9 1.c 2.c 3.c 4.c new.tar.gz test.tar.gz

```

解压到指定目录: -C (解压时可以不指定-z选项)

```

1 python@ubuntu:~/test/code$ ls number/
2 python@ubuntu:~/test/code$ tar xvf test.tar.gz -C number/
3
4 python@ubuntu:~/test/code$ ls number/
5 1.c 2.c 3.c 4.c

```

bzip2命令跟gzip用法类似

压缩用法: tar jcvf 压缩包名 文件...(tar jcvf bk.tar.bz2 *.c)

解压用法: tar jxvf 压缩包名 (tar jxvf bk.tar.bz2)

文件压缩解压：zip、unzip

通过zip压缩文件的目标文件不需要指定扩展名，默认扩展名为zip。

压缩文件：zip [-r] 目标文件(没有扩展名) 源文件

解压文件：unzip -d 解压后目录文件 压缩文件

```
1 python@ubuntu:~/test$ ls
2 1.txt 2.txt 3.txt test.tar
3 python@ubuntu:~/test$ zip myzip *.txt
4   adding: 1.txt (stored 0%)
5   adding: 2.txt (stored 0%)
6   adding: 3.txt (stored 0%)
7 python@ubuntu:~/test$ ls
8 1.txt 2.txt 3.txt myzip.zip test.tar
9 python@ubuntu:~/test$ rm -f *.txt *.tar
10 python@ubuntu:~/test$ ls
11 myzip.zip
12 python@ubuntu:~/test$ unzip myzip.zip
13 Archive: myzip.zip
14   extracting: 1.txt
15   extracting: 2.txt
16   extracting: 3.txt
17 python@ubuntu:~/test$ ls
18 1.txt 2.txt 3.txt myzip.zip
19 python@ubuntu:~/test$ unzip -d dir myzip.zip
20 Archive: myzip.zip
21   extracting: dir/1.txt
22   extracting: dir/2.txt
23   extracting: dir/3.txt
24 python@ubuntu:~/test$ ls
25 1.txt 2.txt 3.txt dir myzip.zip
```

查看命令位置：which

```
1 python@ubuntu:~$ which ls
2 /bin/ls
3 python@ubuntu:~$ which sudo
4 /usr/bin/sudo
```

用户和用户组管理

用户管理包括用户与组账号的管理。

在Unix/Linux系统中，不论是由本机或是远程登录系统，每个系统都必须拥有一个账号，并且对于不同的系统资源拥有不同的使用权限。

Unix/Linux系统中的root账号通常用于系统的维护和管理，它对Unix/Linux操作系统的所有部分具有不受限制的访问权限。

在Unix/Linux安装的过程中，系统会自动创建许多用户账号，而这些默认的用户就称为“标准用户”。

在大多数版本的Unix/Linux中，都不推荐直接使用root账号登录系统。

查看当前用户：whoami

查看当前系统当前账号的用户名。可通过cat /etc/passwd查看系统用户信息。

```
1 python@ubuntu:~/test$ whoami
2 python
```

查看登录用户：who

who命令用于查看当前所有登录系统的用户信息。

常用选项：

选项	含义
-m或am l	只显示运行who命令的用户名、登录终端和登录时间
-q或--count	只显示用户的登录账号和登录用户的数量
-u	在登录时间后显示该用户最后一次操作到当前的时间间隔
-u或--heading	显示列标题

退出登录账户：exit

如果是图形界面，退出当前终端；

如果是使用ssh远程登录，退出登陆账户；

如果是切换后的登陆用户，退出则返回上一个登陆账号。

添加用户账号：useradd

在Unix/Linux中添加用户账号可以使用adduser或useradd命令，因为adduser命令是指向useradd命令的一个链接，因此，这两个命令的使用格式完全一样。

useradd命令的使用格式如下： useradd [参数] 新建用户账号

参数	含义
-d	指定用户登录系统时的主目录
-m	自动建立目录，未指定-d参数时会在/home/{当前用户}目录下建立主目录
-g	指定组名称

相关说明：

- Linux每个用户都要有一个主目录，主目录就是第一次登陆系统，用户的默认当前目录(/home/用户)；
- 每一个用户必须有一个主目录，所以用useradd创建用户的时候，一定给用户指定一个主目录；
- 如果创建用户的时候，不指定组名，那么系统会自动创建一个和用户名一样的组名。

若创建用户时未指定家目录，后期可通过 usermod -d /home/abc abc 指定

命令	含义
useradd -d /home/abc abc -m	创建abc用户，如果/home/abc目录不存在，就自动创建这个目录，同时用户属于abc组
useradd -d /home/a a -g test -m	创建一个用户名字叫a，主目录在/home/a，如果主目录不存在，就自动创建主目录，同时用户属于test组
cat /etc/passwd	查看系统当前用户名

修改用户：usermod

常用的选项包括 `-c`，`-d`，`-m`，`-g`，`-G`，`-s`，`-u`以及`-o`等，这些选项的意义与 `useradd` 命令中的选项一样，可以为用户指定新的资源值。

修改用户所在组：`usermod -g 用户组 用户名`

```
1 | usermod -g test abc
```

改abc用户的家目录位置：`usermod -d 家目录 用户名`

```
1 | usermod -d /home/abc abc
```

选项 `-l` 新用户名 指定一个新的账号，可修改用户名：

```
1 | python@ubuntu:~/txt$ tail /etc/passwd -n 1
2 | aaa:x:1001:1001::/home/aaa:
3 | python@ubuntu:~/txt$ sudo usermod -l bbb -d /home/bbb aaa
4 | python@ubuntu:~/txt$ tail /etc/passwd -n 1
5 | bbb:x:1001:1001::/home/bbb:
```

设置用户密码：passwd

超级用户可以为自己和其他用户指定口令，普通用户只能用它修改自己的口令。命令的格式为：

```
1 | passwd 选项 用户名
```

可使用的选项：

- `-l` 锁定口令，即禁用账号。
- `-u` 口令解锁。
- `-d` 使账号无口令。
- `-f` 强迫用户下次登录时修改口令。

假设当前用户是sam，则下面的命令修改该用户自己的口令：

```
1 | $ passwd
2 | Old password:*****
3 | New password:*****
4 | Re-enter new password:*****
```

如果是超级用户，可以用下列形式指定任何用户的口令：

```
1 # passwd sam
2 New password:*****
3 Re-enter new password:*****
```

普通用户修改自己的口令时，passwd命令会先询问原口令，验证后再要求用户输入两遍新口令，如果两次输入的口令一致，则将这个口令指定给用户；而超级用户为用户指定口令时，就不需要知道原口令。

为用户指定空口令时，执行下列形式的命令：

```
1 | passwd -d sam
```

此命令将用户 sam 的口令删除，这样用户 sam 下一次登录时，系统就不再允许该用户登录了。

passwd 命令还可以用 -l(lock) 选项锁定某一用户，使其不能登录，例如：

```
1 | passwd -l sam
```

删除用户：userdel

命令	含义
userdel abc(用户名)	删除abc用户，但不会自动删除用户的主目录
userdel -r abc(用户名)	删除用户，同时删除用户的主目录

切换用户：su

su后面可以加“-”会将当前的工作目录自动转换到切换后的用户主目录。

命令	含义
su	切换到root用户
su root	切换到root用户
su -	切换到root用户，同时切换目录到/root
su - root	切换到root用户，同时切换目录到/root
su 普通用户	切换到普通用户
su - 普通用户	切换到普通用户，同时切换普通用户所在的目录

注意：对于ubuntu平台，只能通过sudo su进入root账号。

sudo允许系统管理员让普通用户执行一些或者全部的root命令的一个工具。

以root身份执行指令：sudo

sudo命令可以临时获取root权限

使用权限：在 /etc/sudoers 中有出现的使用者。

```
1 | 显示出自己（执行 sudo 的使用者）的权限
2 | sudo -l
3 | 以root权限执行上一条命令
4 | sudo !!
```

sudoers文件配置语法

```
1 | user MACHINE=COMMANDS
2 | 用户 登录的主机=（可以变换的身份） 可以执行的命令
```

例子：

```
1 | 允许root用户执行任意路径下的任意命令
2 | root ALL=(ALL) ALL
3 | 允许wheel用户组中的用户执行所有命令
4 | %wheel ALL=(ALL) ALL
5 | 允许wheel用户组中的用户在不输入该用户的密码的情况下使用所有命令
6 | %wheel ALL=(ALL) NOPASSWD: ALL
7 | 允许support用户在EPG的机器上不输入密码的情况下使用SQUID中的命令
8 | Cmnd_Alias SQUID = /opt/vtbin/squid_refresh, /sbin/service, /bin/rm
9 | Host_Alias EPG = 192.168.1.1, 192.168.1.2
10 | support EPG=(ALL) NOPASSWD: SQUID
```

添加、删除组账号：groupadd、groupdel

groupadd 新建组账号 groupdel 组账号 cat /etc/group 查看用户组

```
1 | python@ubuntu:~/test$ sudo groupadd abc
2 | python@ubuntu:~/test$ sudo groupdel abc
3 | python@ubuntu:~/test$ sudo groupdel abc
4 | groupdel: "abc"组不存在
```

用户组管理：groupmod

修改用户组的属性使用groupmod命令。其语法如下：

```
1 | groupmod 选项 用户组
```

常用的选项有：

- -g GID 为用户组指定新的组标识号。
- -o 与-g选项同时使用，用户组的新GID可以与系统已有用户组的GID相同。
- -n新用户组 将用户组的名字改为新名字

将组group2的组标识号修改为102：

```
1 | groupmod -g 102 group2
```

将组group2的标识号改为10000，组名修改为group3：

```
1 | groupmod -g 10000 -n group3 group2
```

如果一个用户同时属于多个用户组，那么用户可以在用户组之间切换，以便具有其他用户组的权限。

用户可以在登录后，使用命令newgrp切换到其他用户组，这个命令的参数就是目的用户组。例如：

```
1 | $ newgrp root
```

这条命令将当前用户切换到root用户组，前提条件是root用户组确实是该用户的主组或附加组。类似于用户账号的管理，用户组的管理也可以通过集成的系统管理工具来完成。

系统管理

查看当前日历：cal

cal命令用于查看当前日历，-y显示整年日历：

```
1 | python@ubuntu:~$ cal
2 |      十一月 2019
3 | 日 一 二 三 四 五 六
4 |           1  2
5 |  3  4  5  6  7  8  9
6 | 10 11 12 13 14 15 16
7 | 17 18 19 20 21 22 23
8 | 24 25 26 27 28 29 30
```

显示或设置时间：date

设置时间格式（需要管理员权限）：

```
1 | date [MMDDhhmm[[CC]YY][.ss]] +format
```

MM为月，DD为天，hh为小时，mm为分钟；CC为年前两位，YY为年的后两位，ss为秒。

如：date 010203042016.55。

显示时间格式（date '+%y,%m,%d,%H,%M,%S'）：

format格式	含义
%Y, %y	年
%m	月
%d	日
%H	时
%M	分
%S	秒

查看网络状态：netstat

netstat命令用于显示网络状态。

利用netstat指令可让你得知整个Linux系统的网络情况。

语法：

```
1 | netstat [-acCeFghilMnNoprstuvVwx] [-A<网络类型>] [--ip]
```

参数说明：

- -a或--all 显示所有连线中的Socket。
- -A<网络类型>或--<网络类型> 列出该网络类型连线中的相关地址。
- -c或--continuous 持续列出网络状态。
- -C或--cache 显示路由器配置的快取信息。
- -e或--extend 显示网络其他相关信息。
- -F或--fib 显示FIB。
- -g或--groups 显示多重广播功能群组组员名单。
- -h或--help 在线帮助。
- -i或--interfaces 显示网络界面信息表单。
- -l或--listening 显示监控中的服务器的Socket。
- -M或--masquerade 显示伪装的网络连线。
- -n或--numeric 直接使用IP地址，而不通过域名服务器。
- -N或--netlink或--symbolic 显示网络硬件外围设备的符号连接名称。
- -o或--timers 显示计时器。
- -p或--programs 显示正在使用Socket的程序识别码和程序名称。
- -r或--route 显示Routing Table。
- -s或--statistic 显示网络工作信息统计表。
- -t或--tcp 显示TCP传输协议的连线状况。
- -u或--udp 显示UDP传输协议的连线状况。
- -v或--verbose 显示指令执行过程。
- -V或--version 显示版本信息。
- -w或--raw 显示RAW传输协议的连线状况。
- -x或--unix 此参数的效果和指定"-A unix"参数相同。
- --ip或--inet 此参数的效果和指定"-A inet"参数相同。

常用：

```
1 | [root@VM_0_9_centos ~]# netstat -nltp
2 | Active Internet connections (only servers)
3 | Proto Recv-Q Send-Q Local Address   Foreign Address State    PID/Program name
4 | tcp        0      0 0.0.0.0:22        0.0.0.0:*        LISTEN  3308/sshd
5 | tcp6       0      0 :::80           :::*             LISTEN  4624/httpd
```

查看进程信息：ps

进程是一个具有一定独立功能的程序，它是操作系统动态执行的基本单元。

ps命令选项：

- ps a 显示现行终端机下的所有程序，包括其他用户的程序。
- ps -A 显示所有程序。
- ps c 列出程序时，显示每个程序真正的指令名称，而不包含路径，参数或常驻服务的标示。
- ps -e 此参数的效果和指定"A"参数相同。

- `ps e` 列出程序时，显示每个程序所使用的环境变量。
- `ps f` 用ASCII字符显示树状结构，表达程序间的相互关系。
- `ps -H` 显示树状结构，表示程序间的相互关系。
- `ps -N` 显示所有的程序，除了执行ps指令终端机下的程序之外。
- `ps s` 采用程序信号的格式显示程序状况。
- `ps u` 以用户为主的格式来显示程序状况。
- `ps x` 显示所有程序，不以终端机来区分。

选项	含义
-a	显示终端上的所有进程，包括其他用户的进程
-u	显示进程的详细状态
-x	显示没有控制终端的进程
-w	显示加宽，以便显示更多的信息
-r	只显示正在运行的进程

常见用法：

- `ps -e` 查看所有进程信息（瞬时的）
- `ps -u root -N` 查看所有不是root运行的进程
- `ps ax` 显示所有进程状态状态
- `ps -ef |grep xxx` 显示含有xxx的进程

实例：

```

1 # ps -A 显示进程信息
2 PID TTY          TIME CMD
3   1 ?        00:00:02 init
4   2 ?        00:00:00 kthreadd
5 .....省略部分结果
6 30749 pts/0    00:00:15 gedit
7 30886 ?        00:01:10 qtcreator.bin
8 30894 ?        00:00:00 qtcreator.bin

```

显示指定用户信息：

```

1 # ps -u root //显示root进程用户信息
2 PID TTY          TIME CMD
3   1 ?        00:00:02 init
4   2 ?        00:00:00 kthreadd
5   3 ?        00:00:00 migration/0
6 .....省略部分结果
7 30487 ?        00:00:06 gnome-terminal
8 30488 ?        00:00:00 gnome-pty-helpe
9 30489 pts/0    00:00:00 bash

```

显示所有进程信息，连同命令行

```

1 # ps -ef //显示所有命令，连带命令行
2 UID      PID PPID C STIME TTY      TIME CMD
3 root      1   0  0 10:22 ?        00:00:02 /sbin/init
4 root      2   0  0 10:22 ?        00:00:00 [kthreadd]
5 root      3   2  0 10:22 ?        00:00:00 [migration/0]
6 root      4   2  0 10:22 ?        00:00:00 [ksoftirqd/0]
7 root      5   2  0 10:22 ?        00:00:00 [watchdog/0]
8 root      6   2  0 10:22 ?        /usr/lib/NetworkManager
9 .....省略部分结果
10 root    31302 2095  0 17:42 ?        00:00:00 sshd: root@pts/2
11 root    31374 31302  0 17:42 pts/2    00:00:00 -bash
12 root    31400   1  0 17:46 ?        00:00:00 /usr/bin/python /usr/sbin/aptd
13 root    31407 31374  0 17:48 pts/2    00:00:00 ps -ef

```

以树状图显示进程关系：pstree

显示进程的关系

```

1 pstree
2 init--amd
3 |-apmd
4 |-atd
5 |-httpd---10*[httpd]
6 %pstree -p
7 init(1)--amd(447)
8 |-apmd(105)
9 |-atd(339)
10 %pstree -c
11 init--amd
12 |-apmd
13 |-atd
14 |-httpd--httpd
15 | |-httpd
16 | |-httpd
17 | |-httpd
18 ....

```

特别表明在运行的进程：

```

1 # pstree -apnh //显示进程间的关系

```

同时显示用户名称：

```

1 # pstree -u //显示用户名称

```

动态显示进程：top

top命令用来动态显示运行中的进程。top命令能够在运行后，在指定的时间间隔更新显示信息。-d参数可以指定显示信息更新的时间间隔。

在top命令执行后，可以按下按键得到对显示的结果进行排序：

按键	含义
M	根据内存使用量来排序
P	根据CPU占有率来排序
T	根据进程运行时间的长短来排序
U	可以根据后面输入的用户名来筛选进程
K	可以根据后面输入的PID来杀死进程。
q	退出
h	获得帮助

```
1 python@ubuntu:~$ top
2 top - 08:31:54 up 2 min, 1 user, load average: 0.25, 0.37, 0.17
3 Tasks: 271 total, 1 running, 270 sleeping, 0 stopped, 0 zombie
4 %Cpu(s): 0.2 us, 0.0 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
5 KiB Mem : 4028880 total, 3210104 free, 331668 used, 487108 buff/cache
6 KiB Swap: 4192252 total, 4192252 free, 0 used. 3414856 avail Mem
7
8      PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
9      4827 redis      20   0   47204   6632   2300 S   0.3   0.2   0:00.21 redis-ser+
10     6371 python    20   0   49000   3896   3176 R   0.3   0.1   0:00.12 top
11         1 root      20   0  119940   6112   4004 S   0.0   0.2   0:02.77 systemd
12         2 root      20   0         0        0        0 S   0.0   0.0   0:00.02 kthreadd
13         3 root      20   0         0        0        0 S   0.0   0.0   0:00.11 ksoftirqd+
```

更高级的命令是htop，但需要安装：

```
1 [root@VM_0_9_centos ~]# htop
2
3      CPU[|] 1.3% Tasks:
4      55, 29 thr; 1 running
5      Mem[|||||||||||||||||||||||||||||||||||||||||184M/1.80G] Load
6      average: 0.00 0.01 0.05
7      Swp[ 0K/0K] Uptime:
8      42 days, 03:29:48
9
10     PID USER      PRI  NI  VIRT   RES   SHR S  CPU% MEM%   TIME+  Command
11     2609 root      20   0  156M   9132  1784 S   0.7   0.5  39:01.13 barad_agent
12     2610 root      20   0  660M  14168  1976 S   0.7   0.8  3h35:20 barad_agent
13     1158 root      20   0   120M   2576  1492 R   0.0   0.1   0:00.01 htop
```

终止进程：kill

kill命令指定进程号的进程，需要配合 ps 使用。

使用格式：

```
1 kill [-signal] pid
```

信号值从0到15，其中9为绝对终止，可以处理一般信号无法终止的进程。

关机重启：reboot、shutdown、init

命令	含义
reboot	重新启动操作系统
shutdown -r now	重新启动操作系统，shutdown会给别的用户提示
shutdown -h now	立刻关机，其中now相当于时间为0的状态
shutdown -h 20:25	系统在今天的20:25 会关机
shutdown -h +10	系统再过十分钟后自动关机
init 0	关机
init 6	重启

检测磁盘空间：df

df命令用于检测文件系统的磁盘空间占用和空余情况，可以显示所有文件系统对节点和磁盘块的使用情况。

选项	含义
-a	显示所有文件系统的磁盘使用情况
-m	以1024字节为单位显示
-t	显示各指定文件系统的磁盘空间使用情况
-T	显示文件系统

```
1 python@ubuntu:~/test$ df -Th
2 文件系统      类型      容量  已用  可用  已用% 挂载点
3 udev          devtmpfs   2.0G    0   2.0G    0% /dev
4 tmpfs         tmpfs      394M   6.4M   388M    2% /run
5 /dev/sda1     ext4       21G    8.7G   11G   45% /
6 tmpfs         tmpfs      2.0G   256K   2.0G    1% /dev/shm
7 tmpfs         tmpfs      5.0M    4.0K   5.0M    1% /run/lock
8 tmpfs         tmpfs      2.0G    0   2.0G    0% /sys/fs/cgroup
9 tmpfs         tmpfs      394M    44K   394M    1% /run/user/1000
10 [root@VM_0_9_centos ~]# df -Th
11 Filesystem    Type      Size  Used Avail Use% Mounted on
12 /dev/vda1     ext4      50G   4.8G   42G  11% /
13 devtmpfs      devtmpfs  909M    0   909M    0% /dev
14 tmpfs         tmpfs     920M   24K   920M    1% /dev/shm
15 tmpfs         tmpfs     920M  472K   919M    1% /run
16 tmpfs         tmpfs     920M    0   920M    0% /sys/fs/cgroup
17 tmpfs         tmpfs     184M    0   184M    0% /run/user/0
```

检测目录所占磁盘空间：du

du命令用于统计目录或文件所占磁盘空间的大小，该命令的执行结果与df类似，du更侧重于磁盘的使用状况。

du命令的使用格式如下： du [选项] 目录或文件名

选项	含义
-a	递归显示指定目录中各文件和子目录中文件占用的数据块
-s	显示指定文件或目录占用的数据块
-b	以字节为单位显示磁盘占用情况
-l	计算所有文件大小，对硬链接文件计算多次

```

1 python@ubuntu:~$ du -sh /home/
2 1.4G    /home/
3 python@ubuntu:~$ du -h
4 4.0K    ./gnome2/acce1s
5 8.0K    ./gnome2
6 32K     ./pki/nssdb
7 36K     ./pki
8 8.0K    ./dbus/session-bus
9 12K     ./dbus
10 4.0K    ./gnome2_private
11 4.0K    ./Music

```

查看或配置网卡信息：ifconfig

ifconfig显示所有网卡的信息：

```

1 python@ubuntu:~$ ifconfig
2 ens33      Link encap:以太网  硬件地址 00:0c:29:59:65:f2
3             inet 地址:192.168.40.11  广播:192.168.40.255  掩码:255.255.255.0
4             inet6 地址: fe80::432f:6c4a:f47d:5f6b/64  Scope:Link
5             UP BROADCAST RUNNING MULTICAST  MTU:1500  跃点数:1
6             接收数据包:422794  错误:0  丢弃:0  过载:0  帧数:0
7             发送数据包:208666  错误:0  丢弃:0  过载:0  载波:0
8             碰撞:0  发送队列长度:1000
9             接收字节:603741383 (603.7 MB)  发送字节:12819550 (12.8 MB)
10
11 lo         Link encap:本地环回
12             inet 地址:127.0.0.1  掩码:255.0.0.0
13             inet6 地址: ::1/128  Scope:Host
14             UP LOOPBACK RUNNING  MTU:65536  跃点数:1
15             接收数据包:2248  错误:0  丢弃:0  过载:0  帧数:0
16             发送数据包:2248  错误:0  丢弃:0  过载:0  载波:0
17             碰撞:0  发送队列长度:1
18             接收字节:497588 (497.5 KB)  发送字节:497588 (497.5 KB)

```

修改ip:

```
1 python@ubuntu:~$ sudo ifconfig ens33 192.168.40.10
2 python@ubuntu:~$ ifconfig ens33
3 ens33      Link encap:以太网  硬件地址 00:0c:29:59:65:f2
4            inet 地址:192.168.40.10  广播:192.168.40.255  掩码:255.255.255.0
5            inet6 地址: fe80::432f:6c4a:f47d:5f6b/64  Scope:Link
6            UP BROADCAST RUNNING MULTICAST  MTU:1500  跃点数:1
7            接收数据包:422818  错误:0  丢弃:0  过载:0  帧数:0
8            发送数据包:208692  错误:0  丢弃:0  过载:0  载波:0
9            碰撞:0  发送队列长度:1000
10           接收字节:603745130 (603.7 MB)  发送字节:12822615 (12.8 MB)
```

测试远程主机连通性：ping

```
1 python@ubuntu:~$ ping 192.168.40.1
2 PING 192.168.40.1 (192.168.40.1) 56(84) bytes of data.
3 64 bytes from 192.168.40.1: icmp_seq=1 ttl=64 time=0.699 ms
4 64 bytes from 192.168.40.1: icmp_seq=2 ttl=64 time=0.372 ms
5 ^C
6 --- 192.168.40.1 ping statistics ---
7 2 packets transmitted, 2 received, 0% packet loss, time 1001ms
8 rtt min/avg/max/mdev = 0.372/0.535/0.699/0.165 ms
9 python@ubuntu:~$ ping 192.168.40.1 -c 3
10 PING 192.168.40.1 (192.168.40.1) 56(84) bytes of data.
11 64 bytes from 192.168.40.1: icmp_seq=1 ttl=64 time=0.409 ms
12 64 bytes from 192.168.40.1: icmp_seq=2 ttl=64 time=0.367 ms
13 64 bytes from 192.168.40.1: icmp_seq=3 ttl=64 time=0.373 ms
14
15 --- 192.168.40.1 ping statistics ---
16 3 packets transmitted, 3 received, 0% packet loss, time 2000ms
17 rtt min/avg/max/mdev = 0.367/0.383/0.409/0.018 ms
18 python@ubuntu:~$
```

Linux 磁盘管理

Linux磁盘管理常用三个命令为df、du和fdisk。

- df：列出文件系统的整体磁盘使用量
- du：检查磁盘空间使用量
- fdisk：用于磁盘分区

df

获取硬盘被占用了多少空间，目前还剩下多少空间等信息。

语法：

```
1 df [-ahikHTm] [目录或文件名]
```

选项与参数：

- -a：列出所有的文件系统，包括系统特有的 /proc 等文件系统；
- -k：以 KBytes 的容量显示各文件系统；
- -m：以 MBytes 的容量显示各文件系统；
- -h：以人们较易阅读的 GBytes, MBytes, KBytes 等格式自行显示；
- -H：以 M=1000K 取代 M=1024K 的进位方式；

- -T：显示文件系统类型, 连同该 partition 的 filesystem 名称 (例如 ext3) 也列出;
- -i：不用硬盘容量, 而以 inode 的数量来显示

```
1 [root@VM_0_9_centos ~]# df -hT
2 Filesystem      Type      Size  Used Avail Use% Mounted on
3 /dev/vda1       ext4      50G   4.9G   42G   11% /
4 devtmpfs        devtmpfs  909M    0   909M    0% /dev
5 tmpfs           tmpfs     920M   24K   920M    1% /dev/shm
6 tmpfs           tmpfs     920M  496K   919M    1% /run
7 tmpfs           tmpfs     920M    0   920M    0% /sys/fs/cgroup
8 tmpfs           tmpfs     184M    0   184M    0% /run/user/0
```

将系统内的所有特殊文件格式及名称都列出来

```
1 [root@www ~]# df -aT
2 Filesystem      Type 1k-blocks    Used Available Use% Mounted on
3 /dev/hdc2       ext3  9920624 3823112   5585444  41% /
4 proc           proc          0         0           0  -  /proc
5 sysfs          sysfs          0         0           0  -  /sys
6 devpts         devpts         0         0           0  -  /dev/pts
7 /dev/hdc3       ext3  4956316 141376   4559108   4% /home
8 /dev/hdc1       ext3  101086  11126    84741   12% /boot
9 tmpfs          tmpfs        371332      0   371332    0% /dev/shm
10 none binfmt_misc      0         0           0  -  /proc/sys/fs/binfmt_misc
11 sunrpc rpc_pipefs      0         0           0  -  /var/lib/nfs/rpc_pipefs
```

du

du命令是对文件和目录磁盘使用的空间的查看.

语法:

```
1 du [-ahskm] 文件或目录名称
```

选项与参数:

- -a：列出所有的文件与目录容量, 因为默认仅统计目录底下的文件量而已。
- -h：以人们较易读的容量格式 (G/M) 显示;
- -s：列出总量而已, 而不列出每个各别的目录占用容量;
- -S：不包括子目录下的总计, 与 -s 有点差别。
- -k：以 KBytes 列出容量显示;
- -m：以 MBytes 列出容量显示;

du没有加任何选项时, 只列出当前目录下的所有文件夹容量 (包括隐藏文件夹):

```
1 [root@www ~]# du
2 8      ./test4      <==每个目录都会列出来
3 8      ./test2
4 ....中间省略....
5 12     ./gconfd     <==包括隐藏文件的目录
6 220    .            <==这个目录(.)所占用的总量
```

直接输入 du 没有加任何选项时, 则 du 会分析当前所在目录的文件与目录所占用的硬盘空间。

加 -a 选项才显示文件的容量:


```

1 [root@www ~]# du -a
2 12      ./install.log.syslog    <==有文件的列表了
3 8       ./bash_logout
4 8       ./test4
5 8       ./test2
6 ....中间省略....
7 12      ./gconfd
8 220     .

```

检查根目录下每个目录所占用的容量

```

1 [root@www ~]# du -sh /*
2 0       /bin
3 108M    /boot
4 4.0K    /data
5 .....中间省略....
6 0       /proc
7 .....中间省略....
8 40K     /tmp
9 2.4G    /usr
10 2.4G    /var

```

fdisk

fdisk 是 Linux 的磁盘分区表操作工具。

语法：

```

1 fdisk [-l] 装置名称

```

选项与参数：

- -l：输出后面接的装置所有的分区内容。若仅有 fdisk -l 时，则系统将会把整个系统内能够搜寻到的装置的分区均列出来。

列出所有分区信息：

```

1 [root@AY120919111755c246621 tmp]# fdisk -l
2
3 Disk /dev/xvda: 21.5 GB, 21474836480 bytes
4 255 heads, 63 sectors/track, 2610 cylinders
5 Units = cylinders of 16065 * 512 = 8225280 bytes
6 Sector size (logical/physical): 512 bytes / 512 bytes
7 I/O size (minimum/optimal): 512 bytes / 512 bytes
8 Disk identifier: 0x00000000
9
10      Device Boot      Start         End      Blocks    Id  System
11  /dev/xvda1    *           1         2550       2048000    83  Linux
12  /dev/xvda2             2550         2611        490496    82  Linux swap / Solaris
13
14 Disk /dev/xvdb: 21.5 GB, 21474836480 bytes
15 255 heads, 63 sectors/track, 2610 cylinders
16 Units = cylinders of 16065 * 512 = 8225280 bytes
17 Sector size (logical/physical): 512 bytes / 512 bytes
18 I/O size (minimum/optimal): 512 bytes / 512 bytes
19 Disk identifier: 0x56f40944
20

```

21	Device	Boot	Start	End	Blocks	Id	System
22	/dev/xvdb2		1	2610	20964793+	83	Linux

查看根目录所在磁盘，并查阅该硬盘内的相关信息：

```

1 [root@www ~]# df / <==注意：重点在找出磁盘文件名而已
2 Filesystem          1k-blocks      Used Available Use% Mounted on
3 /dev/hdc2           9920624      3823168    5585388   41% /
4
5 [root@www ~]# fdisk /dev/hdc <==不要加上数字！
6 The number of cylinders for this disk is set to 5005.
7 There is nothing wrong with that, but this is larger than 1024,
8 and could in certain setups cause problems with:
9 1) software that runs at boot time (e.g., old versions of LILO)
10 2) booting and partitioning software from other OSS
11    (e.g., DOS FDISK, OS/2 FDISK)
12
13 Command (m for help): <==等待你的输入！

```

输入 m 后，就会看到底下这些命令介绍

```

1 Command (m for help): m <== 输入 m 后，就会看到底下这些命令介绍
2 Command action
3   a  toggle a bootable flag
4   b  edit bsd disklabel
5   c  toggle the dos compatibility flag
6   d  delete a partition <==删除一个partition
7   l  list known partition types
8   m  print this menu
9   n  add a new partition <==新增一个partition
10  o  create a new empty DOS partition table
11  p  print the partition table <==在屏幕上显示分割表
12  q  quit without saving changes <==不储存离开fdisk程序
13  s  create a new empty Sun disklabel
14  t  change a partition's system id
15  u  change display/entry units
16  v  verify the partition table
17  w  write table to disk and exit <==将刚刚的动作写入分割表
18  x  extra functionality (experts only)

```

离开 fdisk 时按下 q，那么所有的动作都不会生效！相反的，按下 w 就是动作生效的意思。

```

1 Command (m for help): p <== 这里可以输出目前磁盘的状态
2
3 Disk /dev/hdc: 41.1 GB, 41174138880 bytes <==这个磁盘的文件名与容量
4 255 heads, 63 sectors/track, 5005 cylinders <==磁头、扇区与磁柱大小
5 units = cylinders of 16065 * 512 = 8225280 bytes <==每个磁柱的大小
6
7   Device Boot      Start          End      Blocks   Id  System
8   /dev/hdc1    *            1           13       104391    83  Linux
9   /dev/hdc2                14          1288     10241437+   83  Linux
10  /dev/hdc3           1289          1925     5116702+   83  Linux
11  /dev/hdc4           1926          5005     24740100    5  Extended
12  /dev/hdc5           1926          2052     1020096    82  Linux swap / Solaris
13 # 装置文件名 启动区否 开始磁柱      结束磁柱 1K大小容量 磁盘分区槽内的系统
14

```

使用 `p` 可以列出目前这颗磁盘的分割表信息，这个信息的上半部在显示整体磁盘的状态。

磁盘格式化

磁盘分割完毕后自然就是要进行文件系统的格式化，格式化的命令非常的简单，使用 `mkfs` (make filesystem) 命令。

语法：

```
1 | mkfs [-t 文件系统格式] 装置文件名
```

选项与参数：

- `-t`：可以接文件系统格式，例如 `ext3`, `ext2`, `vfat` 等(系统有支持才会生效)

查看 `mkfs` 支持的文件格式：

```
1 | [root@VM_0_9_centos web]# mkfs[tab]
2 | mkfs          mkfs.cramfs  mkfs.ext3      mkfs.minix
3 | mkfs.btrfs    mkfs.ext2    mkfs.ext4      mkfs.xfs
```

按下两个`[tab]`，会发现 `mkfs` 支持的文件格式如上所示。

将分区 `/dev/hdc6` (可指定其他分区) 格式化为`ext3`文件系统：

```
1 | [root@www ~]# mkfs -t ext3 /dev/hdc6
2 | mke2fs 1.39 (29-May-2006)
3 | Filesystem label=          <==这里指的是分割槽的名称(label)
4 | OS type: Linux
5 | Block size=4096 (log=2)    <==block 的大小配置为 4k
6 | Fragment size=4096 (log=2)
7 | 251392 inodes, 502023 blocks <==由此配置决定的inode/block数量
8 | 25101 blocks (5.00%) reserved for the super user
9 | First data block=0
10 | Maximum filesystem blocks=515899392
11 | 16 block groups
12 | 32768 blocks per group, 32768 fragments per group
13 | 15712 inodes per group
14 | Superblock backups stored on blocks:
15 |     32768, 98304, 163840, 229376, 294912
16 |
17 | writing inode tables: done
18 | Creating journal (8192 blocks): done <==有日志记录
19 | writing superblocks and filesystem accounting information: done
20 |
21 | This filesystem will be automatically checked every 34 mounts or
22 | 180 days, whichever comes first. Use tune2fs -c or -i to override.
23 | # 这样就创建起来我们所需要的 Ext3 文件系统了！简单明了！
```

磁盘检验

fsck (file system check) 用来检查和维护不一致的文件系统。

若系统掉电或磁盘发生问题，可利用fsck命令对文件系统进行检查。

语法：

```
1 | fsck [-t 文件系统] [-ACay] 装置名称
```

选项与参数：

- -t：给定档案系统的型式，若在 /etc/fstab 中已有定义或 kernel 本身已支援的则不需加上此参数
- -s：依序一个一个地执行 fsck 的指令来检查
- -A：对/etc/fstab 中所有列出来的 分区 (partition) 做检查
- -C：显示完整的检查进度
- -d：打印出 e2fsck 的 debug 结果
- -p：同时有 -A 条件时，同时有多个 fsck 的检查一起执行
- -R：同时有 -A 条件时，省略 / 不检查
- -V：详细显示模式
- -a：如果检查有错则自动修复
- -r：如果检查有错则由使用者回答是否修复
- -y：选项指定检测每个文件是自动输入yes，在不确定那些是不正常的时候，可以执行 # fsck -y 全部检查修复。

查看系统有多少文件系统支持的 fsck 命令：

```
1 | [root@www ~]# fsck[tab][tab]
2 | fsck          fsck.cramfs  fsck.ext2      fsck.ext3      fsck.msdos     fsck.vfat
```

强制检测 /dev/hdc6 分区：

```
1 | [root@www ~]# fsck -C -f -t ext3 /dev/hdc6
2 | fsck 1.39 (29-May-2006)
3 | e2fsck 1.39 (29-May-2006)
4 | Pass 1: Checking inodes, blocks, and sizes
5 | Pass 2: Checking directory structure
6 | Pass 3: Checking directory connectivity
7 | Pass 4: Checking reference counts
8 | Pass 5: Checking group summary information
9 | vbird_logical: 11/251968 files (9.1% non-contiguous), 36926/1004046 blocks
```

如果没有加上 -f 的选项，则由于这个文件系统不曾出现问题，检查的经过非常快速！若加上 -f 强制检查，才会一项一项的显示过程。

磁盘挂载与卸除

Linux 的磁盘挂载使用 `mount` 命令，卸载使用 `umount` 命令。

磁盘挂载语法：

```
1 | mount [-t 文件系统] [-L Label名] [-o 额外选项] [-n] 装置文件名 挂载点
```

用默认的方式，将刚刚创建的 /dev/hdc6 挂载到 /mnt/hdc6 上面！

```

1 [root@www ~]# mkdir /mnt/hdc6
2 [root@www ~]# mount /dev/hdc6 /mnt/hdc6
3 [root@www ~]# df
4 Filesystem            1k-blocks      Used Available Use% Mounted on
5 .....中间省略.....
6 /dev/hdc6              1976312       42072   1833836    3% /mnt/hdc6

```

磁盘卸载命令 `umount` 语法：

```

1 umount [-fn] 装置文件名或挂载点

```

选项与参数：

- -f：强制卸除！可用在类似网络文件系统 (NFS) 无法读取到的情况下；
- -n：不升级 /etc/mtab 情况下卸除。

卸载/dev/hdc6

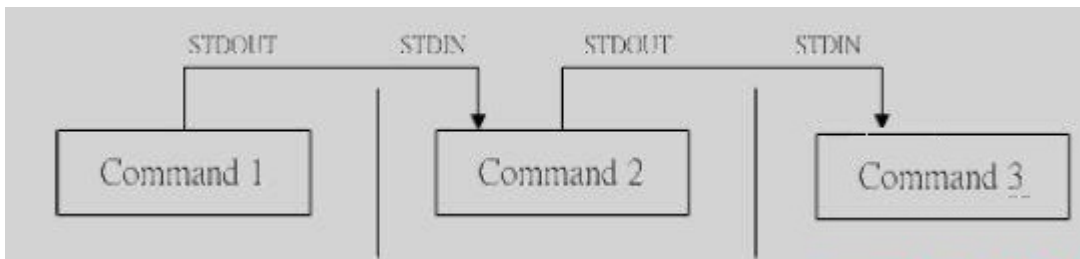
```

1 [root@www ~]# umount /dev/hdc6

```

Linux管道命令

Linux的管道命令是'|'，通过它可以对数据进行连续处理，其示意图如下：



注意：

- 1) 管道命令仅处理标准输出，对于标准错误输出，将忽略
- 2) 管道命令右边命令，必须能够接收标准输入流命令才行，否则传递过程中数据会抛弃。

常用来作为接收数据管道命令有：less,more,head,tail，而ls, cp, mv就不行。

wc - 统计字数

可以计算文件的Byte数、字数、或是列数，若不指定文件名称、或是所给予的文件名为"-", 则wc指令会从标准输入设备读取数据。

```

1 wc [-lwm] [filename]
2 -l: 统计行数
3 -w: 统计英文单词
4 -m: 统计字符数
5 python@xxx:~$ wc -l /etc/passwd
6 49 /etc/passwd
7 python@xxx:~$ wc -w /etc/passwd
8 81 /etc/passwd
9 python@xxx:~$ wc -m /etc/passwd
10 2696 /etc/passwd

```

在默认的情况下，wc将计算指定文件的行数、字数，以及字节数。使用的命令为：

```
1 $ wc testfile          # testfile文件的统计信息
2 3 92 598 testfile      # testfile文件的行数为3、单词数92、字节数598
```

其中，3 个数字分别表示testfile文件的行数、单词数，以及该文件的字节数。

如果想同时统计多个文件的信息，例如同时统计testfile、testfile_1、testfile_2，可使用如下命令：

```
1 $ wc testfile testfile_1 testfile_2 #统计三个文件的信息
2 3 92 598 testfile                  #第一个文件行数为3、单词数92、字节数598
3 9 18 78 testfile_1                 #第二个文件的行数为9、单词数18、字节数78
4 3 6 32 testfile_2                  #第三个文件的行数为3、单词数6、字节数32
5 15 116 708 总用量                 #三个文件总共的行数为15、单词数116、字节数708
```

cut - 列选取命令

```
1 选项与参数：
2 -d   ：后面接分隔字符。与 -f 一起使用；
3 -f   ：依据 -d 的分隔字符将一段信息分割成为数段，用 -f 取出第几段的意思；
4 -c   ：以字符（characters）的单位取出固定字符区间；
```

cut以行为单位，根据分隔符把行分成若干列，这样就可以指定选取哪些列了。

```
1 cut -d '分隔字符' -f 选取的列数
2 echo $PATH|cut -d ':' -f 2      --选取第2列
3 echo $PATH|cut -d ':' -f 3,5    --选取第3列和第5列
4 echo $PATH|cut -d ':' -f 3-5    --选取第3列到第5列
5 echo $PATH|cut -d ':' -f 3-     --选取第3列到最后1列
6 echo $PATH|cut -d ':' -f 1-3,5  --选取第1到第3列还有第5列
```

只显示/etc/passwd的用户和shell：

```
1 #cat /etc/passwd | cut -d ':' -f 1,7
2 root:/bin/bash
3 daemon:/bin/sh
4 bin:/bin/sh
```

grep - 行选取命令

grep一般格式为：

```
1 grep [-cinv] '查找的字符串' filename
```

在grep命令中输入字符串参数时，最好引号或双引号括起来。例如：`grep 'a' 1.txt`。

常用选项说明：

选项	含义
-v	显示不包含匹配文本的所有行（相当于求反）
-n	显示匹配行及行号
-i	忽略大小写
-c	计算找到的行数

grep搜索内容串可以是正则表达式，常用正则表达式：

参数	含义
^a	行首,搜寻以 m 开头的行; grep -n '^a' 1.txt
ke\$	行尾,搜寻以 ke 结束的行; grep -n 'ke\$' 1.txt
[Ss]igna[Ll]	匹配 [] 里中一系列字符中的一个; 搜寻匹配单词signal、signal、Signal、Signal的行; grep -n '[Ss]igna[Ll]' 1.txt
.	.匹配一个非换行符的字符; grep -n 'e.e' 1.txt可以匹配 eee, eae, eve, 但是不匹配 ee, eaae;
*	匹配零个或多个先前字符
[^]	匹配一个不在指定范围内的字符
\(.\.)	标记匹配字符
\	锚定单词的开始
\<	锚定单词的开头
\>	锚定单词的结束
x\{m\}	重复字符x, m次
x\{m,\}	重复字符x,至少m次
x\{m,n\}	重复字符x, 至少m次, 不多于n次
\w	匹配文字和数字字符, 也就是[A-Za-z0-9]
\b	单词锁定符

实例：
显示所有以“h”结尾的行
grep h\$
匹配所有以“a”开头且以“e”结尾的，中间包含2个字符的单词
grep '<a..e>'
显示所有包含一个“y”或“h”字符的行
grep [yh]
显示不包含字母a~k 且后紧跟“pple”的单词
grep '[^a-k]pple'

从系统词典中选择所有以“c”开头且以“o”结尾的单词

```
grep '\<c.*o\>'
```

找出一个文件中或者输出中找到包含 * 的行

```
grep '\>'
```

显示所有包含每个字符串至少有20个连续字母的单词的行

```
grep [a-z]\{20,\}
```

sort - 排序

语法:

```
1 | sort [-fbMnrtuk] [file or stdin]
```

参数说明:

- -f : 忽略大小写的差异, 例如 A 与 a 视为编码相同;
- -b : 忽略最前面的空格符部分;
- -M : 以月份的名字来排序, 例如 JAN, DEC 等等的排序方法;
- -n : 使用『纯数字』进行排序(默认是以文字型态来排序的);
- -r : 反向排序;
- -u : 就是 uniq , 相同的数据中, 仅出现一行代表;
- -t : 分隔符, 默认是用 [tab] 键来分隔;
- -k : 以哪个区间 (field) 来进行排序

默认是以第一个字符升序排序:

```
1 | # cat /etc/passwd | sort
2 | adm:x:3:4:adm:/var/adm:/sbin/nologin
3 | avahi-autoipd:x:100:156:avahi-autoipd:/var/lib/avahi-autoipd:/sbin/nologin
4 | avahi:x:70:70:Avahi daemon:./sbin/nologin
5 | bin:x:1:1:bin:/bin:/sbin/nologin
6 | .....
```

以第3列排序:

```
1 | [root@www ~]# cat /etc/passwd | sort -t ':' -k 3
2 | root:x:0:0:root:/root:/bin/bash
3 | uucp:x:10:14:uucp:/var/spool/uucp:/sbin/nologin
4 | operator:x:11:0:operator:/root:/sbin/nologin
5 | bin:x:1:1:bin:/bin:/sbin/nologin
6 | games:x:12:100:games:/usr/games:/sbin/nologin
```

使用数字排序:

```
1 | cat /etc/passwd | sort -t ':' -k 3n
2 | root:x:0:0:root:/root:/bin/bash
3 | daemon:x:1:1:daemon:/usr/sbin:/bin/sh
4 | bin:x:2:2:bin:/bin:/bin/sh
```

倒序排序:


```

1 cat /etc/passwd | sort -t ':' -k 3nr
2 nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
3 ntp:x:106:113::/home/ntp:/bin/false
4 messagebus:x:105:109::/var/run/dbus:/bin/false
5 sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
6 或者
7 cat /etc/passwd | sort -t ':' -k 3 -nr

```

先以第六个域的第2个字符到第4个字符进行正向排序，再基于第一个域进行反向排序：

```

1 cat /etc/passwd | sort -t ':' -k 6.2,6.4 -k 1r
2 sync:x:4:65534:sync:/bin:/bin/sync
3 proxy:x:13:13:proxy:/bin:/bin/sh
4 bin:x:2:2:bin:/bin:/bin/sh
5 sys:x:3:3:sys:/dev:/bin/sh

```

查看/etc/passwd有多少个shell:

方法对/etc/passwd的第七个域排序并去重，然后统计行数：

```

1 [root@VM_0_9_centos ~]# cat /etc/passwd | sort -t':' -k 7 -u
2 root:x:0:0:root:/root:/bin/bash
3 syslog:x:996:994::/home/syslog:/bin/false
4 sync:x:5:0:sync:/sbin:/bin/sync
5 halt:x:7:0:halt:/sbin:/sbin/halt
6 bin:x:1:1:bin:/bin:/sbin/nologin
7 shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
8 [root@VM_0_9_centos ~]# cat /etc/passwd | sort -t':' -k 7 -u|wc -l
9 6

```

uniq - 去重

```

1 选项与参数：
2 -i   : 忽略大小写字符的不同；
3 -c   : 进行计数
4 -u   : 只显示唯一的行

```

该命令用于排完序之后，对排序结果进行去重

```

1 python@xxx:~$ last | cut -d ' ' -f 1 | sort | uniq
2
3 haha
4 python
5 reboot
6 wtmp
7 python@xxx:~$ last | cut -d ' ' -f 1 | sort | uniq -c
8      1
9      2 haha
10     22 python
11      7 reboot
12      1 wtmp

```

排序文件，默认是去重：

```
1 #cat words | sort | uniq
2 friend
3 hello
4 world
```

排序之后删除了重复行，同时在行首位置输出该行重复的次数：

```
1 #sort testfile | uniq -c
2 1 friend
3 3 hello
4 2 world
```

仅显示存在重复的行，并在行首显示该行重复的次数：

```
1 #sort testfile | uniq -dc
2 3 hello
3 2 world
```

仅显示不重复的行：

```
1 sort testfile | uniq -u
2 friend
```

tee - 同时输出多个文件

从标准输入设备读取数据，将其内容输出到标准输出设备，同时保存成文件。

一般情况下用重定向实现，需要同时输出多个文件时可以使用该命令。

参数：

- -a或--append 附加到既有文件的后面，而非覆盖它。

将输出同时保存到多个文件中，同时将输出内容显示到控制台：

```
1 python@xxx:~/test$ echo "hello world"|tee f1 f2
2 hello world
3 python@xxx:~/test$ cat f1
4 hello world
5 python@xxx:~/test$ echo "hello world"|tee f1 f2 -a
6 hello world
7 python@xxx:~/test$ cat f1
8 hello world
9 hello world
```

tr - 替换指定的字符

不指定参数时，即表示替换指定的字符为另一个字符，支持指定的字符集合。

参数说明：

- -d, --delete：删除指定的字符
- -s, --squeeze-repeats：缩减连续重复的字符成指定的单个字符

字符集合的范围：

- \NNN 八进制值的字符 NNN (1 to 3 为八进制值的字符)

- \反斜杠
- \a Ctrl-G 铃声
- \b Ctrl-H 退格符
- \f Ctrl-L 走行换页
- \n Ctrl-J 新行
- \r Ctrl-M 回车
- \t Ctrl-I tab键
- \v Ctrl-X 水平制表符
- CHAR1-CHAR2：字符范围从 CHAR1 到 CHAR2 的指定，范围的指定以 ASCII 码的次序为基础，只能由小到大，不能由大到小。
- [CHAR*]：这是 SET2 专用的设定，功能是重复指定的字符到与 SET1 相同长度为止
- [CHAR*REPEAT]：这也是 SET2 专用的设定，功能是重复指定的字符到设定的 REPEAT 次数为止(REPEAT 的数字采 8 进位制计算，以 0 为开始)
- [:alnum:]：所有字母字符与数字
- [:alpha:]：所有字母字符
- [:blank:]：所有水平空格
- [:cntrl:]：所有控制字符
- [:digit:]：所有数字
- [:graph:]：所有可打印的字符(不包含空格符)
- [:lower:]：所有小写字母
- [:print:]：所有可打印的字符(包含空格符)
- [:punct:]：所有标点字符
- [:space:]：所有水平与垂直空格符
- [:upper:]：所有大写字母
- [:xdigit:]：所有 16 进位制的数字
- [=CHAR=]：所有符合指定的字符(等号里的 CHAR，代表你可自订的字符)

将文件testfile中的小写字母全部转换成大写字母：

```
1 cat testfile |tr a-z A-Z
2 或
3 cat testfile |tr [:lower:] [:upper:]
```

缩减连续重复的字符成指定的单个字符：

```
1 python@xxx:~/test$ cat t
2 dddddddssssdd
3 eeeeeeeee
4 aaaaaaaaaaaaa
5 vvvvvvvvvvvvvv
6 python@xxx:~/test$ cat t|tr -s 'se'
7 ddddddsdd
8 e
9 aaaaaaaaaaaaa
10 vvvvvvvvvvvvvv
11 python@xxx:~/test$ cat t|tr -s 'sdeav'
12 dsd
13 e
14 a
15 v
```

删除指定的字符：

```
1 python@xxx:~/test$ cat t|tr -d 'dv'
2 sssss
3 eeeeeeeeeee
4 aaaaaaaaaaaaaa
```

join - 文件按行连接

将两个文件中指定栏位相同的行连接起来。即按照两个文件中共同拥有的某一列，将对应的行拼接成一行。

注意：在使用join之前所处理的文件要事先经过排序。

```
1 $ cat testfile_1
2 Hello 95 #例如，本例中第一列为姓名，第二列为数额
3 Linux 85
4 test 30
5 cmd@hdd-desktop:~$ cat testfile_2
6 Hello 2005 #例如，本例中第一列为姓名，第二列为年份
7 Linux 2009
8 test 2006
```

使用join命令，将两个文件连接：

```
1 $ join testfile_1 testfile_2 #连接testfile_1、testfile_2中的内容
2 Hello 95 2005 #连接后显示的内容
3 Linux 85 2009
4 test 30 2006
```

两个文件互换，输出结果的变化：

```
1 $ join testfile_2 testfile_1 #改变文件顺序连接两个文件
2 Hello 2005 95 #连接后显示的内容
3 Linux 2009 85
4 test 2006 30
```

参数：

- -a<1或2> 除了显示原来的输出内容之外，还显示指令文件中没有相同栏位的行。
- -e<字符串> 若[文件1]与[文件2]中找不到指定的栏位，则在输出中填入选项中的字符串。
- -i或--ignore-case 比较栏位内容时，忽略大小写的差异。
- -o<格式> 按照指定的格式来显示结果。
- -t<字符> 使用栏位的分隔字符。
- -v<1或2> 跟-a相同，但是只显示文件中没有相同栏位的行。
- -1<栏位> 连接[文件1]指定的栏位。
- -2<栏位> 连接[文件2]指定的栏位。

paste-将多个文件对应行链接在一起

paste 指令会把每个文件以列对列的方式，一列列地加以合并。

语法：

```
1 paste [-s] [-d <间隔字符>] [文件...]
```

参数：

- -d<间隔字符>或--delimiters=<间隔字符> 用指定的间隔字符取代跳格字符。
- -s或--serial 串行进行而非平行处理。
- [文件...] 指定操作的文件路径

使用paste指令将文件"file"、"testfile"、"testfile1"进行合并，输入如下命令：

```
1 $ cat file #file文件的内容
2 xiongdan 200
3 lihaihui 233
4 lym1r1 231
5 $ cat testfile #testfile文件的内容
6 liangyuanm ss
7 $ cat testfile1 #testfile1文件的内容
8 huanggai 56
9 zhixi 73
10 $ paste file testfile testfile1
11 xiongdan 200 liangyuanm ss huanggai 56
12 lihaihui 233 zhixi 73
13 lym1r1 231
14 $ paste -d ':' file testfile testfile1
15 xiongdan 200:liangyuanm ss:huanggai 56
16 lihaihui 233::zhixi 73
17 lym1r1 231::
```

参数"-s"可以将一个文件中的多行数据合并为一行进行显示：

```
1 $ paste -s file #合并指定文件的多行数据
2 xiongdan 200 lihaihui 233 lym1r1 231
```

如果将文件位置改为-，表示接收标准输入：

```
1 $ cat file |paste testfile1 -
2 huanggai 56 xiongdan 200
3 zhixi 73 lihaihui 233
4 lym1r1 231
```

split - 文件切割

split命令用于将一个文件分割成数个。

该指令将大文件分割成较小的文件，在默认情况下将按照每1000行切割成一个小文件。

语法：

```
1 split [-bl] file prefix
2 -b: 以大小切割
3 -l: 以行数切割
4 prefix: 切割后文件的前缀
```

参数说明：

- -<行数>：指定每多少行切成一个小文件
- -b<字节>：指定每多少字节切成一个小文件
- -C<字节>：与参数"-b"相似，但是在切割时将尽量维持每行的完整性
- [输出文件名]：设置切割后文件的前置文件名，split会自动在前置文件名后再加上编号

使用指令"split"将文件"README"每6行切割成一个文件，输入如下命令：

```
1 $ split -6 README          #将README文件每六行分割成一个文件
2 $ ls                      #执行ls指令
3 #获得当前目录结构
4 README xaa xad xag xab xae xah xac xaf xai
```

以上命令执行后，指令"split"会将原来的大文件"README"切割成多个以"x"开头的小文件。而在这些小文件中，每个文件都只有6行内容。

以大小切割：

```
1 $ ls -lh disease.dmp
2 -rwxr-xr-x 1 root root 122M Jul  4 2013 disease.dmp
3 $ split -b 50m disease.dmp disease.dmp
4 $ ls -lh disease.dmp*
5 -rwxr-xr-x 1 root root 122M Jul  4 2013 disease.dmp
6 -rw-r--r-- 1 root root  50M Jan  9 16:10 disease.dmpaa
7 -rw-r--r-- 1 root root  50M Jan  9 16:10 disease.dmpab
8 -rw-r--r-- 1 root root  22M Jan  9 16:10 disease.dmpac
```

xargs - 参数代换

不是所有的命令都支持管道，如ls，对于不支持管道的命令，可以通过xargs让其有管道命令的效果，如下所示：

```
1 # find /sbin -perm +7000 | xargs ls -l
2 -rwsr-x--- 1 root ecryptfs 19896 Feb 23 2012 /sbin/mount.ecryptfs_private
3 -rwsr-xr-x 1 root root      75496 Jan  9 2013 /sbin/mount.nfs
4 -rwsr-xr-x 1 root root      75504 Jan  9 2013 /sbin/mount.nfs4
5 -rwxr-sr-x 1 root root       8544 Feb 22 2012 /sbin/netreport
6 -rwsr-xr-x 1 root root      14112 Nov  2 2010 /sbin/pam_timestamp_check
7 -rwsr-xr-x 1 root root      75504 Jan  9 2013 /sbin/umount.nfs
8 -rwsr-xr-x 1 root root      75504 Jan  9 2013 /sbin/umount.nfs4
9 -rwsr-xr-x 1 root root      19768 Nov  2 2010 /sbin/unix_chkpwd
```

如果没有xargs，ls -l的结果将不是前面find的标准输出，因为ls不支持管道命令。

xargs 用作替换工具，读取输入数据重新格式化后输出。

定义一个测试文件，内有多行文本数据：

```
1 # cat test.txt
2 a b c d e f g
3 h i j k l m n
4 o p q
5 r s t
6 u v w x y z
```

多行输入单行输出：

```
1 # cat test.txt | xargs
2 a b c d e f g h i j k l m n o p q r s t u v w x y z
```

-n 选项多行输出：

```
1 | # cat test.txt | xargs -n3
2 | a b c
3 | d e f
4 | g h i
5 | j k l
6 | m n o
7 | p q r
8 | s t u
9 | v w x
10 | y z
```

-d 选项可以自定义一个定界符：

```
1 | # echo "nameXnameXnameXname" | xargs -dx
2 | name name name name
```

结合 -n 选项使用：

```
1 | # echo "nameXnameXnameXname" | xargs -dx -n2
2 | name name
3 | name name
```

读取 stdin，将格式化后的参数传递给命令：

```
1 | # cat sk.sh
2 | #!/bin/bash
3 | #sk.sh命令内容，打印出所有参数。
4 | echo $*
5 | # cat arg.txt
6 | aaa
7 | bbb
8 | ccc
9 | # cat arg.txt | xargs -I {} ./sk.sh -p {} -l
10 | -p aaa -l
11 | -p bbb -l
12 | -p ccc -l
```

选项 -I 指定一个替换字符串 {}，这个字符串在 xargs 扩展时会被替换掉。

复制所有图片文件到 /data/images 目录下：

```
1 | ls *.jpg | xargs -n1 -I {} cp {} /data/images
```

选项 -n 后面加次数，表示命令在执行的时候一次用的 argument 的个数，默认是用所有的。

xargs 结合 find 使用

用 rm 删除太多的文件时候，可能得到一个错误信息：**/bin/rm Argument list too long**。用 xargs 去避免这个问题：

```
1 | find . -type f -name "*.log" -print0 | xargs -0 rm -f
```

xargs -0 将 \0 作为定界符。

统计一个源代码目录中所有 php 文件的行数：

```
1 | find . -type f -name "*.php" -print0 | xargs -0 wc -l
```

查找所有的 jpg 文件，并且压缩它们：

```
1 | find . -type f -name "*.jpg" -print | xargs tar -czvf images.tar.gz
```

批量下载：

```
1 | # cat url-list.txt | xargs wget -c
```

wget的 `-c` 选项表示断点续传。

linux命令练习

常用命令

```
1 | 1. 进入到用户根目录
2 | cd ~ 或 cd
3 |
4 | 2. 查看当前所在目录
5 | pwd
6 |
7 | 3. 进入到hadoop用户根目录
8 | cd ~hadoop
9 |
10 | 4. 返回到原来目录
11 | cd -
12 |
13 | 5. 返回到上一级目录
14 | cd ..
15 |
16 | 6. 查看hadoop用户根目录下的所有文件
17 | ls -la
18 |
19 | 7. 在根目录下创建一个hadoop的文件夹
20 | mkdir /hadoop
21 |
22 | 8. 在/hadoop目录下创建src和WebRoot两个文件夹
23 | 分别创建: mkdir /hadoop/src
24 |           mkdir /hadoop/WebRoot
25 | 同时创建: mkdir /hadoop/{src,WebRoot}
26 |
27 | 进入到/hadoop目录，在该目录下创建.classpath和README文件
28 | 分别创建: touch .classpath
29 |           touch README
30 | 同时创建: touch {.classpath,README}
31 |
32 | 查看/hadoop目录下面的所有文件
33 | ls -la
34 |
35 | 在/hadoop目录下面创建一个test.txt文件,同时写入内容"this is test"
36 | echo "this is test" > test.txt
37 |
38 | 查看一下test.txt的内容
39 | cat test.txt
```



```
40 more test.txt
41 less test.txt
42
43 向README文件追加写入"please read me first"
44 echo "please read me first" >> README
45
46 将test.txt的内容追加到README文件中
47 cat test.txt >> README
48
49 拷贝/hadoop目录下的所有文件到/hadoop-bak
50 cp -r /hadoop /hadoop-bak
51
52 进入到/hadoop-bak目录，将test.txt移动到src目录下，并修改文件名为Student.java
53 mv test.txt src/Student.java
54
55 在src目录下创建一个struts.xml
56 > struts.xml
57
58 删除所有的xml类型的文件
59 rm -rf *.xml
60
61 删除/hadoop-bak目录和下面的所有文件
62 rm -rf /hadoop-bak
63
64 返回到/hadoop目录，查看一下README文件有多单词，多少个少行
65 wc -w README
66 wc -l README
67
68 返回到根目录，将/hadoop目录先打包，再用gzip压缩
69 分步完成: tar -cvf hadoop.tar hadoop
70           gzip hadoop.tar
71 一步完成: tar -zcvf hadoop.tar.gz hadoop
72
73 将其解压缩，再取消打包
74 分步完成: gzip -d hadoop.tar.gz 或 gunzip hadoop.tar.gz
75 一步完成: tar -zxvf hadoop.tar.gz
76
77 将/hadoop目录先打包，同时用bzip2压缩，并保存到/tmp目录下
78 tar -jcvf /tmp/hadoop.tar.bz2 hadoop
79
80 将/tmp/hadoop.tar.bz2解压到/usr目录下面
81 tar -jxvf hadoop.tar.bz2 -C /usr/
```

系统命令

```
1 1. 查看主机名
2 hostname
3
4 2. 修改主机名(重启后无效)
5 hostname hadoop
6
7 3. 修改主机名(重启后永久生效)
8 vi /etc/sysconfig/network
9
10 4. 修改IP(重启后无效)
11 ifconfig eth0 192.168.12.22
12
```

```
13 5. 修改IP(重启后永久生效)
14 vi /etc/sysconfig/network-scripts/ifcfg-eth0
15
16 6. 查看系统信息
17 uname -a
18 uname -r
19
20 7. 查看ID命令
21 id -u
22 id -g
23
24 8. 日期
25 date
26 date +%Y-%m-%d
27 date +%T
28 date +%Y-%m-%d" "%T
29
30 9. 日历
31 cal 2012
32
33 10. 查看文件信息
34 file filename
35
36 11. 挂载硬盘
37 mount
38 umount
39 加载windows共享
40 mount -t cifs //192.168.1.100/tools /mnt
41
42 12. 查看文件大小
43 du -h
44 du -ah
45
46 13. 查看分区
47 df -h
48
49 14. ssh
50 ssh hadoop@192.168.1.1
51
52 15. 关机
53 shutdown -h now /init 0
54 shutdown -r now /reboot
```

用户和组

```
1 添加一个tom用户，设置它属于users组，并添加注释信息
2 分步完成: useradd tom
3             usermod -g users tom
4             usermod -c "hr tom" tom
5 一步完成: useradd -g users -c "hr tom" tom
6
7 设置tom用户的密码
8 passwd tom
9
10 修改tom用户的登陆名为tomcat
11 usermod -l tomcat tom
12
```

```
13 将tomcat添加到sys和root组中
14 usermod -G sys,root tomcat
15
16 查看tomcat的组信息
17 groups tomcat
18
19 添加一个jerry用户并设置密码
20 useradd jerry
21 passwd jerry
22
23 添加一个交america的组
24 groupadd america
25
26 将jerry添加到america组中
27 usermod -g america jerry
28
29 将tomcat用户从root组和sys组删除
30 gpasswd -d tomcat root
31 gpasswd -d tomcat sys
32
33 将america组名修改为am
34 groupmod -n am america
```

权限

```
1 创建a.txt和b.txt文件，将他们设为其拥有者和所在组可写入，但其他以外的人则不可写入：
2 chmod ug+w,o-w a.txt b.txt
3
4 创建c.txt文件所有人都可以写和执行
5 chmod a=wx c.txt 或chmod 666 c.txt
6
7 将/hadoop目录下的所有文件与子目录皆设为任何人可读取
8 chmod -R a+r /hadoop
9
10 将/hadoop目录下的所有文件与子目录的拥有者设为root，用户拥有组为users
11 chown -R root:users /hadoop
12
13 将当前目录下的所有文件与子目录的用户皆设为hadoop，组设为users
14 chown -R hadoop:users *
```

帮助文档

```
1 1.内部命令：echo
2 查看内部命令帮助：help echo 或者 man echo
3
4 2.外部命令：ls
5 查看外部命令帮助：ls --help 或者 man ls 或者 info ls
6
7 3.man文档的类型(1~9)
8 man 7 man
9 man 5 passwd
10
11 4.快捷键：
12 ctrl + c：停止进程
```

```
13 ctrl + l: 清屏
14 ctrl + r: 搜索历史命令
15 ctrl + q: 退出
16
17 5.善于用tab键
```

文件相关命令

```
1 1.进入到用户根目录
2 cd ~ 或者 cd
3 cd ~hadoop
4 回到原来路径
5 cd -
6
7 2.查看文件详情
8 stat a.txt
9
10 3.移动
11 mv a.txt /ect/
12 改名
13 mv b.txt a.txt
14 移动并改名
15 mv a.txt ../b.txt
16
17 4拷贝并改名
18 cp a.txt /etc/b.txt
19
20 5.vi撤销修改
21 ctrl + u (undo)
22 恢复
23 ctrl + r (redo)
24
25 6.名令设置别名(重启后无效)
26 alias ll="ls -l"
27 取消
28 unalias ll
29
30 7.如果想让别名重启后仍然有效需要修改
31 vi ~/.bashrc
32
33 8.添加用户
34 useradd hadoop
35 passwd hadoop
36
37 9创建多个文件
38 touch a.txt b.txt
39 touch /home/{a.txt,b.txt}
40
41 10.将一个文件的内容复制到里另一个文件中
42 cat a.txt > b.txt
43 追加内容
44 cat a.txt >> b.txt
45
46
47 11.将a.txt 与b.txt设为其拥有者和其所属同一个组者可写入，但其他以外的人则不可写入：
48 chmod ug+w,o-w a.txt b.txt
49
50 chmod a=wx c.txt
```

```
51
52 12.将当前目录下的所有文件与子目录皆设为任何人可读取：
53 chmod -R a+r *
54
55 13.将a.txt的用户拥有者设为users,组的拥有者设为jessie:
56 chown users:jessie a.txt
57
58 14.将当前目录下的所有文件与子目录的用户的使用者为lampport,组拥有者皆设为users,
59 chown -R lampport:users *
60
61 15.将所有的java语言程式拷贝至finished子目录中：
62 cp *.java finished
63
64 16.将目前目录及其子目录下所有扩展名是java的文件列出来。
65 find -name "*.java"
66 查找当前目录下扩展名是java 的文件
67 find -name *.java
68
69 17.删除当前目录下扩展名是java的文件
70 rm -f *.java
```

VIM

```
1 i
2 a/A
3 o/O
4 r + ?替换
5
6 0: 文件当前行的开头
7 $: 文件当前行的末尾
8 G: 文件的最后一行开头
9 1 + G到第一行
10 9 + G到第九行 = :9
11
12 dd: 删除一行
13 3dd: 删除3行
14 yy: 复制一行
15 3yy: 复制3行
16 p: 粘贴
17 u: undo
18 ctrl + r: redo
19
20 "a剪切板a
21 "b剪切板b
22
23 "ap粘贴剪切板a的内容
24
25 每次进入vi就有行号
26 vi ~/.vimrc
27 set nu
28
29 :w a.txt另存为
30 :w >> a.txt内容追加到a.txt
31
32 :e! 恢复到最初状态
33
34 :1,$s/hadoop/root/g 将第一行到追后一行的hadoop替换为root
```

查找

```
1  1.查找可执行的命令:
2  which ls
3
4  2.查找可执行的命令和帮助的位置:
5  whereis ls
6
7  3.查找文件(需要更新库:update-db)
8  locate hadoop.txt
9
10 4.从某个文件夹开始查找
11 find / -name "hadoop*"
12 find / -name "hadoop*" -ls
13
14 5.查找并删除
15 find / -name "hadoop*" -ok rm {} \;
16 find / -name "hadoop*" -exec rm {} \;
17
18 6.查找用户为hadoop的文件
19 find /usr -user hadoop -ls
20
21 7.查找用户为hadoop并且(-a)拥有组为root的文件
22 find /usr -user hadoop -a -group root -ls
23
24 8.查找用户为hadoop或者(-o)拥有组为root并且是文件夹类型的文件
25 find /usr -user hadoop -o -group root -a -type d
26
27 9.查找权限为777的文件
28 find / -perm -777 -type d -ls
29
30 10.显示命令历史
31 history
32
33 11.grep
34 grep hadoop /etc/passwd
```

打包与压缩

```
1  1.gzip压缩
2  gzip a.txt
3
4  2.解压
5  gunzip a.txt.gz
6  gzip -d a.txt.gz
7
8  3.bzip2压缩
9  bzip2 a
10
11 4.解压
12 bunzip2 a.bz2
13 bzip2 -d a.bz2
```

```
14
15 5.将当前目录的文件打包
16 tar -cvf bak.tar .
17 将/etc/password追加文件到bak.tar中(r)
18 tar -rvf bak.tar /etc/password
19
20 6.解压
21 tar -xvf bak.tar
22
23 7.打包并压缩gzip
24 tar -zcvf a.tar.gz
25
26 8.解压缩
27 tar -zxvf a.tar.gz
28 解压到/usr/下
29 tar -zxvf a.tar.gz -C /usr
30
31 9.查看压缩包内容
32 tar -ztvf a.tar.gz
33
34 zip/unzip
35
36 10.打包并压缩成bz2
37 tar -jcvf a.tar.bz2
38
39 11.解压bz2
40 tar -jxvf a.tar.bz2
```

正则表达式

```
1 规则:
2 . : 任意一个字符
3 a* : 任意多个a(零个或多个a)
4 a? : 零个或一个a
5 a+ : 一个或多个a
6 .* : 任意多个任意字符
7 \. : 转义.
8 \<h.*p\> : 以h开头, p结尾的一个单词
9 o\{2\} : o重复两次
10
11 grep '^i.\{18\}n$' /usr/share/dict/words
12
13 查找不是以#开头的行
14 grep -v '^#' a.txt | grep -v '^$'
15
16 以h或r开头的
17 grep '^[hr]' /etc/passwd
18
19 不是以h和r开头的
20 grep '^[^hr]' /etc/passwd
21
22 不是以h到r开头的
23 grep '^[^h-r]' /etc/passwd
```

输入输出重定向

```
1 1.新建一个文件
2 touch a.txt
3 > b.txt
4
5 2.错误重定向:2>
6 find /etc -name zhaoxing.txt 2> error.txt
7
8 3.将正确或错误的信息都输入到log.txt中
9 find /etc -name passwd > /tmp/log.txt 2>&1
10 find /etc -name passwd &> /tmp/log.txt
11
12 4.追加>>
13
14 5.将小写转为大写（输入重定向）
15 tr "a-z" "A-Z" < /etc/passwd
16
17 6.自动创建文件
18 cat > log.txt << EXIT
19 > ccc
20 > ddd
21 > EXI
22
23 7.查看/etc下的文件有多少个？
24 ls -l /etc/ | grep '^d' | wc -l
25
26 8.查看/etc下的文件有多少个，并将文件详情输入到result.txt中
27 ls -l /etc/ | grep '^d' | tee result.txt | wc -l
```

进程控制

```
1 1.查看用户最近登录情况
2 last
3 lastlog
4
5 2.查看硬盘使用情况
6 df
7
8 3.查看文件大小
9 du
10
11 4.查看内存使用情况
12 free
13
14 5.查看文件系统
15 /proc
16
17 6.查看日志
18 ls /var/log/
19
20 7.查看系统报错日志
21 tail /var/log/messages
22
23 8.查看进程
24 top
25
26 9.结束进程
27 kill 1234
```


其他命令

远程文件复制：scp

scp 命令用于 Linux 之间复制文件和目录，scp 是 secure copy 的缩写是linux系统下基于ssh登陆进行安全的远程文件拷贝命令。

scp 是加密的，[rcp](#) 是不加密的，scp 是 rcp 的加强版。

使用scp命令要确保使用的用户具有可读取远程服务器相应文件的权限，否则scp命令是无法起作用的。

从本地复制到远程命令格式：

```

1  复制文件
2  scp local_file remote_username@remote_ip:remote_folder
3  或者
4  scp local_file remote_username@remote_ip:remote_file
5  或者
6  scp local_file remote_ip:remote_folder
7  或者
8  scp local_file remote_ip:remote_file
9
10 复制文件夹
11 scp -r local_folder remote_username@remote_ip:remote_folder
12 或者
13 scp -r local_folder remote_ip:remote_folder

```

实例：

```

1  scp /home/space/music/1.mp3 root@xiaoxiaoming.xyz:/home/root/others/music
2  scp /home/space/music/1.mp3 root@xiaoxiaoming.xyz:/home/root/others/music/001.mp3
3  scp /home/space/music/1.mp3 xiaoxiaoming.xyz:/home/root/others/music
4  scp /home/space/music/1.mp3 xiaoxiaoming.xyz:/home/root/others/music/001.mp3
5
6  scp -r /home/space/music/ root@xiaoxiaoming.xyz:/home/root/others/
7  scp -r /home/space/music/ xiaoxiaoming.xyz:/home/root/others/

```

从远程复制到本地：

```

1  scp root@xiaoxiaoming.xyz:/home/root/others/music /home/space/music/1.mp3
2  scp -r xiaoxiaoming.xyz:/home/root/others/ /home/space/music/

```

-P 参数来设置命令的端口号：

```

1  #scp 命令使用端口号 4588
2  scp -P 4588 remote@xiaoxiaoming.xyz:/usr/local/sin.sh /home/administrator

```

locate查找

locate命令会去保存文档和目录名称的数据库内，查找文件或目录。

一般情况我们只需要输入 `locate your_file_name` 即可查找指定文件。

参数：

- -d或--database= 配置locate指令使用的数据库。locate指令预设的数据库位于/var/lib/mlocate目录里，文档名为mlocate.db。

查找passwd文件，输入以下命令：

```
1 | locate passwd
```

locate与find的区别: find 是去硬盘找，locate 只在/var/lib/slocate资料库中找。

locate的速度比find快，它并不是真的查找，而是查数据库，一般文件数据库在/var/lib/mlocate/mlocate.db中，所以locate的查找并不是实时的，而是以数据库的更新为准，一般是系统自己维护，也可以手工升级数据库，命令为：

```
1 | updatedb
```

which命令

which查找\$PATH中设置命令及安装文件目录所在位置

```
1 | python@ubuntu:/var/lib/mlocate$ which locate
2 | /usr/bin/locate
```

echo

常见用法：

```
1 | python@ubuntu:~$ echo -e "hello\t\t world! "  解析转义字符
2 | hello          world!
3 | python@ubuntu:~$ echo -E "hello\t\t world! "  不解析转义字符
4 | hello\t\t world!
5 | python@ubuntu:~$ echo $a  输出环境变量
6 | b
```

设置或显示环境变量：export

在 shell 中执行程序时，shell 会提供一组环境变量。export 可新增，修改或删除环境变量，供后续执行的程序使用。export 的效力仅限于该次登陆操作。

```
1 | export [-fnp][变量名称]=[变量设置值]
```

参数说明：

- -f 代表[变量名称]中为函数名称。
- -n 删除指定的变量。变量实际上并未删除，只是不会输出到后续指令的执行环境中。
- -p 列出所有的shell赋予程序的环境变量。

```
1 | # export MYENV=7 //定义环境变量并赋值
2 | # export -p //列出当前的环境变量
```

修改主机名&ip地址

显示主机名：hostname

临时修改：hostname xxx

永久修改

对于Ubuntu 系统

```
1 | vim /etc/hostname
```

对于centos系统

```
1 | vim /etc/sysconfig/network
```

在此配置文件中添加一条HOSTNAME=node1

针对centos7系统，可以使用如下命令

```
1 | hostnamectl set-hostname xxx
```

一般需要重开shell甚至重启操作系统才能生效。

修改IP地址

ifconfig eth0 192.168.12.22(重启后无效)

```
1 | vim /etc/sysconfig/network-scripts/ifcfg-eth0
```

mount挂载

mount 挂载外部存储设备到文件系统中

```
1 | mkdir /mnt/cdrom 创建一个目录，用来挂载
2 | mount -t iso9660 -o ro /dev/cdrom /mnt/cdrom/
```

将设备/dev/cdrom挂载到 挂载点： /mnt/cdrom中

```
1 | umount /mnt/cdrom
```

ssh免密登陆

假如 A 要登陆 B

在A上操作：

首先生成密钥对

```
1 | ssh-keygen （提示时，直接回车即可）
```

再将A自己的公钥拷贝并追加到B的授权列表文件authorized_keys中

批量添加用户

与用户账号有关的系统文件

完成用户管理的工作本质都是对有关的系统文件进行修改，这些系统文件包括/etc/passwd, /etc/shadow, /etc/group等。

/etc/passwd记录用户的基本属性

它的内容类似下面的例子：

```
1 # cat /etc/passwd
2
3 root:x:0:0:Superuser:/:
4 daemon:x:1:1:System daemons:/etc:
5 bin:x:2:2:Owner of system commands:/bin:
6 sys:x:3:3:Owner of system files:/usr/sys:
7 adm:x:4:4:System accounting:/usr/adm:
8 uucp:x:5:5:UUCP administrator:/usr/lib/uucp:
9 auth:x:7:21:Authentication administrator:/tcb/files/auth:
10 cron:x:9:16:Cron daemon:/usr/spool/cron:
11 listen:x:37:4:Network daemon:/usr/net/nls:
12 lp:x:71:18:Printer administrator:/usr/spool/lp:
13 sam:x:200:50:Sam san:/home/sam:/bin/sh
```

/etc/passwd中一行记录对应着一个用户，每行记录又被冒号(:)分隔为7个字段，其格式和具体含义如下：

```
1 | 用户名:口令:用户标识号:组标识号:注释性描述:主目录:登录Shell
```

用户名：

通常长度不超过8个字符，由大小写字母和/或数字组成，不能有冒号(:)。登录名中不能有冒号(:)，因为冒号在这里是分隔符。

为了兼容起见，登录名中最好不要包含点字符(.)，并且不使用连字符(-)和加号(+)打头。

口令：

本身存放用户口令的加密串，但现在许多Linux 系统都使用了shadow技术，把真正的加密后的用户口令字存放在/etc/shadow文件中，而/etc/passwd文件的口令字段中只存放一个特殊的字符，例如“x”或者“*”。

用户标识号：

是一个整数，系统内部用它来标识用户。一般情况下它与用户名是一一对应的，如果几个用户名对应的用户标识号是一样的，系统内部将把它们视为同一个用户，但是它们可以有不同的口令、不同的主目录以及不同的登录Shell等。

通常用户标识号的取值范围是0 ~ 65 535。0是超级用户root的标识号，1 ~ 99由系统保留，作为管理账号，普通用户的标识号从100开始。

组标识号：

记录用户所属的用户组，对应着/etc/group文件中的一条记录。

注释性描述：

一段任意编写的注释，创建账户时可以通过 `useradd -c 用户名` 的 `-c` 参数指定。

主目录：

用户的起始工作目录，用户在登录到系统之后所处的目录。

登录Shell：

用户登录后，要启动一个进程，负责将用户的操作传给内核，这个进程是用户登录到系统后运行的命令解释器或某个特定的程序，即Shell。

Shell是用户与Linux系统之间的接口。Linux的Shell有许多种，每种都有不同的特点。常用的有sh(Bourne Shell), csh(C Shell), ksh(Korn Shell), tcsh(TENEX/TOPS-20 type C Shell), bash(Bourne Again Shell)等。

可以通过usermod的 `-s` 参数为用户指定某个Shell。如果useradd不通过 `-s` 参数指定shell，那么系统使用bash为默认的登录Shell，即这个字段的值为/bin/bash。

为用户的登录指定特定的Shell可以限制用户只能运行指定的应用程序，在该应用程序运行结束后，用户就自动退出了系统。不过大部分Linux系统要求只有在系统中登记过的shell才能出现在这个字段中。

伪用户 (pseudo users)

这些用户的登陆shell为/usr/sbin/nologin，即不能登录。它们的存在主要是方便系统管理，满足相应的系统进程对文件属主的要求。

常见的伪用户如下所示：

- | | | |
|---|---------------|--------------|
| 1 | bin | 拥有可执行的用户命令文件 |
| 2 | sys | 拥有系统文件 |
| 3 | adm | 拥有帐户文件 |
| 4 | uucp | UUCP使用 |
| 5 | lp | lp或lpd子系统使用 |
| 6 | nobody | NFS使用 |

还有许多标准的伪用户，例如：audit, cron, mail, usenet等，它们也都各自为相关的进程和文件所需要。

/etc/shadow

对安全性要求较高的Linux系统都把/etc/passwd文件中的口令字段保存在/etc/shadow文件中，超级用户才拥有该文件读权限。

/etc/shadow中的记录行与/etc/passwd中的——对应，它由pwconv命令根据/etc/passwd中的数据自动产生
字段是：

- | | |
|---|--|
| 1 | 登录名:加密口令:最后一次修改时间:最小时间间隔:最大时间间隔:警告时间:不活动时间:失效时间:标志 |
|---|--|

1. "登录名"是与/etc/passwd文件中的登录名相一致的用户账号
2. "口令"字段存放的是加密后的用户口令字，长度为13个字符。如果为空，则对应用户没有口令，登录时不需要口令；如果含有不属于集合 { . / 0 - 9 A - Z a - z } 中的字符，则对应的用户不能登录。
3. "最后一次修改时间"表示的是从某个时刻起，到用户最后一次修改口令时的天数。大部分linux系统的时间起点是1970年1月1日。
4. "最小时间间隔"指的是两次修改口令之间所需的最小天数。
5. "最大时间间隔"指的是口令保持有效的最大天数。
6. "警告时间"字段表示的是从系统开始警告用户到用户密码正式失效之间的天数。
7. "不活动时间"表示的是用户没有登录活动但账号仍能保持有效的最大天数。
8. "失效时间"字段给出的是一个绝对的天数，如果使用了这个字段，那么就给出相应账号的生存期。期满后，该账号就不再是一个合法的账号，也就不能再用来登录了。

下面是/etc/shadow的一个例子：

```

1 | # cat /etc/shadow
2 |
3 | root:Dnakfw28zf38w:8764:0:168:7:::
4 | daemon:*::0:0:::
5 | bin:*::0:0:::
6 | sys:*::0:0:::
7 | adm:*::0:0:::
8 | uucp:*::0:0:::
9 | nuucp:*::0:0:::
10 | auth:*::0:0:::
11 | cron:*::0:0:::
12 | listen:*::0:0:::
13 | lp:*::0:0:::
14 | sam:EkdiSECLWPdSa:9740:0:0:::

```

/etc/group记录用户组信息

每个用户都属于某个用户组；一个组中可以有多个用户，一个用户也可以属于不同的组。

当一个用户同时是多个组中的成员时，在/etc/passwd文件中记录的是用户所属的主组，也就是登录时所属的默认组，而其他组称为附加组。

用户要访问属于附加组的文件时，必须首先使用newgrp命令使自己成为所要访问的组中的成员。

用户组的所有信息都存放在/etc/group文件中，字段有：

```

1 | 组名:口令:组标识号:组内用户列表

```

1. "组名"是用户组的名称，由字母或数字构成。与/etc/passwd中的登录名一样，组名不应重复。
2. "口令"字段存放的是用户组加密后的口令字。一般Linux系统的用户组都没有口令，即这个字段一般为空，或者是*。
3. "组标识号"与用户标识号类似，也是一个整数，被系统内部用来标识组。
4. "组内用户列表"是属于这个组的所有用户的列表，不同用户之间用逗号(,)分隔。这个用户组可能是用户的主组，也可能是附加组。

/etc/group文件的一个例子如下：

```

1 | root::0:root
2 | bin::2:root,bin
3 | sys::3:root,uucp
4 | adm::4:root,adm
5 | daemon::5:root,daemon
6 | lp::7:root,lp
7 | users::20:root,sam

```

实操

先编辑一个文本用户文件

每一列按照 /etc/passwd 密码文件的格式书写，要注意每个用户的用户名、UID、宿主目录都不可以相同，其中密码栏可以留做空白或输入x号。一个范例文件user.txt内容如下：

```
1 user001::601:100:user:/home/user001:/bin/bash
2 user002::602:100:user:/home/user002:/bin/bash
3 user003::603:100:user:/home/user003:/bin/bash
4 user004::604:100:user:/home/user004:/bin/bash
5 user005::605:100:user:/home/user005:/bin/bash
6 user006::606:100:user:/home/user006:/bin/bash
```

执行 `/usr/sbin/newusers` 命令

需要root权限：

```
1 # newusers < user.txt
```

可以执行命令 `vipw` 或 `vi /etc/passwd` 检查 `/etc/passwd` 文件是否已经出现这些用户的数据，并且用户的宿主目录是否已经创建。

取消 `shadow password` 功能

将 `/etc/shadow` 产生的 `shadow` 密码解码，然后回写到 `/etc/passwd` 中，并将 `/etc/shadow` 的 `shadow` 密码栏删掉。这是为了方便下一步的密码转换工作，即先取消 `shadow password` 功能。

执行 `/usr/sbin/pwunconv` 命令：

```
1 # pwunconv
```

编辑每个用户的密码对照文件

格式为：

```
1 用户名:密码
```

实例文件 `passwd.txt` 内容如下：

```
1 user001:123456
2 user002:123456
3 user003:123456
4 user004:123456
5 user005:123456
6 user006:123456
```

执行 `/usr/sbin/chpasswd` 命令

需要root权限：

创建用户密码，`chpasswd` 会将经过 `/usr/bin/passwd` 命令编码过的密码写入 `/etc/passwd` 的密码栏。

```
1 # chpasswd < passwd.txt
```

将密码编码为 `shadow password`

执行命令 `/usr/sbin/pwconv` 将密码编码为 `shadow password`，并将结果写入 `/etc/shadow`。

```
1 # pwconv
```

这样就完成了大量用户的创建了，之后您可以到/home下检查这些用户宿主目录的权限设置是否都正确，并登录验证用户密码是否正确。

完整步骤

先准备好用户文件user和密码文件passwd

然后运行：

```
1 newusers < user
2 pwunconv
3 chpasswd < passwd
4 pwconv
```

crontab的使用

crontab命令是cron table的简写，它是cron的配置文件，也可以叫它作业列表。

相关配置文件如下：

- /var/spool/cron/ 目录下存放的是每个用户包括root的crontab任务，每个任务以创建者的名字命名
- /etc/crontab 这个文件负责调度各种管理和维护任务。
- /etc/cron.d/ 这个目录用来存放任何要执行的crontab文件或脚本。
- 还可以把脚本放在/etc/cron.hourly、/etc/cron.daily、/etc/cron.weekly、/etc/cron.monthly目录中，让它每小时/天/星期、月执行一次。

命令格式：

```
1 crontab [ -u user ] { -l | -r | -e }
2 //省略用户表表示操作当前用户的crontab
3     -e      (编辑工作表)
4     -l      (列出工作表里的命令)
5     -r      (删除工作表)
```

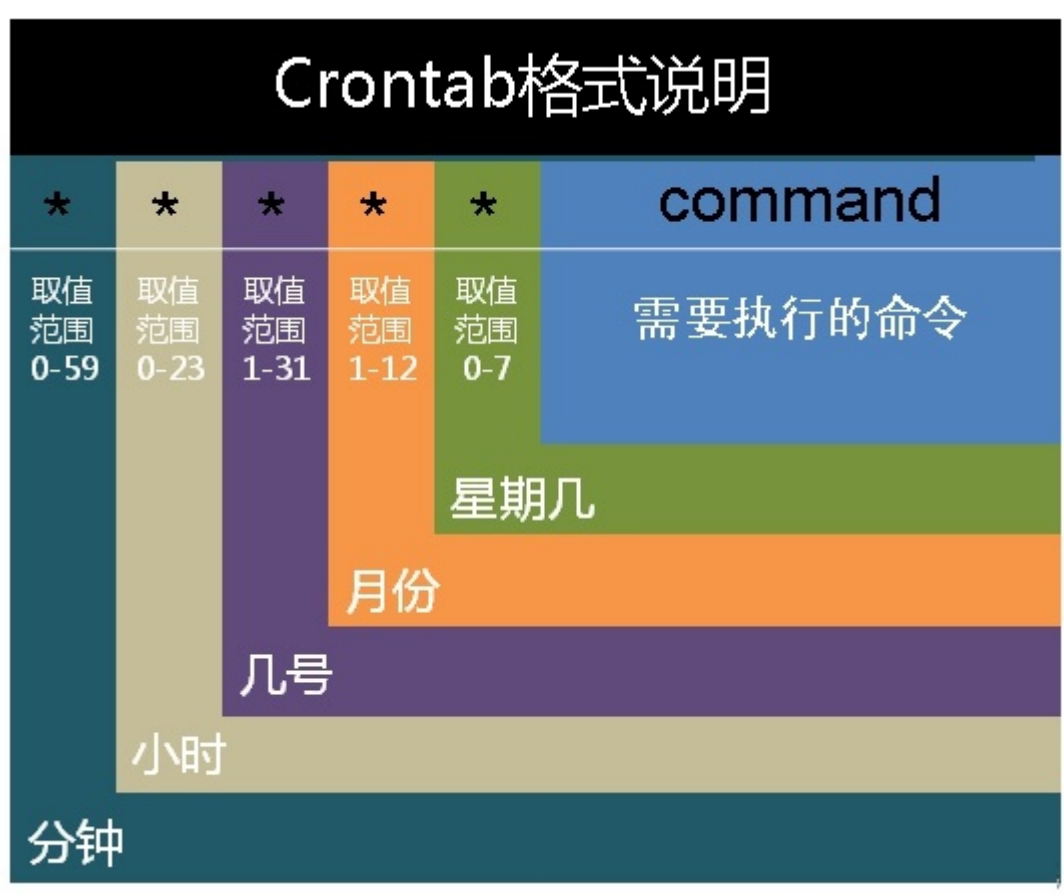
crontab -e进入当前用户的工作表编辑，是常见的vim界面。每行是一条命令。

crontab的命令构成为 时间+动作，其时间有分、时、日、月、周五种，操作符有

- * 取值范围内的所有数字
- / 每过多少个数字
- - 从X到Z
- , 散列数字

基本格式：

```
1 f1 f2 f3 f4 f5 command
2 分 时 日 月 周 命令
3 第1列表示分钟0~59 每分钟用*或者 */1表示
4 第2列表示小时0~23 (0表示0点)
5 第3列表示日期1~31
6 第4列表示月份1~12
7 第5列标识号星期0~6 (0表示星期天)
8 第6列要运行的命令
```

- 其中 f1 是表示分钟，f2 表示小时，f3 表示一个月份中的第几日，f4 表示月份，f5 表示一个星期中的第几天。command表示要执行的命令。
- 当 f1 为 * 时表示每分钟都要执行 program，f2 为 * 时表示每小时都要执行程序，以此类推
- 当 f1 为 a-b 时表示从第 a 分钟到第 b 分钟这段时间内要执行，f2 为 a-b 时表示从第 a 到第 b 小时都要执行，以此类推
- 当 f1 为 */n 时表示每 n 分钟个时间间隔执行一次，f2 为 */n 表示每 n 小时个时间间隔执行一次，以此类推
- 当 f1 为 a, b, c,... 时表示第 a, b, c,... 分钟要执行，f2 为 a, b, c,... 时表示第 a, b, c,... 个小时要执行，以此类推

在 12 月内, 每天的早上 6 点到 12 点, 每隔 3 个小时 0 分钟执行一次 /usr/bin/backup

```
1 | 0 6-12/3 * 12 * /usr/bin/backup
```

周一到周五每天下午 5:00 寄一封信给 alex@domain.name

```
1 | 0 17 * * 1-5 mail -s "hi" alex@domain.name < /tmp/maildata
```

每月每天的午夜 0 点 20 分, 2 点 20 分, 4 点 20 分....执行 echo "haha"

```
1 | 20 0-23/2 * * * echo "haha"
```

示例1:

```

1 45 4 1,10,22 * * /etc/init.d/smb restart 每月1、10、22日的4 : 45重启smb
2
3 10 1 * * 6,0 /etc/init.d/smb restart 每周六、周日的1 : 10重启smb
4
5 0,30 18-23 * * * /etc/init.d/smb restart 每天18 : 00至23 : 00之间每隔30分钟重启smb
6
7 0 23 * * 6 /etc/init.d/smb restart 每星期六的晚上11 : 00 pm重启smb
8
9 * */1 * * * /etc/init.d/smb restart 每一小时重启smb
10
11 * 23-7/1 * * * /etc/init.d/smb restart 晚上11点到早上7点之间, 每隔一小时重启smb

```

示例2:

```

1 0 */2 * * * /sbin/service httpd restart 每两个小时重启一次apache
2
3 50 7 * * * /sbin/service sshd start 每天7: 50开启ssh服务
4
5 50 22 * * * /sbin/service sshd stop 每天22: 50关闭ssh服务
6
7 0 0 1,15 * * fsck /home 每月1号和15号检查/home 磁盘
8
9 1 * * * * /home/bruce/backup 每小时的第一分执行 /home/bruce/backup这个文件
10
11 00 03 * * 1-5 find /home "*.xxx" -mtime +4 -exec rm {} \; 每周一至周五3点钟, 在目录/home
    中, 查找文件名为*.xxx的文件, 并删除4天前的文件。
12
13 30 6 */10 * * ls 每月的1、11、21、31日是6: 30执行一次ls命令

```

环境变量问题:

有时创建了一个crontab, 但是这个任务却无法自动执行, 而手动执行这个任务却没有问题, 这种情况一般是由于在crontab文件中没有配置环境变量引起的。

所以注意如下3点:

- 1) 脚本中涉及文件路径时写全局路径;
- 2) 脚本执行要用到java或其他环境变量时, 通过source命令引入环境变量, 如:

```

1 cat start_cbp.sh
2 #!/bin/sh
3 source /etc/profile
4 export RUN_CONF=/home/d139/conf/platform/cbp/cbp_jboss.conf
5 /usr/local/jboss-4.0.5/bin/run.sh -c mev &

```

- 3) 当手动执行脚本OK, 但是crontab死活不执行时, 可以尝试在crontab中直接引入环境变量解决问题。如:

```

1 0 * * * * . /etc/profile; bin/sh /var/www/java/audit_no_count/bin/restart_audit.sh

```

特殊权限

linux共12位权限, 除了9位基础权限还有3个特殊权限。

三种特殊的权限

SetUID(suid)

命令功能：临时使用命令的属主权限执行该命令。即如果文件有suid权限时，那么普通用户去执行该文件时，会以该文件的所属用户的身份去执行。

SetUID（简写suid）：会在属主权限位的执行权限上写个s。如果该属主权限位上有执行权限，则会在属主权限位的执行权限上写个s（小写）；如果该属主权限位上没有执行权限，则会在属主权限位的执行权限上写个S（大写）。

suid数字权限是4000,设置方法：

```
1 方式1:
2 [root@centos7 ~]# chmod u+s filename
3 方式2:
4 [root@centos7 ~]# chmod 4755 filename
```

查看passwd命令的权限

```
1 `[root@localhost ftl]` `# ll /usr/bin/passwd ` `问题： ``passwd``文件的属组是root,表示只有root用户可以访问的文件，为什么普通用户依然可以使用该命令更改自己的密码？ ``答案：当普通用户[omd]使用``passwd``命令的时候，系统看到``passwd``命令文件的属性有大写s后，表示这个命令的属主权限被omd用户获得，也就是omd用户获得文件``/etc/shadow``的root的rwx权限`
```

由于passwd具有s权限，普通用户使用该命令的时候，就会以该命令的属主身份root执行该命令，于是能够顺利修改普通用户不具备修改权限的/etc/shadow文件。

希望普通用户user1可以删除某个自己没有权限删除的文件的操作方法：

- sudo给用户1授权rm权限
- rm设置suid
- 修改被删除文件上级目录的权限

SetUID（简称suid）总结：

1. 让普通用户对可执行的二进制文件，临时拥有二进制文件的属主权限；
2. 如果设置的二进制文件没有执行权限，那么suid的权限显示就是S（大写字母S）；
3. 特殊权限suid仅对二进制可执行程序有效，其他文件或目录则无效。
4. suid极其危险，如果给vim或者rm命令设置了setUID，那么任何文件都能编辑或者删除了，相当于有root权限了。

setGID (sgid)

命令功能：使用sgid可以使得多个用户之间共享一个目录的所有文件变得简单。当某个目录设置了sgid后，在该目录中新建的文件不在是创建该文件的默认所属组。

如果该属组权限位上有执行权限，则会在属组主权限位的执行权限上写个s（小写字母）；如果该属组权限位上没有执行权限，则会在属组主权限位的执行权限上写个S（大写字母S）。

write命令的权限：

```
1 [root@VM_0_9_centos ~]# ll /bin/write
2 -rwxr-sr-x 1 root tty 19544 Aug  9 11:10 /bin/write
```

sgid数字权限是2000，设置方法：

```
1 方式1:
2  [root@VM_0_9_centos ~]# chmod 2755 test/
3  方式2:
4  [root@VM_0_9_centos ~]# chmod g+s test/
5
6  [root@VM_0_9_centos ~]# ll -d test/
7  drwxr-sr-x 2 root root 4096 Nov 22 21:02 test/
```

在设置SetGID的文件夹创建文件的属组是父目录的属组：

```
1  [root@VM_0_9_centos ~]# cd test/
2  [root@VM_0_9_centos test]# su aaa
3  [aaa@VM_0_9_centos test]$ touch bbb
4  [aaa@VM_0_9_centos test]$ ll
5  -rw-rw-r-- 1 aaa  root 0 Nov 22 21:14 bbb
```

sticky(sbit)粘滞位

命令功能：粘滞位，只对目录有效，对某目录设置粘滞位后，普通用户就算有w权限也只能删除该目录下自己建立的文件，而不能删除其他用户建立的文件。

如果该其他用户权限位上有执行权限，则会在其他用户权限位的执行权限上写个t（小写）；如果该其它用户权限位上没有执行权限，则会在其他用户权限位的执行权限上写个T（大写）。

系统中存在的/tmp目录是经典的粘滞位目录，谁都有写权限，因此安全问题，常常是木马第一手跳板。

```
1  [aaa@VM_0_9_centos ~]$ ll -d /tmp/
2  drwxrwxrwt. 9 root root 4096 Nov 22 21:15 /tmp/
```

sbit数字权限是1000，设置方法：

```
1  方法1:
2  [root@VM_0_9_centos ~]# chmod 1755 test/
3  方法2:
4  [root@VM_0_9_centos ~]# chmod o+t test/
5  查看权限:
6  [root@VM_0_9_centos ~]# ll -d test/
7  drwxr-xr-t 2 root root 4096 Nov 22 21:15 test/
```

chattr权限

chattr概述：凌驾于r、w、x、suid、sgid之上的权限。

lsattr：查看特殊权限

```
1  [root@VM_0_9_centos ~]# lsattr /etc/passwd
2  -----e-- /etc/passwd
```

chattr：设置特殊权限

权限说明	
-i	锁定文件，不能编辑，不能修改，不能删除，不能移动，可以执行
-a	仅可以追加文件，不能编辑，不能删除，不能移动，可以执行

防止系统中某个关键文件被修改：

```
1 [root@VM_0_9_centos ~]# chattr +i /etc/fstab
2 [root@VM_0_9_centos ~]# lsattr /etc/fstab
3 ----i-----e-- /etc/fstab
```

让某个文件只能往里面追加内容，不能删除，一些日志文件适用于这种操作：

```
1 [root@VM_0_9_centos ~]# chattr +a user_act.log
2 [root@VM_0_9_centos ~]# lsattr user_act.log
3 -----a-----e-- user_act.log
```

掩码umask

umask的作用

umask值用于设置用户在创建文件时的默认权限，当我们在系统中创建目录或文件时，目录或文件所具有的默认权限就是由umask值决定的。

对于root用户，系统默认的umask值是0022；对于普通用户，系统默认的umask值是0002。执行umask命令可以查看当前用户的umask值。

```
1 [root@VM_0_9_centos ~]# umask
2 0022
```

umask是如何改变新文件的权限

umask值一共有4组数字，其中第1组数字用于定义特殊权限，一般不予考虑，与一般权限有关的是后3组数字。

默认情况下，对于目录，用户所能拥有的最大权限是777；对于文件，用户所能拥有的最大权限是目录的最大权限去掉执行权限，即666。因为x执行权限对于目录是必须的，没有执行权限就无法进入目录，而对于文件则不必默认赋予x执行权限。

对于root用户，他的umask值是022。当root用户创建目录时，默认的权限就是用最大权限777去掉相应位置的umask值权限，即对于所有者不必去掉任何权限，对于所属组要去掉w权限，对于其他用户也要去掉w权限，所以目录的默认权限就是755；当root用户创建文件时，默认的权限则是用最大权限666去掉相应位置的umask值，即文件的默认权限是644。

通过umask命令可以修改umask值，比如将umask值设为0077。

```
1 [root@VM_0_9_centos ~]# umask 0077
2 [root@VM_0_9_centos ~]# umask
3 0077
```

永久修改umask

umask命令只能临时修改umask值，系统重启之后umask将还原成默认值。如果要永久修改umask值，可修改 `/etc/bashrc` 或 `/etc/profile` 文件。

例如要将默认umask值设置为027，那么可以在文件中增加一行 `umask 027`。

linux软件安装

Ubuntu软件安装与卸载

更新Ubuntu软件下载地址

开源软件镜像站：<https://mirrors.tuna.tsinghua.edu.cn/help/ubuntu/>

Ubuntu 的软件源配置文件是 `/etc/apt/sources.list`。将系统自带的该文件做个备份，将该文件替换为下面内容，即可使用 TUNA 的软件源镜像。

ubuntu版本: 16.04 LTS

```
1 # 默认注释了源码镜像以提高 apt update 速度，如有需要可自行取消注释
2 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial main restricted universe
  multiverse
3 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial main restricted universe
  multiverse
4 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-updates main restricted
  universe multiverse
5 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-updates main restricted
  universe multiverse
6 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-backports main restricted
  universe multiverse
7 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-backports main restricted
  universe multiverse
8 deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-security main restricted
  universe multiverse
9 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-security main restricted
  universe multiverse
10
11 # 预发布软件源，不建议启用
12 # deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-proposed main restricted
  universe multiverse
13 # deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ xenial-proposed main restricted
  universe multiverse
```

然后

```
1 | sudo cp /etc/apt/sources.list /etc/apt/sources.list.backup
```

再sudo vim /etc/apt/sources.list修改为以上内容

Ubuntu软件操作的相关命令

```
1 sudo apt-get update 更新源
2 sudo apt-get install package 安装包
3 sudo apt-get remove package 删除包
4 sudo apt-cache search package 搜索软件包
5 sudo apt-cache show package 获取包的相关信息，如说明、大小、版本等
6 sudo apt-get install package --reinstall 重新安装包
7 sudo apt-get -f install 修复安装
8 sudo apt-get remove package --purge 删除包，包括配置文件等
9 sudo apt-get build-dep package 安装相关的编译环境
10 sudo apt-get upgrade 更新已安装的包
11 sudo apt-get dist-upgrade 升级系统
12 sudo apt-cache depends package 了解使用该包依赖那些包
13 sudo apt-cache rdepends package 查看该包被哪些包依赖
14 sudo apt-get source package 下载该包的源代码
15 sudo apt-get clean && sudo apt-get autoclean 清理无用的包
16 sudo apt-get check 检查是否有损坏的依赖
```

yum安装命令

yum (Yellow dog Updater, Modified) 是一个在Fedora和RedHat以及SUSE中的Shell前端软件包管理器。

基於RPM包管理，能够从指定的服务器自动下载RPM包并且安装，可以自动处理依赖性关系，并且一次安装所有依赖的软体包，无须繁琐地一次次下载、安装。

yum提供了查找、安装、删除某一个、一组甚至全部软件包的命令，而且命令简洁而又好记。

更新国内yum源

网易 (163) yum源是国内最好的yum源之一，无论是速度还是软件版本，都非常的不错。

将yum源设置为163 yum，可以提升软件包安装和更新的速度，同时避免一些常见软件版本无法找到。

首先备份/etc/yum.repos.d/CentOS-Base.repo

```
1 mv /etc/yum.repos.d/CentOS-Base.repo /etc/yum.repos.d/CentOS-Base.repo.backup
```

下载对应版本 repo 文件, 放入 /etc/yum.repos.d/

- CentOS5 : <http://mirrors.163.com/.help/CentOS5-Base-163.repo>
- CentOS6 : <http://mirrors.163.com/.help/CentOS6-Base-163.repo>
- CentOS7 : <http://mirrors.163.com/.help/CentOS7-Base-163.repo>

```
1 wget http://mirrors.163.com/.help/CentOS7-Base-163.repo
2 mv CentOS7-Base-163.repo /etc/yum.repos.d/CentOS-Base.repo
```

运行以下命令生成缓存

```
1 yum clean all
2 yum makecache
```

除了网易之外，国内还有其他不错的 yum 源，比如中科大和搜狐。

中科大的 yum 源，安装方法查看: <https://lug.ustc.edu.cn/wiki/mirrors/help/centos>

sohu 的 yum 源安装方法查看: <http://mirrors.sohu.com/help/centos.html>

yum常用命令

yum 语法:

```
1 | yum [options] [command] [package ...]
```

选项:

- **options:** 可选, 选项包括-h (帮助), -y (当安装过程提示选择全部为"yes"), -q (不显示安装的过程) 等等。
- **command:** 要进行的操作。
- **package:** 操作的对象。

实例:

- 列出所有可更新的软件清单命令: `yum check-update`
- 更新所有软件命令: `yum update`
- 仅安装指定的软件命令: `yum install <package_name>`
- 仅更新指定的软件命令: `yum update <package_name>`
- 显示包信息: `yum info <package_name>`
- 列出所有可安装的软件清单命令: `yum list`
- 删除软件包命令: `yum remove <package_name>`
- 查找软件包 命令: `yum search <keyword>`
- 清除缓存命令:
 - `yum clean packages`: 清除缓存目录下的软件包
 - `yum clean headers`: 清除缓存目录下的 headers
 - `yum clean oldheaders`: 清除缓存目录下旧的 headers
 - `yum clean, yum clean all (= yum clean packages; yum clean oldheaders)`: 清除缓存目录下的软件包及旧的 headers

yum在线安装MySQL5.7

Step1: 检测系统是否自带安装mysql

```
1 | yum list installed | grep mysql
```

Step2: 删除系统自带的mysql及其依赖

```
1 | yum -y remove mysql-libs.x86_64
```

Step3: 给CentOS添加rpm源, 并且选择较新的源

```
1 | wget dev.mysql.com/get/mysql-community-release-el7-5.noarch.rpm
2 | yum localinstall mysql-community-release-el7-5.noarch.rpm
3 | yum repolist all | grep mysql
4 | yum-config-manager --disable mysql55-community
5 | yum-config-manager --disable mysql56-community
6 | yum-config-manager --enable mysql57-community-dmr
7 | yum repolist enabled | grep mysql
```


Step4:安装mysql 服务器

```
1 | yum install mysql-community-server
```

Step5: 启动mysql

```
1 | service mysqld start
```

grep "password" /var/log/mysqld.log(查看临时密码)

```
1 | SET PASSWORD = PASSWORD('your new password');
2 | ALTER USER 'root'@'localhost' PASSWORD EXPIRE NEVER;
3 | flush privileges;
```

默认的要求必须的设置格式:

包含数字、小写或大写字母以及特殊字符

默认的要求必须的设置格式:

包含数字、小写或大写字母以及特殊字符

如果不想复杂, 可以使用以下方式

```
1 | set global validate_password_policy=0;
2 | set global validate_password_length=1;
```

Step6: 查看mysql是否自启动,并且设置开启自启动

```
1 | # chkconfig --list | grep mysqld
2 | # chkconfig mysqld on
```

Step7: mysql安全设置

```
1 | mysql_secure_installation
```

rpm

RPM是Red Hat公司随Redhat Linux推出了一个软件包管理器, 通过它能够更加轻松容易地实现软件的安装。

常见用法:

- rpm -ivh <rpm包名> 安装软件
- rpm -e <rpm包名> 卸载安装
- rpm -qi <rpm包名> 显示软件安装信息
- rpm -qa | grep xxx 查询软件是否安装 (包括相关依赖)
- rpm -Uvh <rpm包名> 升级一个rpm

具体参数详解:

- i, --install 安装包
- v, --verbose 列出更多详细信息, 安装进度
- h, --hash 安装时列出hash标记 (与 -v连用)
- e, --erase 卸载安装包
- U, --upgrade=+ 升级包
 - replacepkge 无论软件包是否已被安装, 都强行安装软件包
 - test 安装测试, 并不实际安装

--nodeps 忽略软件包的依赖关系强行安装
--force 忽略软件包及文件的冲突
-q,--query:
 -a, --all 查询/校验所有的安装包
 -p, --package 查询/校验一个安装文件
 -l, --list 列出安装文件
 -d, --docfiles 列出所有文档文件
 -f, --file 查询/校验安装包中所包含的文件

安装软件:

```
1 # rpm -hvi dejagnu-1.4.2-10.noarch.rpm
2 警告: dejagnu-1.4.2-10.noarch.rpm: V3 DSA 签名: NOKEY, key ID db42a60e
3 准备...
4 ##### [100%]
```

显示软件安装信息:

```
1 # rpm -qi dejagnu-1.4.2-10.noarch.rpm
2
3 【第1次更新 教程、类似命令关联】
```

Linux的基本配置

1.修改主机名

```
1 vi /etc/sysconfig/network
2 NETWORKING=yes
3 HOSTNAME=hadoop1
```

2.修改ip地址

```
1 vi /etc/sysconfig/network-scripts/ifcfg-eth0
2 DEVICE=eth0
3 TYPE=Ethernet
4 ONBOOT=yes
5 BOOTPROTO=static
6 IPADDR=192.168.100.101
7 NETMASK=255.255.255.0
8
9 service network restart
```

3.修改ip地址和主机名的映射关系

```
1 vi /etc/hosts
2 127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
3 ::1          localhost localhost.localdomain localhost6 localhost6.localdomain6
4 192.168.100.101 hadoop1
```

4.关闭iptables并设置其开机启动/不启动

```
1 service iptables stop
2 chkconfig iptables on
3 chkconfig iptables off
```

安装JDK

```
1 1. 上传jdk-7u45-linux-x64.tar.gz到Linux上
2 2. 解压jdk到/usr/local目录
3 tar -zxvf jdk-7u45-linux-x64.tar.gz -C /usr/local/
4 3. 设置环境变量, 在/etc/profile文件最后追加相关内容(技巧r:!pwd)
5 vi /etc/profile
6 export JAVA_HOME=/usr/local/jdk1.7.0_45
7 export PATH=$PATH:$JAVA_HOME/bin
8 4. 刷新环境变量
9 source /etc/profile
10 5. 测试java命令是否可用
11 java -version
```

制作本地YUM源

上传CentOS-6.7-x86_64-bin-DVD1.iso到服务器

将CentOS-6.7-x86_64-bin-DVD1.iso镜像挂载到某个目录:

```
1 mkdir /var/iso
2 mount -o loop CentOS-6.7-x86_64-bin-DVD1.iso /var/iso
```

安装并启动Apache服务器:

```
1 yum install -y httpd
2 service httpd start
```

使用浏览器访问<http://192.168.100.101> (如果访问不通, 检查防火墙是否开启了80端口或关闭防火墙)

将YUM源配置到httpd中:

```
1 cp -r /var/iso/ /var/www/html/CentOS-6.7
2 umount /var/iso
```

在浏览器中访问<http://192.168.100.101/CentOS-6.7/>

Index of /CentOS-6.7

Name	Last modified	Size	Description
 Parent Directory		-	
 CentOS_BuildTag	28-Feb-2016 06:29	14	
 EFI/	28-Feb-2016 06:29	-	
 EULA	28-Feb-2016 06:29	212	
 GPL	28-Feb-2016 06:29	18K	
 Packages/	28-Feb-2016 06:30	-	
 RELEASE-NOTES-en-US.html	28-Feb-2016 06:30	1.3K	
 RPM-GPG-KEY-CentOS-6	28-Feb-2016 06:30	1.7K	
 RPM-GPG-KEY-CentOS-Debug-6	28-Feb-2016 06:30	1.7K	
 RPM-GPG-KEY-CentOS-Security-6	28-Feb-2016 06:30	1.7K	
 RPM-GPG-KEY-CentOS-Testing-6	28-Feb-2016 06:30	1.7K	
 TRANS.TBL	28-Feb-2016 06:30	3.3K	
 images/	28-Feb-2016 06:29	-	
 isolinux/	28-Feb-2016 06:29	-	
 repodata/	28-Feb-2016 06:30	-	

Apache/2.2.15 (CentOS) Server at 192.168.0.100 Port 80

配置使用YUM源:

备份原有的YUM源的配置文件

```
1 | cd /etc/yum.repos.d/
2 | rename .repo .repo.bak *
```

修改YUM源配置文件

```
1 | vi CentOS-Local.repo
2 | [base]
3 | name=CentOS-Local
4 | baseurl=http://192.168.100.101/CentOS-6.7
5 | gpgcheck=1
6 | enabled=1    #很重要, 1才启用
7 | gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-CentOS-6
```

重建yum缓存

```
1 | #清除yum缓存文件, 重新新建
2 | yum clean all && yum makecache
3 | #列出可用的YUM源
4 | yum repolist
```

rpm包生成yum源目录

如果已经下载好了rpm包, 可以自行制作一个yum源 (yum仓库)。将下载的rpm包上传到centos服务器上 (比如/data/rpm目录下), 然后进入存放rpm包的目录, 执行以下命令:

```
1 | # cd /data/rpm
2 | # createrepo .
```

这样，rpm包存放的目录就可以作为yum源目录使用。

如果提示找不到createrepo命令，可以使用yum install createrepo安装该程序。

Shell编程

这里说的Shell 脚本（shell script），是在Linux 环境下运行的脚本程序

Shell 编程跟 JavaScript、php 编程一样，只要有一个能编写代码的文本编辑器和一个能解释执行的脚本解释器就可以了。

Linux 的 Shell 种类众多，常见的有：

- Bourne Shell（/usr/bin/sh或/bin/sh）
- Bourne Again Shell（/bin/bash）
- C Shell（/usr/bin/csh）
- K Shell（/usr/bin/ksh）
- Shell for Root（/sbin/sh）
-

Bash是大多数Linux 系统默认的 Shell，本文也仅关注Bash Shell。

在一般情况下，并不区分 Bourne Shell 和 Bourne Again Shell，所以，像 **#!/bin/sh**，它同样也可以改为 **#!/bin/bash**。

#! 告诉系统其后路径所指定的程序即是解释此脚本文件的 Shell 程序。

入门

运行Shell脚本

编写shell脚本：

```
1 | vi test.sh
2 |
3 | #!/bin/bash
4 | echo "Hello world !"
```

#! 是一个约定的标记，它告诉系统这个脚本需要什么解释器来执行，即使用哪一种 Shell。

echo 命令用于向窗口输出文本。

运行 Shell 脚本有两种方法：

1、作为可执行程序

```
1 | chmod +x ./test.sh #使脚本具有执行权限
2 | ./test.sh #执行脚本
```

默认情况下，一定要写成 **./test.sh**，而不是 **test.sh**，运行其它二进制的程序也一样。

除非将当前目录加入到PATH环境变量中，配置方法：

```
1 | sudo vi /etc/profile
2 | 加入一行
3 | export PATH=$PATH:.
4 | 保存之后，执行
5 | source /etc/profile
```

2、作为解释器参数

直接运行解释器，其参数就是 shell 脚本的文件名：

```
1 | /bin/sh test.sh
```

这种方式运行的脚本，不需要在第一行指定解释器信息，写了也没用。

编写一个快捷创建shell脚本的命令

```
1 | #!/bin/bash
2 | if test -z $1;then
3 |     newfile="./script_`date +%m%d_%s`"
4 | else
5 |     newfile=$1
6 | fi
7 | echo $newfile
8 | if ! grep "^#!" $newfile &>/dev/null; then
9 |     cat >> $newfile << EOF
10 |    #!/bin/bash
11 |    # Author:
12 |    # Date & Time: `date +%F %T`
13 |    #Description:
14 |    EOF
15 | fi
16 | vim +5 $newfile
17 | chmod +x $newfile
```

将以上内容编写好之后保存为shell文件，然后执行

```
1 | chmod u+x shell
2 | sudo mv shell /usr/bin/
```

echo命令

Shell 的 echo 指令与 PHP 的 echo 指令类似，都是用于字符串的输出。命令格式：

```
1 | echo string
```

显示普通字符串：

```
1 | echo "It is a test"
```

这里的双引号完全可以省略，以下命令与上面实例效果一致：

```
1 | echo It is a test
```

显示转义字符：

```
1 | echo "\"It is a test\""
```

结果将是:

```
1 | "It is a test"
```

同样, 双引号也可以省略

read 命令从标准输入中读取一行, 并把输入行的每个字段的值指定给 shell 变量

```
1 | #!/bin/sh
2 | read name
3 | echo "$name It is a test"
```

以上代码保存为 test.sh, name 接收标准输入的变量, 结果将是:

```
1 | [root@www ~]# sh test.sh
2 | OK                      #标准输入
3 | OK It is a test        #输出
```

显示换行

```
1 | echo -e "OK! \n" # -e 开启转义
2 | echo "It is a test"
```

输出结果:

```
1 | OK!
2 |
3 | It is a test
```

显示不换行

```
1 | #!/bin/sh
2 | echo -e "OK! \c" # -e 开启转义 \c 不换行
3 | echo "It is a test"
```

输出结果:

```
1 | OK! It is a test
```

printf 命令

printf 命令的语法:

```
1 | printf format-string [arguments...]
```

参数说明:

- **format-string:** 为格式控制字符串
- **arguments:** 为参数列表。

实例如下:

```

1  #!/bin/bash
2
3  printf "%-10s %-8s %-4s\n" 姓名 性别 体重kg
4  printf "%-10s %-8s %-4.2f\n" 郭靖 男 66.1234
5  printf "%-10s %-8s %-4.2f\n" 杨过 男 48.6543
6  printf "%-10s %-8s %-4.2f\n" 郭芙 女 47.9876

```

执行脚本，输出结果如下所示：

```

1  姓名      性别      体重kg
2  郭靖      男         66.12
3  杨过      男         48.65
4  郭芙      女         47.99

```

- %s %c %d %f都是格式替代符
- %-10s 指一个宽度为10个字符（-表示左对齐，没有则表示右对齐），任何字符都会被显示在10个字符宽的字符内，如果不足则自动以空格填充，超过也会将内容全部显示出来。
- %-4.2f 指格式化为小数，其中.2指保留2位小数。

printf的转义序列：

序列	说明
\a	警告字符，通常为ASCII的BEL字符
\b	后退
\c	抑制（不显示）输出结果中任何结尾的换行字符（只在%b格式指示符控制下的参数字符串中有效），而且，任何留在参数里的字符、任何接下来的参数以及任何留在格式字符串中的字符，都被忽略
\f	换页（formfeed）
\n	换行
\r	回车（Carriage return）
\t	水平制表符
\v	垂直制表符
\\	一个字面上的反斜杠字符
\ddd	表示1到3位数八进制值的字符。仅在格式字符串中有效
\0ddd	表示1到3位的八进制值字符

例子：

```

1  python@ubuntu:~/test$ printf "a string, no processing:<%s>\n" "A\nB"
2  a string, no processing:<A\nB>
3  python@ubuntu:~/test$ printf "a string, no processing:<%b>\n" "A\nB"
4  a string, no processing:<A
5  B>
6  python@ubuntu:~/test$ printf "www.runoob.com \a"
7  www.runoob.com python@ubuntu:~/test$

```


Shell 注释

以 # 开头的行就是注释，会被解释器忽略：

```
1  #-----
2  # 这是一个注释
3  # author: 菜鸟教程
4  # site: www.taobao.com
5  # slogan: 学的不仅是技术，更是梦想！
6  #-----
7  ##### 用户配置区 开始 #####
8  #
9  #
10 # 这里可以添加脚本描述信息
11 #
12 #
13 ##### 用户配置区 结束 #####
```

多行注释还可以使用以下格式：

```
1  :<<EOF
2  注释内容...
3  注释内容...
4  注释内容...
5  EOF
```

EOF 也可以使用其他符号：

```
1  :<<'
2  注释内容...
3  注释内容...
4  注释内容...
5  '
6
7  :<<!
8  注释内容...
9  注释内容...
10 注释内容...
11  !
```

Shell变量

定义变量

```
1  your_name="taobao.com"
```

变量名的命名须遵循如下规则：

- 命名只能使用英文字母，数字和下划线，首个字符不能以数字开头。
- 中间不能有空格，可以使用下划线（_）。
- 不能使用标点符号。
- 不能使用bash里的关键字（可用help命令查看保留关键字）。

使用变量

在变量名前面加美元符号即可，如：

```
1 your_name="qinx"
2 echo $your_name
3 echo ${your_name}
```

加花括号可以帮助解释器识别变量的边界，比如：

```
1 for skill in Ada Coffe Action Java; do
2     echo "I am good at ${skill}Script"
3 done
```

只读变量

使用 readonly 命令可以将变量定义为只读变量，只读变量的值不能被改变。

下面的例子尝试更改只读变量，结果报错：

```
1 python@ubuntu:~/shell$ myUrl="http://www.google.com"
2 python@ubuntu:~/shell$ readonly myUrl
3 python@ubuntu:~/shell$ myUrl="http://www.baidu.com"
4 -bash: myUrl: 只读变量
```

删除变量

使用 unset 命令可以删除变量，但不能删除只读变量：

```
1 #!/bin/sh
2 myUrl="http://www.baidu.com"
3 unset myUrl
4 echo $myUrl
```

变量类型

运行shell时，会同时存在三种变量：

- **1) 局部变量** 局部变量在脚本或命令中定义，仅在当前shell实例中有效，其他shell启动的程序不能访问局部变量。
- **2) 环境变量** 所有的程序，包括shell启动的程序，都能访问环境变量，有些程序需要环境变量来保证其正常运行。必要的时候shell脚本也可以定义环境变量。
- **3) shell变量** shell变量是由shell程序设置的特殊变量。shell变量中有一部分是环境变量，有一部分是局部变量，这些变量保证了shell的正常运行

Shell 函数

shell中函数的定义格式如下：

```
1 [ function ] funname [()]
2 {
3     action;
4     [return int;]
5 }
```

说明：

- 1、可以带function fun() 定义，也可以直接fun() 定义,不带任何参数。
- 2、参数返回，可以显示加：return 返回，如果不加，将以最后一条命令运行结果，作为返回值。return后跟数值n(0-255)

示例：

```
1  #!/bin/bash
2
3  funWithReturn(){
4      echo "这个函数会对输入的两个数字进行相加运算..."
5      echo "输入第一个数字： "
6      read aNum
7      echo "输入第二个数字： "
8      read anotherNum
9      echo "两个数字分别为 $aNum 和 $anotherNum !"
10     return $(( $aNum+$anotherNum ))
11 }
12 funWithReturn
13 echo "输入的两个数字之和为 $? !"
```

输出，类似下面：

```
1  这个函数会对输入的两个数字进行相加运算...
2  输入第一个数字：
3  1
4  输入第二个数字：
5  2
6  两个数字分别为 1 和 2 !
7  输入的两个数字之和为 3 !
```

函数返回值在调用该函数后通过 \$? 来获得。

注意：所有函数在使用前必须定义。这意味着必须将函数放在脚本开始部分，直至shell解释器首次发现它时，才可以使用。调用函数仅使用其函数名即可。

在Shell中，调用函数时可以向其传递参数。在函数体内部，通过 \$n 的形式来获取参数的值，例如，\$1 表示第一个参数，\$2 表示第二个参数...

带参数的函数示例：

```
1  #!/bin/bash
2
3  funWithParam(){
4      echo "第一个参数为 $1 !"
5      echo "第二个参数为 $2 !"
6      echo "第十个参数为 $10 !"
7      echo "第十个参数为 ${10} !"
8      echo "第十一个参数为 ${11} !"
9      echo "参数总数有 $# 个!"
10     echo "作为一个字符串输出所有参数 $* !"
11 }
12 funWithParam 1 2 3 4 5 6 7 8 9 34 73
```

输出结果：

```
1 第一个参数为 1 !
2 第二个参数为 2 !
3 第十个参数为 10 !
4 第十个参数为 34 !
5 第十一个参数为 73 !
6 参数总数有 11 个!
7 作为一个字符串输出所有参数 1 2 3 4 5 6 7 8 9 34 73 !
```

当 $n \geq 10$ 时，需要使用 `${n}` 来获取参数。

另外，还有几个特殊字符用来处理参数：

参数处理	说明
<code>\$#</code>	传递到脚本的参数个数
<code>\$*</code>	以一个单字符串显示所有向脚本传递的参数
<code>\$\$</code>	脚本运行的当前进程ID号
<code>\$_</code>	后台运行的最后一个进程的ID号
<code>@</code>	与 <code>*</code> 相同，但是使用时加引号，并在引号中返回每个参数。
<code>-</code>	显示Shell使用的当前选项，与set命令功能相同。
<code>?</code>	显示最后命令的退出状态。0表示没有错误，其他任何值表明有错误。

文件包含

Shell 文件包含的语法格式如下：

```
1 . filename    # 注意点号(.)和文件名中间有一空格
2 或
3 source filename
```

实例

创建两个 shell 脚本文件。

test1.sh 代码如下：

```
1 #!/bin/bash
2
3 url="http://www.baidu.com"
```

test2.sh 代码如下：

```
1 #!/bin/bash
2
3 #使用 . 号来引用test1.sh 文件
4 . ./test1.sh
5 # 或者使用以下包含文件代码
6 # source ./test1.sh
7
8 echo "url地址: $url"
```

接下来，我们为 test2.sh 添加可执行权限并执行：

```
1 $ chmod +x test2.sh
2 $ ./test2.sh
3 url地址: http://www.baidu.com
```

注：被包含的文件 test1.sh 不需要可执行权限。

shell数据类型

字符串

字符串可以用单引号，也可以用双引号，也可以不用引号。

单引号：

```
1 str='this is a string'
```

单引号字符串的限制：

- 单引号里的任何字符都会原样输出，单引号字符串中的变量是无效的；
- 单引号字符串中不能出现单独一个的单引号（对单引号使用转义符后也不行），但可成对出现，作为字符串拼接使用。

双引号：

```
1 your_name='taobao'
2 str="Hello, I know you are \"$your_name\"! \n"
3 echo -e $str
```

输出结果为：

```
1 Hello, I know you are "taobao"!
```

双引号的优点：

- 双引号里可以有变量
- 双引号里可以出现转义字符

拼接字符串：

```
1 your_name="taobao"
2 # 使用双引号拼接
3 greeting="hello, \"$your_name\" !"
4 greeting_1="hello, ${your_name} !"
5 echo $greeting $greeting_1
6 # 使用单引号拼接
7 greeting_2='hello, '$your_name' !'
8 greeting_3='hello, ${your_name} !'
9 echo $greeting_2 $greeting_3
```

输出结果为：

```
1 hello, taobao ! hello, taobao !
2 hello, taobao ! hello, ${your_name} !
```

获取字符串长度 `${#s}`

```
1 string="abcd"
2 echo ${#string} #输出 4
```

截取字符串`{s:n1:n2}`

以下实例从字符串第 2 个字符开始截取 4 个字符：

```
1 string="taobao is a great site"
2 echo ${string:1:4} # 输出 unoo
```

查找字符出现的位置 `expr index`

查找字符 **i** 或 **o** 的位置(哪个字母先出现就计算哪个)：

```
1 string="taobao is a great site"
2 echo `expr index "$string" io` # 输出 3
```

注意： 以上脚本中 ``` 是反引号，而不是单引号 `'`。

数组

bash支持一维数组（不支持多维数组），并且没有限定数组的大小。

数组元素的下标由 0 开始编号。

定义数组

在 Shell 中，用括号来表示数组，数组元素用"空格"符号分割开。定义数组的一般形式为：

```
1 array_name=(value0 value1 value2 value3)
```

或者

```
1 array_name=(
2 value0
3 value1
4 value2
5 value3
6 )
```

或单独定义数组的各个分量：

```
1 array_name[0]=value0
2 array_name[1]=value1
3 array_name[n]=valuen
```

可以不使用连续的下标，而且下标的范围没有限制。

读取数组

读取数组元素值的一般格式是：

```
1 | valuen=${array_name[n]}
```

例子：

```
1 | #!/bin/bash
2 |
3 | my_array=(A B "C" D)
4 |
5 | echo "第一个元素为：${my_array[0]}"
6 | echo "第二个元素为：${my_array[1]}"
7 | echo "第三个元素为：${my_array[2]}"
8 | echo "第四个元素为：${my_array[3]}"
```

执行脚本，输出结果如下所示：

```
1 | $ chmod +x test.sh
2 | $ ./test.sh
3 | 第一个元素为：A
4 | 第二个元素为：B
5 | 第三个元素为：C
6 | 第四个元素为：D
```

使用 `@` 或 `*` 符号可以获取数组中的所有元素，例如：

```
1 | echo ${array_name[@]}
```

例子：

```
1 | #!/bin/bash
2 |
3 | my_array[0]=A
4 | my_array[1]=B
5 | my_array[2]=C
6 | my_array[3]=D
7 |
8 | echo "数组的元素为：${my_array[*]}"
9 | echo "数组的元素为：${my_array[@]}"
```

执行脚本，输出结果如下所示：

```
1 | $ chmod +x test.sh
2 | $ ./test.sh
3 | 数组的元素为：A B C D
4 | 数组的元素为：A B C D
```

获取数组的长度

获取数组长度的方法与获取字符串长度的方法相同，例如：

```
1 # 取得数组元素的个数
2 length=${#array_name[@]}
3 # 或者
4 length=${#array_name[*]}
5 # 取得数组单个元素的长度
6 lengthn=${#array_name[n]}
```

例子：

```
1 #!/bin/bash
2
3 my_array[0]=A
4 my_array[1]=B
5 my_array[2]=C
6 my_array[3]=D
7
8 echo "数组元素个数为：${#my_array[*]}"
9 echo "数组元素个数为：${#my_array[@]}"
```

执行脚本，输出结果如下所示：

```
1 $ chmod +x test.sh
2 $ ./test.sh
3 数组元素个数为： 4
4 数组元素个数为： 4
```

Shell传递参数

执行 Shell 脚本时，向脚本传递参数，脚本内获取参数的格式为：**\$n**。

n 代表一个数字，1 为执行脚本的第一个参数，2 为执行脚本的第二个参数，以此类推.....

\$0 为执行的文件名

test.sh 文件内容如下：

```
1 vi test.sh
2 #!/bin/bash
3
4 echo "Shell 传递参数实例！";
5 echo "执行的文件名：$0";
6 echo "第一个参数为：$1";
7 echo "第二个参数为：$2";
8 echo "第三个参数为：$3";
```

运行结果：


```
1 python@ubuntu:~/test$ sh test.sh 1 2 3
2 Shell 传递参数实例！
3 执行的文件名：test.sh
4 第一个参数为：1
5 第二个参数为：2
6 第三个参数为：3
```

参数获取：

参数处理	说明
<code>\$#</code>	传递到脚本的参数个数
<code>\$*</code>	传递的参数作为一个字符串显示
<code>\$\$</code>	脚本运行的当前进程ID号
<code>\$_</code>	后台运行的最后一个进程的ID号
<code>\$@</code>	与 <code>\$*</code> 相同，但是使用时加引号
<code>\$?</code>	显示最后命令的退出状态。0表示没有错误，其他任何值表明有错误。

```
1 #!/bin/bash
2
3 echo "参数个数为：$#";
4 echo "$*传递的参数作为一个字符串显示：$*";
5 echo "$@传递的参数作为一个字符串显示：$@";
6 echo "脚本运行的当前进程ID号：$$";
7 echo "后台运行的最后一个进程的ID号：$_";
8 echo "$?"
```

执行脚本，输出结果如下所示：

```
1 python@ubuntu:~/test$ ./test.sh 1 2 3
2 参数个数为：3
3 1 2 3传递的参数作为一个字符串显示：1 2 3
4 1 2 3传递的参数作为一个字符串显示：1 2 3
5 脚本运行的当前进程ID号：5059
6 后台运行的最后一个进程的ID号：
7 0
```

`$*`与`$@`的区别：

- 只有在双引号中体现出来。假设在脚本运行时写了三个参数 1、2、3，，则 `$*` 等价于 "1 2 3"（传递了一个参数），而 `$@` 等价于 "1" "2" "3"（传递了三个参数）。

```
1 #!/bin/bash
2
3 echo "-- \"\$*\" 演示 ---"
4 for i in "$*"; do
5     echo $i
6 done
7
8 echo "-- \"@$\" 演示 ---"
9 for i in "$@"; do
```

```
10     echo $i
11 done
12
13 echo "-- \${*} 演示 ---"
14 for i in ${*}; do
15     echo $i
16 done
17
18 echo "-- \${@} 演示 ---"
19 for i in ${@}; do
20     echo $i
21 done
```

执行脚本，输出结果如下所示：

```
1 python@ubuntu:~/test$ sh test 1 2 3
2 -- "${*}" 演示 ---
3 1 2 3
4 -- "${@}" 演示 ---
5 1
6 2
7 3
8 -- ${*} 演示 ---
9 1
10 2
11 3
12 -- ${@} 演示 ---
13 1
14 2
15 3
```

Shell基本运算符

Shell 和其他编程语言一样，支持多种运算符，包括：

- 算数运算符
- 关系运算符
- 布尔运算符
- 字符串运算符
- 文件测试运算符

原生bash不支持简单的数学运算，但是可以通过其他命令来实现，例如 `awk` 和 `expr`，`expr` 最常用。

`expr` 是一款表达式计算工具，使用它能完成表达式的求值操作。

```
1 #!/bin/bash
2
3 val=`expr 2 + 2`
4 echo "两数之和为 : $val"
```

执行脚本，输出结果如下所示：

```
1 两数之和为 : 4
```

两点注意：

- 表达式和运算符之间要有空格，例如 `2+2` 是不对的，必须写成 `2 + 2`。

- 完整的表达式要被 `\` 包含，这个字符是**反引号**在 Esc 键下边。

算术运算符

下表列出了常用的算术运算符，假定变量 a 为 10，变量 b 为 20：

运算符	说明	举例
+	加法	<code>expr \$a + \$b</code> 结果为 30。
-	减法	<code>expr \$a - \$b</code> 结果为 -10。
*	乘法	<code>expr \$a * \$b</code> 结果为 200。
/	除法	<code>expr \$b / \$a</code> 结果为 2。
%	取余	<code>expr \$b % \$a</code> 结果为 0。
=	赋值	<code>a=\$b</code> 将把变量 b 的值赋给 a。
==	相等。用于比较两个数字，相同则返回 true。	<code>[\$a == \$b]</code> 返回 false。
!=	不相等。用于比较两个数字，不相同则返回 true。	<code>[\$a != \$b]</code> 返回 true。

算术运算符实例如下：

```
1 #!/bin/bash
2
3 a=10
4 b=20
5
6 val=`expr $a + $b`
7 echo "a + b : $val"
8
9 val=`expr $a - $b`
10 echo "a - b : $val"
11
12 val=`expr $a \* $b`
13 echo "a * b : $val"
14
15 val=`expr $b / $a`
16 echo "b / a : $val"
17
18 val=`expr $b % $a`
19 echo "b % a : $val"
20
21 if [ $a == $b ]
22 then
23     echo "a 等于 b"
24 fi
25 if [ $a != $b ]
26 then
27     echo "a 不等于 b"
28 fi
```

执行脚本，输出结果如下所示：

```
1 a + b : 30
2 a - b : -10
3 a * b : 200
4 b / a : 2
5 b % a : 0
6 a 不等于 b
```

注意：

- 乘号(*)前边必须加反斜杠\才能实现乘法运算；
- if...then...fi 是条件语句，后续将会讲解。
- 在 MAC 中 shell 的 expr 语法是：\$(表达式)，此处表达式中的 "*" 不需要转义符号 \。

```
1 let varName=算术表达式
2 varName=${算术表达式}
3 varName=$((算术表达式))
```

关系运算符

关系运算符只支持数字，不支持字符串，除非字符串的值是数字。

下表列出了常用的关系运算符，假定变量 a 为 10，变量 b 为 20：

运算符	说明	举例
-eq	检测两个数是否相等，相等返回 true。	[\$a -eq \$b] 返回 false。
-ne	检测两个数是否不相等，不相等返回 true。	[\$a -ne \$b] 返回 true。
-gt	检测左边的数是否大于右边的，如果是，则返回 true。	[\$a -gt \$b] 返回 false。
-lt	检测左边的数是否小于右边的，如果是，则返回 true。	[\$a -lt \$b] 返回 true。
-ge	检测左边的数是否大于等于右边的，如果是，则返回 true。	[\$a -ge \$b] 返回 false。
-le	检测左边的数是否小于等于右边的，如果是，则返回 true。	[\$a -le \$b] 返回 true。

关系运算符实例如下：

```
1 #!/bin/bash
2
3 a=10
4 b=20
5
6 if [ $a -eq $b ]
7 then
8     echo "$a -eq $b : a 等于 b"
9 else
10    echo "$a -eq $b: a 不等于 b"
11 fi
12 if [ $a -ne $b ]
13 then
14    echo "$a -ne $b: a 不等于 b"
15 else
16    echo "$a -ne $b : a 等于 b"
17 fi
18 if [ $a -gt $b ]
```

```
19 then
20     echo "$a -gt $b: a 大于 b"
21 else
22     echo "$a -gt $b: a 不大于 b"
23 fi
24 if [ $a -lt $b ]
25 then
26     echo "$a -lt $b: a 小于 b"
27 else
28     echo "$a -lt $b: a 不小于 b"
29 fi
30 if [ $a -ge $b ]
31 then
32     echo "$a -ge $b: a 大于或等于 b"
33 else
34     echo "$a -ge $b: a 小于 b"
35 fi
36 if [ $a -le $b ]
37 then
38     echo "$a -le $b: a 小于或等于 b"
39 else
40     echo "$a -le $b: a 大于 b"
41 fi
```

执行脚本，输出结果如下所示：

```
1 10 -eq 20: a 不等于 b
2 10 -ne 20: a 不等于 b
3 10 -gt 20: a 不大于 b
4 10 -lt 20: a 小于 b
5 10 -ge 20: a 小于 b
6 10 -le 20: a 小于或等于 b
```

布尔运算符

下表列出了常用的布尔运算符，假定变量 a 为 10，变量 b 为 20：

运算符	说明	举例
!	非运算，表达式为 true 则返回 false，否则返回 true。	[! false] 返回 true。
-o	或运算，有一个表达式为 true 则返回 true。	[\$a -lt 20 -o \$b -gt 100] 返回 true。
-a	与运算，两个表达式都为 true 才返回 true。	[\$a -lt 20 -a \$b -gt 100] 返回 false。

布尔运算符实例如下：

```
1 #!/bin/bash
2
3 a=10
4 b=20
5
```

```
6  if [ $a != $b ]
7  then
8      echo "$a != $b : a 不等于 b"
9  else
10     echo "$a == $b: a 等于 b"
11  fi
12  if [ $a -lt 100 -a $b -gt 15 ]
13  then
14     echo "$a 小于 100 且 $b 大于 15 : 返回 true"
15  else
16     echo "$a 小于 100 且 $b 大于 15 : 返回 false"
17  fi
18  if [ $a -lt 100 -o $b -gt 100 ]
19  then
20     echo "$a 小于 100 或 $b 大于 100 : 返回 true"
21  else
22     echo "$a 小于 100 或 $b 大于 100 : 返回 false"
23  fi
24  if [ $a -lt 5 -o $b -gt 100 ]
25  then
26     echo "$a 小于 5 或 $b 大于 100 : 返回 true"
27  else
28     echo "$a 小于 5 或 $b 大于 100 : 返回 false"
29  fi
```

执行脚本，输出结果如下所示：

```
1  10 != 20 : a 不等于 b
2  10 小于 100 且 20 大于 15 : 返回 true
3  10 小于 100 或 20 大于 100 : 返回 true
4  10 小于 5 或 20 大于 100 : 返回 false
```

逻辑运算符

以下介绍 Shell 的逻辑运算符，假定变量 a 为 10，变量 b 为 20:

运算符	说明	举例
&&	逻辑的 AND	<code>[[\$a -lt 100 && \$b -gt 100]]</code> 返回 false
	逻辑的 OR	<code>[[\$a -lt 100 \$b -gt 100]]</code> 返回 true

逻辑运算符实例如下：

```
1  #!/bin/bash
2
3  a=10
4  b=20
5
6  if [[ $a -lt 100 && $b -gt 100 ]]
7  then
8      echo "返回 true"
9  else
10     echo "返回 false"
11  fi
12
```

```
13 | if [[ $a -lt 100 || $b -gt 100 ]]
14 | then
15 |     echo "返回 true"
16 | else
17 |     echo "返回 false"
18 | fi
```

执行脚本，输出结果如下所示：

```
1 | 返回 false
2 | 返回 true
```

字符串运算符

下表列出了常用的字符串运算符，假定变量 a 为 "abc"，变量 b 为 "efg"：

运算符	说明	举例
=	检测两个字符串是否相等，相等返回 true。	[\$a = \$b] 返回 false。
!=	检测两个字符串是否相等，不相等返回 true。	[\$a != \$b] 返回 true。
-z	检测字符串长度是否为0，为0返回 true。	[-z \$a] 返回 false。
-n	检测字符串长度是否为0，不为0返回 true。	[-n "\$a"] 返回 true。
\$	检测字符串是否为空，不为空返回 true。	[\$a] 返回 true。

字符串运算符实例如下：

```
1 | #!/bin/bash
2 |
3 | a="abc"
4 | b="efg"
5 |
6 | if [ $a = $b ]
7 | then
8 |     echo "$a = $b : a 等于 b"
9 | else
10 |    echo "$a = $b: a 不等于 b"
11 | fi
12 | if [ $a != $b ]
13 | then
14 |    echo "$a != $b : a 不等于 b"
15 | else
16 |    echo "$a != $b: a 等于 b"
17 | fi
18 | if [ -z $a ]
19 | then
20 |    echo "-z $a : 字符串长度为 0"
21 | else
22 |    echo "-z $a : 字符串长度不为 0"
23 | fi
24 | if [ -n "$a" ]
25 | then
26 |    echo "-n $a : 字符串长度不为 0"
27 | else
```

```
28     echo "-n $a : 字符串长度为 0"
29 fi
30 if [ $a ]
31 then
32     echo "$a : 字符串不为空"
33 else
34     echo "$a : 字符串为空"
35 fi
```

执行脚本，输出结果如下所示：

```
1 abc = efg: a 不等于 b
2 abc != efg : a 不等于 b
3 -z abc : 字符串长度不为 0
4 -n abc : 字符串长度不为 0
5 abc : 字符串不为空
```

文件测试运算符

文件测试运算符用于检测 Unix 文件的各种属性。

属性检测描述如下：

操作符	说明	举例
-b file	检测文件是否是块设备文件，如果是，则返回 true。	[-b \$file] 返回 false。
-c file	检测文件是否是字符设备文件，如果是，则返回 true。	[-c \$file] 返回 false。
-d file	检测文件是否是目录，如果是，则返回 true。	[-d \$file] 返回 false。
-f file	检测文件是否是普通文件（既不是目录，也不是设备文件），如果是，则返回 true。	[-f \$file] 返回 true。
-g file	检测文件是否设置了 SGID 位，如果是，则返回 true。	[-g \$file] 返回 false。
-k file	检测文件是否设置了粘着位(Sticky Bit)，如果是，则返回 true。	[-k \$file] 返回 false。
-p file	检测文件是否是有名管道，如果是，则返回 true。	[-p \$file] 返回 false。
-u file	检测文件是否设置了 SUID 位，如果是，则返回 true。	[-u \$file] 返回 false。
-r file	检测文件是否可读，如果是，则返回 true。	[-r \$file] 返回 true。
-w file	检测文件是否可写，如果是，则返回 true。	[-w \$file] 返回 true。
-x file	检测文件是否可执行，如果是，则返回 true。	[-x \$file] 返回 true。
-s file	检测文件是否为空（文件大小是否大于0），不为空返回 true。	[-s \$file] 返回 true。
-e file	检测文件（包括目录）是否存在，如果是，则返回 true。	[-e \$file] 返回 true。

其他检查符：

- **-S**: 判断某文件是否 socket。
- **-L**: 检测文件是否存在并且是一个符号链接。

变量 file 表示文件 `/var/www/runoob/test.sh`，它的大小为 100 字节，具有 **rwX** 权限。下面的代码，将检测该文件的各种属性：

```

1  #!/bin/bash
2
3  file="/var/www/runoob/test.sh"
4  if [ -r $file ]
5  then
6      echo "文件可读"
7  else
8      echo "文件不可读"
```

```
9  fi
10 if [ -w $file ]
11 then
12     echo "文件可写"
13 else
14     echo "文件不可写"
15 fi
16 if [ -x $file ]
17 then
18     echo "文件可执行"
19 else
20     echo "文件不可执行"
21 fi
22 if [ -f $file ]
23 then
24     echo "文件为普通文件"
25 else
26     echo "文件为特殊文件"
27 fi
28 if [ -d $file ]
29 then
30     echo "文件是个目录"
31 else
32     echo "文件不是个目录"
33 fi
34 if [ -s $file ]
35 then
36     echo "文件不为空"
37 else
38     echo "文件为空"
39 fi
40 if [ -e $file ]
41 then
42     echo "文件存在"
43 else
44     echo "文件不存在"
45 fi
```

执行脚本，输出结果如下所示：

```
1  文件可读
2  文件可写
3  文件可执行
4  文件为普通文件
5  文件不是个目录
6  文件不为空
7  文件存在
```

test命令

Shell中的 test 命令用于检查某个条件是否成立，它可以进行数值、字符和文件三个方面的测试。

数值测试

参数	说明
-eq	等于则为真
-ne	不等于则为真
-gt	大于则为真
-ge	大于等于则为真
-lt	小于则为真
-le	小于等于则为真

实例演示：

```
1 num1=100
2 num2=100
3 if test ${num1} -eq ${num2}
4 then
5     echo '两个数相等！'
6 else
7     echo '两个数不相等！'
8 fi
```

输出结果：

```
1 两个数相等！
```

代码中的 `[]` 执行基本的算数运算，如：

```
1 #!/bin/bash
2
3 a=5
4 b=6
5
6 result=$((a+b)) # 注意等号两边不能有空格
7 echo "result 为: $result"
```

结果为：

```
1 result 为: 11
```

字符串测试

参数	说明
=	等于则为真
!=	不相等则为真
-z 字符串	字符串的长度为零则为真
-n 字符串	字符串的长度不为零则为真

实例演示：

```
1 num1="ru1noob"
2 num2="runoob"
3 if test $num1 = $num2
4 then
5     echo '两个字符串相等！'
6 else
7     echo '两个字符串不相等！'
8 fi
```

输出结果：

```
1 两个字符串不相等！
```

文件测试

参数	说明
-e 文件名	如果文件存在则为真
-r 文件名	如果文件存在且可读则为真
-w 文件名	如果文件存在且可写则为真
-x 文件名	如果文件存在且可执行则为真
-s 文件名	如果文件存在且至少有一个字符则为真
-d 文件名	如果文件存在且为目录则为真
-f 文件名	如果文件存在且为普通文件则为真
-c 文件名	如果文件存在且为字符型特殊文件则为真
-b 文件名	如果文件存在且为块特殊文件则为真

实例演示：

```
1 cd /bin
2 if test -e ./bash
3 then
4     echo '文件已存在！'
5 else
6     echo '文件不存在！'
7 fi
```

输出结果：

```
1 | 文件已存在！
```

另外，Shell还提供了与(-a)、或(-o)、非(!)三个逻辑操作符用于将测试条件连接起来，其优先级为："!"最高，"-a"次之，"-o"最低。例如：

```
1 | cd /bin
2 | if test -e ./notFile -o -e ./bash
3 | then
4 |     echo '至少有一个文件存在！'
5 | else
6 |     echo '两个文件都不存在'
7 | fi
```

输出结果：

```
1 | 至少有一个文件存在！
```

Shell 流程控制

if else判断语句

if 语句语法格式：

```
1 | if condition
2 | then
3 |     command1
4 |     command2
5 |     ...
6 |     commandN
7 | fi
```

写成一行（适用于终端命令提示符）：

```
1 | if [ $(ps -ef | grep -c "ssh") -gt 1 ]; then echo "true"; fi
```

if else 语法格式：

```
1 | if condition
2 | then
3 |     command1
4 |     command2
5 |     ...
6 |     commandN
7 | else
8 |     command
9 | fi
```

if else-if else 语法格式：

```
1 | if condition1
2 | then
3 |     command1
4 | elif condition2
5 | then
6 |     command2
7 | else
8 |     commandN
9 | fi
```

以下实例判断两个变量是否相等：

```
1 | a=10
2 | b=20
3 | if [ $a == $b ]
4 | then
5 |     echo "a 等于 b"
6 | elif [ $a -gt $b ]
7 | then
8 |     echo "a 大于 b"
9 | elif [ $a -lt $b ]
10 | then
11 |     echo "a 小于 b"
12 | else
13 |     echo "没有符合的条件"
14 | fi
```

输出结果：

```
1 | a 小于 b
```

if else语句经常与test命令结合使用，如下所示：

```
1 | num1=$((2*3))
2 | num2=$((1+5))
3 | if test $[num1] -eq $[num2]
4 | then
5 |     echo '两个数字相等!'
6 | else
7 |     echo '两个数字不相等!'
8 | fi
```

输出结果：

```
1 | 两个数字相等！
```

for循环

for循环一般格式为：

```
1 | for var in item1 item2 ... itemN
2 | do
3 |     command1
4 |     command2
5 |     ...
6 |     commandN
7 | done
```

写成一行:

```
1 | for var in item1 item2 ... itemN; do command1; command2... done;
```

例如，顺序输出当前列表中的数字：

```
1 | for loop in 1 2 3 4 5
2 | do
3 |     echo "The value is: $loop"
4 | done
```

输出结果：

```
1 | The value is: 1
2 | The value is: 2
3 | The value is: 3
4 | The value is: 4
5 | The value is: 5
```

顺序输出字符串中的字符：

```
1 | for str in 'This is a string'
2 | do
3 |     echo $str
4 | done
```

输出结果：

```
1 | This is a string
```

while循环

while循环格式为：

```
1 | while condition
2 | do
3 |     command
4 | done
```

示例：

```
1  #!/bin/bash
2  int=1
3  while(( $int<=5 ))
4  do
5      echo $int
6      let "int++"
7  done
```

运行脚本，输出：

```
1  1
2  2
3  3
4  4
5  5
```

while循环可用于读取键盘信息。下面的例子中，输入信息被设置为变量FILM，按 `ctrl-d` 结束循环。

```
1  echo '按下 <CTRL-D> 退出 '
2  echo -n '输入你最喜欢的网站名： '
3  while read FILM
4  do
5      echo "是的！$FILM 是一个好网站"
6  done
```

运行脚本，输出类似下面：

```
1  按下 <CTRL-D> 退出
2  输入你最喜欢的网站名:淘宝
3  是的！ 淘宝 是一个好网站
```

无限循环

无限循环语法格式：

```
1  while :
2  do
3      command
4  done
```

或者

```
1  while true
2  do
3      command
4  done
```

或者

```
1  for (( ; ; ))
```


until 循环

until 循环执行一系列命令直至条件为 true 时停止。

until 循环与 while 循环在处理方式上刚好相反。

一般 while 循环优于 until 循环，但在某些时候——也只是极少数情况下，until 循环更加有用。

until 语法格式：

```
1  until condition
2  do
3      command
4  done
```

condition 一般为条件表达式，如果返回值为 false，则继续执行循环体内的语句，否则跳出循环。

以下实例我们使用 until 命令来输出 0 ~ 9 的数字：

```
1  #!/bin/bash
2
3  a=0
4
5  until [ ! $a -lt 10 ]
6  do
7      echo $a
8      a=`expr $a + 1`
9  done
```

运行结果：

输出结果为：

```
1  0
2  1
3  2
4  3
5  4
6  5
7  6
8  7
9  8
10 9
```

case

Shell case 语句为多选择语句。可以用 case 语句匹配一个值与一个模式，如果匹配成功，执行相匹配的命令。case 语句格式如下：

```
1  case 值 in
2  模式1)
3      command1
4      command2
5      ...
6      commandN
7      ;;
8  模式2)
```

```
9      command1
10     command2
11     ...
12     commandN
13     ;;
14 esac
```

下面的脚本提示输入1到4，与每一种模式进行匹配：

```
1  echo '输入 1 到 4 之间的数字:'
2  echo '你输入的数字为:'
3  read aNum
4  case $aNum in
5      1)  echo '你选择了 1'
6          ;;
7      2)  echo '你选择了 2'
8          ;;
9      3)  echo '你选择了 3'
10         ;;
11      4)  echo '你选择了 4'
12         ;;
13      *)  echo '你没有输入 1 到 4 之间的数字'
14         ;;
15  esac
```

输入不同的内容，会有不同的结果，例如：

```
1  输入 1 到 4 之间的数字:
2  你输入的数字为:
3  3
4  你选择了 3
```

跳出循环

在循环过程中，有时候需要在未达到循环结束条件时强制跳出循环，Shell使用两个命令来实现该功能：break和continue。

break命令允许跳出所有循环（终止执行后面的所有循环）。

下面的例子中，脚本进入死循环直至用户输入数字大于5。要跳出这个循环，返回到shell提示符下，需要使用break命令。

```
1  #!/bin/bash
2  while :
3  do
4      echo -n "输入 1 到 5 之间的数字:"
5      read aNum
6      case $aNum in
7          1|2|3|4|5) echo "你输入的数字为 $aNum!"
8                      ;;
9          *) echo "你输入的数字不是 1 到 5 之间的！游戏结束"
10             break
11             ;;
12      esac
13  done
```

执行以上代码，输出结果为：

```
1 | 输入 1 到 5 之间的数字:3
2 | 你输入的数字为 3!
3 | 输入 1 到 5 之间的数字:7
4 | 你输入的数字不是 1 到 5 之间的！游戏结束
```

continue命令与break命令类似，只有一点差别，它不会跳出所有循环，仅仅跳出当前循环。

Shell输入/输出重定向

重定向命令列表如下：

命令	说明
command > file	将输出重定向到 file。
command < file	将输入重定向到 file。
command >> file	将输出以追加的方式重定向到 file。
n > file	将文件描述符为 n 的文件重定向到 file。
n >> file	将文件描述符为 n 的文件以追加的方式重定向到 file。
n >& m	将输出文件 m 和 n 合并。
n <& m	将输入文件 m 和 n 合并。
<< tag	将开始标记 tag 和结束标记 tag 之间的内容作为输入。

需要注意的是文件描述符 0 通常是标准输入（STDIN），1 是标准输出（STDOUT），2 是标准错误输出（STDERR）。

输出重定向

重定向一般通过在命令间插入特定的符号来实现。特别的，这些符号的语法如下所示：

```
1 | command1 > file1
```

上面这个命令执行command1然后将输出的内容存入file1。

注意任何file1内的已经存在的内容将被新内容替代。如果要将新内容添加在文件末尾，请使用>>操作符。

输出重定向会覆盖文件内容：

```
1 | $ echo "www.baidu.com" > users
2 | $ cat users
3 | www.baidu.com
4 | $
```

如果不希望文件内容被覆盖，可以使用 >> 追加到文件末尾，例如：

```
1 | $ echo "www.baidu.com" >> users
2 | $ cat users
3 | www.baidu.com
4 | www.baidu.com
5 | $
```

输入重定向

和输出重定向一样，Unix 命令也可以从文件获取输入，语法为：

```
1 | command1 < file1
```

这样，本来需要从键盘获取输入的命令会转移到文件读取内容。

注意：输出重定向是大于号(>)，输入重定向是小于号(<)。

统计 users 文件的行数,执行以下命令：

```
1 | python@ubuntu:~/test$ wc -l test
2 | 4 test
```

也可以将输入重定向到 users 文件：

```
1 | python@ubuntu:~/test$ wc -l <test
2 | 4
```

注意：上面两个例子的结果不同：第一个例子，会输出文件名；第二个不会，因为它仅仅知道从标准输入读取内容。

同时替换输入和输出，执行command1，从文件infile读取内容，然后将输出写入到outfile中：

```
1 | command1 < infile > outfile
```

重定向深入讲解

一般情况下，每个 Unix/Linux 命令运行时都会打开三个文件：

- 标准输入文件(stdin)：stdin的文件描述符为0，Unix程序默认从stdin读取数据。
- 标准输出文件(stdout)：stdout 的文件描述符为1，Unix程序默认向stdout输出数据。
- 标准错误文件(stderr)：stderr的文件描述符为2，Unix程序会向stderr流中写入错误信息。

默认情况下，command > file 将 stdout 重定向到 file，command < file 将stdin 重定向到 file。

如果希望 stderr 重定向到 file，可以这样写：

```
1 | $ command 2 > file
```

如果希望 stderr 追加到 file 文件末尾，可以这样写：

```
1 | $ command 2 >> file
```

2 表示标准错误文件(stderr)。

如果希望将 stdout 和 stderr 合并后重定向到 file，可以这样写：

```
1 | $ command > file 2>&1
2 | 或者
3 | $ command >> file 2>&1
```

如果希望对 stdin 和 stdout 都重定向，可以这样写：

```
1 | $ command < file1 >file2
```

command 命令将 stdin 重定向到 file1，将 stdout 重定向到 file2。

Here Document

Here Document 是 Shell 中的一种特殊的重定向方式，用来将输入重定向到一个交互式 Shell 脚本或程序。

它的基本的形式如下：

```
1 | command << delimiter
2 |     document
3 | delimiter
```

它的作用是将两个 delimiter 之间的内容(document) 作为输入传递给 command。

注意：结尾的delimiter 一定要顶格写，前面不能有任何字符，后面也不能有任何字符，包括空格和 tab 缩进。

在命令行中通过 wc -l 命令计算 Here Document 的行数：

```
1 | $ wc -l << EOF
2 |     欢迎来到
3 |     菜鸟教程
4 |     www.runoob.com
5 | EOF
6 | 3           # 输出结果为 3 行
7 | $
```

/dev/null 文件

如果希望执行某个命令，但又不希望在屏幕上显示输出结果，那么可以将输出重定向到 /dev/null：

```
1 | $ command > /dev/null
```

/dev/null 是一个特殊的文件，写入到它的内容都会被丢弃；如果尝试从该文件读取内容，那么什么也读不到。但是 /dev/null 文件非常有用，将命令的输出重定向到它，会起到"禁止输出"的效果。

如果希望屏蔽 stdout 和 stderr，可以这样写：

```
1 | $ command > /dev/null 2>&1
```

0 是标准输入 (STDIN)，1 是标准输出 (STDOUT)，2 是标准错误输出 (STDERR)。

实例

杨辉三角：

```
1  #!/bin/bash
2
3  if (test -z $1) ;then
4      read -p "Input high Int Lines:" high
5  else
6      high=$1
7  fi
8  if (test -z $2) ;then
9      space=4
10 else
11     space=$2
12 fi
13
14 printspace(){
15     #空位填充
16     for((z=1;z<=$1;z++));do
17         echo -n " "
18     done
19 }
20
21 a[0]=1
22 for((i=0;i<=high;i++));do
23     #产生当前列数据数组
24     for ((j=$i;j>0;j--));do
25         ((a[$j]+=a[$j-1]))
26     done
27     printspace $(((high-i)*space/2))
28     for ((j=0;j<=$i;j++));do
29         num=$((space-${#a[$j]}))
30         printspace $((num/2))
31         echo -n "${a[$j]}"
32         printspace $((num-$num/2))
33     done
34     echo ""
35 done
```

sum()&max():

```
1  #!/bin/bash
2
3  echo "shell的函数返回值只能为0~255的整数，高位自动丢弃"
4  sum(){
5      sum=0
6      for i in $@
7      do
8          if test $i -ne $1;then
9              echo -n "+"
10             fi
11             echo -n "$i"
12             sum=$((sum+$i))
13         done
14         echo "=$sum"
15         return $((sum))
16     }
17     sum $@
```

```

18 echo "'sum()'函数返回值: "$?
19
20 max(){
21     max=0
22     for i in $@;do
23         if test $i -ge $max;then
24             max=$i
25         fi
26     done
27     echo "参数最大值: $max"
28     return $((($max))
29 }
30
31 max $@
32
33 echo "'max()'函数返回值: "$?
34

```

99乘法表：

```

1  #!/bin/bash
2
3  for i in {1..9};do
4      for((j=1;j<=i;j++));do
5          echo -en "$i*$j=$(($i*$j))\t"
6      done
7      echo ""
8  done
9
10 for a in {1..9};do
11     for b in {0..9};do
12         for c in {0..9};do
13             number1=$((a*100+b*10+c))
14             number2=$((a**3+b**3+c**3))
15             if test $number1 -eq $number2; then
16                 echo "Found number $number1"
17             fi
18         done
19     done
20 done

```

文本编辑命令

cut命令

```

1  选项与参数：
2  -d   ：后面接分隔字符。与 -f 一起使用；
3  -f   ：依据 -d 的分隔字符将一段信息分割为数段，用 -f 取出第几段的意思；
4  -c   ：以字符（characters）的单位取出固定字符区间；

```

cut以行为单位，根据分隔符把行分成若干列，这样就可以指定选取哪些列了。

```

1 | cut -d '分隔字符' -f 选取的列数
2 | echo $PATH|cut -d ':' -f 2          --选取第2列
3 | echo $PATH|cut -d ':' -f 3,5       --选取第3列和第5列
4 | echo $PATH|cut -d ':' -f 3-5       --选取第3列到第5列
5 | echo $PATH|cut -d ':' -f 3-        --选取第3列到最后1列
6 | echo $PATH|cut -d ':' -f 1-3,5     --选取第1到第3列还有第5列

```

只显示/etc/passwd的用户和shell:

```

1 | #cat /etc/passwd | cut -d ':' -f 1,7
2 | root:/bin/bash
3 | daemon:/bin/sh
4 | bin:/bin/sh

```

sed命令

sed 可依照脚本的指令来处理、编辑文本文件。

Sed 主要用来自动编辑一个或多个文件、简化对文件的反复操作、编写转换程序等。

语法:

```

1 | sed [-e<script>][-f<script文件>][文本文件]

```

参数说明:

- `-e <script>` 以指定的script来处理输入的文本文件。
- `-f<script文件>` 以指定的script文件来处理输入的文本文件。
- `-n` 仅显示script处理后的结果，一般跟p动作搭配使用。
- `-i` 使用处理后的结果修改文件。

动作说明:

- a: 在指定行后面插入内容
- i: 在指定行前面插入内容
- d: 删除指定行
- c: 替换指定行
- p: 打印指定行的数据，通常需要跟 `-n` 选项搭配使用
- s: 替换指定字符，兼容vim的替换语法，例如 `1,20s/old/new/g`

元字符集

sed支持一般的正则表达式，下面是支持的正则语法:

- ^ 行的开始 如: `/^sed/` 匹配所有以sed开头的行。
- \$ 行的结束 如: `/sed$/` 匹配所有以sed结尾的行。
- . 匹配一个非换行符的任意字符 如: `/s.d/` 匹配s后接一个任意字符，然后是d。
- * 匹配零或多个字符 如: `/*sed/` 匹配所有模板是一个或多个空格后紧跟sed的行。
- [] 匹配一个指定范围内的字符，如 `/[ss]ed/` 匹配sed和Sed。
- [^] 匹配一个不在指定范围内的字符，如: `/[^A-RT-Z]ed/` 匹配不包含A-R和T-Z的一个字母开头，紧跟ed的行。
- \(. . \) 保存匹配的字符，如 `s/(love)able\1rs,` loveable被替换成lovers。
- & 保存搜索字符用来替换其他字符，如 `s/love/**&*/`，love这成 `**love**`。
- \< 单词的开始，如: `/<love/` 匹配包含以love开头的单词的行。
- \> 单词的结束，如 `/love>/` 匹配包含以love结尾的单词的行。
- x\+ 重复字符x，至少1次，如: `/o\+/` 匹配至少有1个o的行。

`x\{m\}` 重复字符x, m次, 如: `/o\{5\}/` 匹配包含5个o的行。

`x\{m,\}` 重复字符x,至少m次, 如: `/o\{5,\}/` 匹配至少有5个o的行。

`x\{m,n\}` 重复字符x, 至少m次, 不多于n次, 如: `/o\{5,10\}/` 匹配5-10个o的行。

a|i:在指定行位置添加行

```
1 python@xxx:~/test$ cat testfile
2   LINUX!
3   Linux is a free unix-type operating system.
4   This is a linux testfile!
5   Linux test
6
7 python@xxx:~/test$ sed -e 2a\newline testfile
8   LINUX!
9   Linux is a free unix-type operating system.
10  newline
11  This is a linux testfile!
12  Linux test
13
```

默认情况下 `-e` 参数可以省略:

```
1 python@xxx:~/test$ cat testfile|sed '2a\newline'
2   LINUX!
3   Linux is a free unix-type operating system.
4  newline
5   This is a linux testfile!
6   Linux test
7
8 python@xxx:~/test$ sed '2a newline' testfile
9   LINUX!
10  Linux is a free unix-type operating system.
11  newline
12  This is a linux testfile!
13  Linux test
```

`<script>` 中的内容最好用单引号括起来, 如果脚本内容不存在空白字符也可以省略单引号。

在第二行之前添加一行:

```
1 python@xxx:~/test$ sed '2i newline' testfile
2   LINUX!
3  newline
4   Linux is a free unix-type operating system.
5   This is a linux testfile!
6   Linux test
```

最后一行加入 `# This is a test`:

```
1 python@xxx:~/test$ sed '$a # This is a test' testfile
2   LINUX!
3   Linux is a free unix-type operating system.
4   This is a linux testfile!
5   Linux test
6  # This is a test
```

同时添加多行：

```
1 python@xxx:~/test$ cat testfile|sed '2a\newline1\  
2 > newline2'  
3 LINUX!  
4 Linux is a free unix-type operating system.  
5 newline1  
6 newline2  
7 This is a linux testfile!  
8 Linux test
```

d:删除指定行

将 /etc/passwd 的内容列出行号，并将第 2~5 行删除！

```
1 [root@www ~]# nl /etc/passwd | sed '2,5d'  
2 1 root:x:0:0:root:/root:/bin/bash  
3 6 sync:x:5:0:sync:/sbin:/bin/sync  
4 7 shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown  
5 .....(后面省略).....
```

只删除第2行：

```
1 nl /etc/passwd | sed '2d'
```

删除第3到最后一行：

```
1 nl /etc/passwd | sed '3,$d'
```

删除/etc/passwd所有包含 /usr/sbin/nologin 的行，其他行输出：

```
1 python@xxx:~/test$ nl /etc/passwd | sed '/\usr\/sbin\/nologin/d'  
2 1 root:x:0:0:root:/root:/bin/bash  
3 5 sync:x:4:65534:sync:/bin:/bin/sync  
4 19 systemd-timesync:x:100:102:systemd Time Synchronization,,,:/run/systemd:/bin/false  
5 20 systemd-network:x:101:103:systemd Network  
Management,,,:/run/systemd/netif:/bin/false  
6 ....下面忽略
```

c:替换指定行

将第2-5行的内容替换成为『No 2-5 number』：

```
1 python@xxx:~/test$ nl /etc/passwd | sed '2,5c No 2-5 number'  
2 1 root:x:0:0:root:/root:/bin/bash  
3 No 2-5 number  
4 6 games:x:5:60:games:/usr/games:/usr/sbin/nologin  
5 .....(后面省略).....
```

p:仅显示指定行

不加 `-n` 选项时，除了输出匹配行，还会同时输出所有行，所以需要加 `-n` 选项。

仅列出 `/etc/passwd` 文件内的第 5-7 行：

```
1 python@xxx:~/test$ nl /etc/passwd | sed -n '5,7p'
2 5  sync:x:4:65534:sync:/bin:/bin/sync
3 6  games:x:5:60:games:/usr/games:/usr/sbin/nologin
4 7  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
```

搜索 `/etc/passwd` 有 `root` 关键字的行：

```
1 python@xxx:~/test$ cat /etc/passwd | sed -n '/root/p'
2 root:x:0:0:root:/root:/bin/bash
3 python@xxx:~/test$ sed -n '/root/p' /etc/passwd
4 root:x:0:0:root:/root:/bin/bash
```

打印 `/etc/passwd` 有以 `root` 和 `bin` 开头之间的行：

```
1 python@xxx:~/test$ cat /etc/passwd | sed -n '/^root/,/^bin/p'
2 root:x:0:0:root:/root:/bin/bash
3 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
4 bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

打印从第五行开始到第一个包含以 `/var/mail` 开始的行之间的所有行：

```
1 python@xxx:~/test$ nl /etc/passwd | sed -n '5,\n/var\n/mail/p'
2      5  sync:x:4:65534:sync:/bin:/bin/sync
3      6  games:x:5:60:games:/usr/games:/usr/sbin/nologin
4      7  man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
5      8  lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
6      9  mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
```

s:字符串替换

语法：

```
1 sed 's/要被取代的字符串/新的字符串/g'
```

不论什么字符，紧跟着 `s` 命令的都被认为是新的分隔符。

`sed 's#10#100#g'` 表示把所有 10 替换成 100，“#”在这里是分隔符，代替了默认的“/”分隔符。

提取本机所有的 ip 地址：

```
1 [root@VM_0_9_centos ~]# ifconfig | grep 'inet '
2      inet 172.16.0.9  netmask 255.255.240.0  broadcast 172.16.15.255
3      inet 127.0.0.1  netmask 255.0.0.0
4 [root@VM_0_9_centos ~]# ifconfig | grep 'inet '|sed 's/^[^0-9]*\([0-9\.\]*\).*$/\1/g'
5 172.16.0.9
6 127.0.0.1
```

对于以 `root` 和 `bin` 开头之间的行，每行的末尾添加 `sed test`：

```
1 python@xxx:~/test$ cat /etc/passwd | sed '/^root/,/^bin/s/$/--sed test/'
2 root:x:0:0:root:/root:/bin/bash--sed test
3 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin--sed test
4 bin:x:2:2:bin:/bin:/usr/sbin/nologin--sed test
5 sys:x:3:3:sys:/dev:/usr/sbin/nologin
```

y:单字符替换

跟s一样也用于替换，不过s替换的是整体，y替换的是每一字母对应的单个字母

把data中的第一行至第三行中的a替换成A，b替换成B，c替换成C：

```
1 sed '1,3y/abc/ABC/' data
```

示例：

```
1 python@ubuntu:~/test$ echo "123" | sed 'y/13/34/'
2 324
3 python@ubuntu:~/test$ echo "axxbxxcxx" | sed 'y/abc/123/'
4 1xx2xx3xx
```

hHgG 模式空间&保持空间

h命令是将当前模式空间中内容覆盖至保持空间，H命令是将当前模式空间中的内容追加至保持空间

g命令是将当前保持空间中内容覆盖至模式空间，G命令是将当前保持空间中的内容追加至模式空间

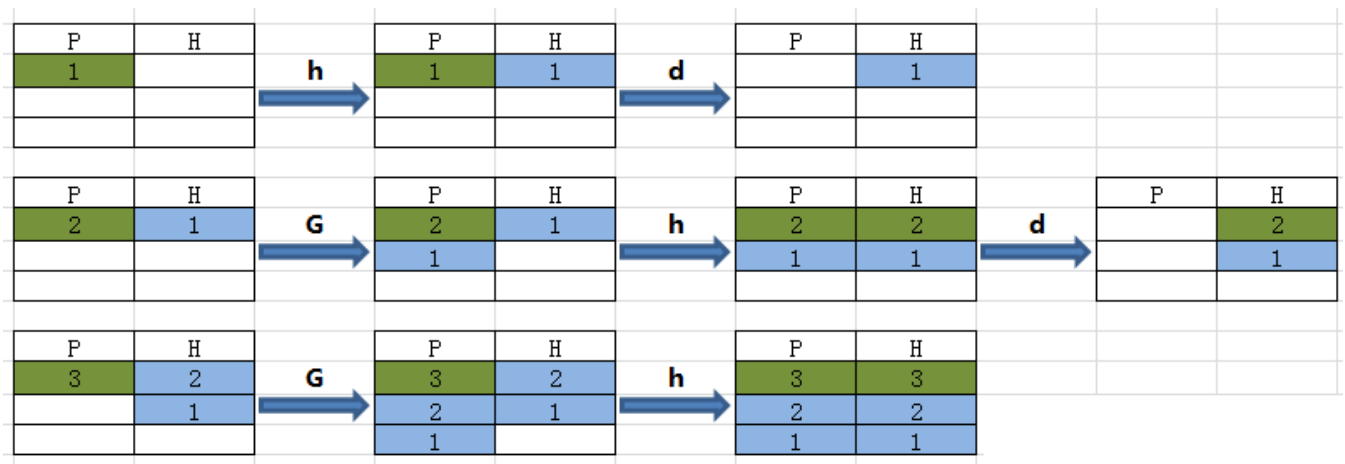
模拟tac命令：

```
1 python@ubuntu:~/test$ cat log.txt
2 2 this is a test
3 3 Are you like awk
4 This's a test
5 10 There are orange,apple,mongo
6 python@ubuntu:~/test$ cat log.txt |sed '1!G;h;$!d'
7 10 There are orange,apple,mongo
8 This's a test
9 3 Are you like awk
10 2 this is a test
```

1!G第1行不执行“G”命令，从第2行开始执行。

\$!d，最后一行不删除（保留最后1行）

下图P表示模式空间，H代表保持空间：



递增序列:

```

1 python@ubuntu:~/test$ seq 3
2 1
3 2
4 3
5 python@ubuntu:~/test$ seq 3|sed 'H;g'
6 1
7
8 1
9 2
10
11 1
12 2
13 3

```

多次指定 -e 选项进行多点编辑

删除/etc/passwd第三行到末尾的数据，并把bash替换为blueshell:

```

1 n1 /etc/passwd | sed -e '3,$d' -e 's/bash/blueshell/'
2 1 root:x:0:0:root:/root:/bin/blueshell
3 2 daemon:x:1:1:daemon:/usr/sbin:/bin/sh

```

删除一个文件以#开头的行和空行:

```

1 python@xxx:~/test$ n1 abc -ba
2 1
3 2 b
4 3 a
5 4
6 5 # aaaa
7 6
8 7 ddd
9 8
10 9 # sss
11 10 eeee
12 11
13 python@xxx:~/test$ sed -e '/^#/d' -e '/^$/d' abc
14 b
15 a
16 ddd
17 eeee

```

也可以通过 ; 实现

```
1 python@ubuntu:~/test$ nl /etc/passwd | sed '3,$d;s/bash/blueshell/'
2      1  root:x:0:0:root:/root:/bin/blueshell
3      2  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
4 python@ubuntu:~/test$ sed '/^#/d;/^$/d' abc
5 b
6 a
7 ddd
8 eeee
```

选项 -i 直接修改文件内容

默认情况下sed命令仅仅只是将处理结果显示在控制台，加 -i 选项则会修改文件内容。

将 regular_express.txt 内每一行结尾若为 . 则换成 !

```
1 [root@www ~]# cat regular_express.txt
2 taobao.
3 google.
4 taobao.
5 facebook.
6 zhihu-
7 weibo-
8 [root@www ~]# sed -i 's/\.$/!/g' regular_express.txt
9 [root@www ~]# cat regular_express.txt
10 taobao!
11 google!
12 taobao!
13 facebook!
14 zhihu-
15 weibo-
```

awk命令

AWK是一种处理文本文件的语言，是一个强大的文本分析工具。

之所以叫AWK是因为其取了三位创始人 Alfred Aho, Peter Weinberger, 和 Brian Kernighan 的 Family Name 的首字符。

语法：

```
1 awk [选项参数] 'script' var=value file(s)
2 或
3 awk [选项参数] -f scriptfile var=value file(s)
```

选项参数说明：

- -F fs or --field-separator fs
指定输入文件折分分隔符，fs是一个字符串或者是一个正则表达式，如-F:。
- -v var=value or --assign var=value
赋值一个用户定义变量。
- -f scripfile or --file scriptfile
从脚本文件中读取awk命令。

基本用法

```
1 | awk '{[pattern] action}' file
```

每行按空格或TAB分割，输出文本中的1、4列：

```
1 | python@ubuntu:~/test$ cat log.txt
2 | 2 this is a test
3 | 3 Are you like awk
4 | This's a test
5 | 10 There are orange,apple,mongo
6 | python@ubuntu:~/test$ awk '{print $1,$4}' log.txt
7 | 2 a
8 | 3 like
9 | This's
10 | 10 orange,apple,mongo
```

格式化输出：

```
1 | python@ubuntu:~/test$ awk '{printf "%-8s %-10s\n",$1,$4}' log.txt
2 | 2          a
3 | 3          like
4 | This's
```

-F 指定分割字符

```
1 | awk -F # -F相当于内置变量FS，指定分割字符
```

使用:分割,取/etc/passwd文件每个用户对应shell：

```
1 | python@ubuntu:~/test$ awk -F: '{print $1,$7}' /etc/passwd
2 | root /bin/bash
3 | daemon /usr/sbin/nologin
4 | bin /usr/sbin/nologin
5 | sys /usr/sbin/nologin
6 | sync /bin/sync
7 | # 或者使用内建变量
8 | python@ubuntu:~/test$ awk 'BEGIN{FS=":"} {print $1,$7}' /etc/passwd
9 | root /bin/bash
10 | daemon /usr/sbin/nologin
11 | bin /usr/sbin/nologin
```

同时使用:和/两个分隔符分割/etc/passwd文件

```
1 | python@ubuntu:~/test$ awk -F '[:\/]' '{print $1,$7}' /etc/passwd
2 | root root
3 | daemon usr
4 | bin bin
```

-v 设置变量

```
1 | awk -v # 设置变量
```

例子:

```
1 | python@ubuntu:~/test$ cat log.txt
2 | 2 this is a test
3 | 3 Are you like awk
4 | This's a test
5 | 10 There are orange,apple,mongo
6 | python@ubuntu:~/test$ awk -va=1 '{print $1,$1+a}' log.txt
7 | 2 3
8 | 3 4
9 | This's 1
10 | 10 11
11 | python@ubuntu:~/test$ awk -va=1 -vb=s '{print $1,$1+a,$1b}' log.txt
12 | 2 3 2s
13 | 3 4 3s
14 | This's 1 This'ss
15 | 10 11 10s
```

-f 指定awk脚本

```
1 | awk -f {awk脚本} {文件名}
```

脚本模块:

- BEGIN{ 这里面放的是执行前的语句 }
- END {这里面放的是处理完所有的行后要执行的语句 }
- {这里面放的是处理每一行时要执行的语句}

假设有这么一个文件 (学生成绩表):

```
1 | $ cat score.txt
2 | Marry    2143 78 84 77
3 | Jack     2321 66 78 45
4 | Tom      2122 48 77 71
5 | Mike     2537 87 97 95
6 | Bob      2415 40 57 62
```

awk脚本如下:

```
1 | $ cat cal.awk
2 | #!/bin/awk -f
3 | #运行前
4 | BEGIN {
5 |     math = 0
6 |     english = 0
7 |     computer = 0
8 |
9 |     printf "NAME    NO.    MATH  ENGLISH  COMPUTER  TOTAL\n"
10 |    printf "-----\n"
11 | }
12 | #运行中
```



```

13 {
14     math+=$3
15     english+=$4
16     computer+=$5
17     printf "%-6s %-6s %4d %8d %8d %8d\n", $1,$2,$3,$4,$5,$3+$4+$5
18 }
19 #运行后
20 END {
21     printf "-----\n"
22     printf "  TOTAL:%10d %8d %8d \n", math, english, computer
23     printf "AVERAGE:%10.2f %8.2f %8.2f\n", math/NR, english/NR, computer/NR
24 }

```

我们来看一下执行结果：

```

1 $ awk -f cal.awk score.txt
2 NAME      NO.    MATH  ENGLISH  COMPUTER  TOTAL
3 -----
4 Marry    2143    78    84      77      239
5 Jack     2321    66    78      45      189
6 Tom      2122    48    77      71      196
7 Mike     2537    87    97      95      279
8 Bob      2415    40    57      62      159
9 -----
10 TOTAL:      319    393    350
11 AVERAGE:    63.80   78.60   70.00

```

AWK工作原理

AWK 工作流程可分为三个部分：

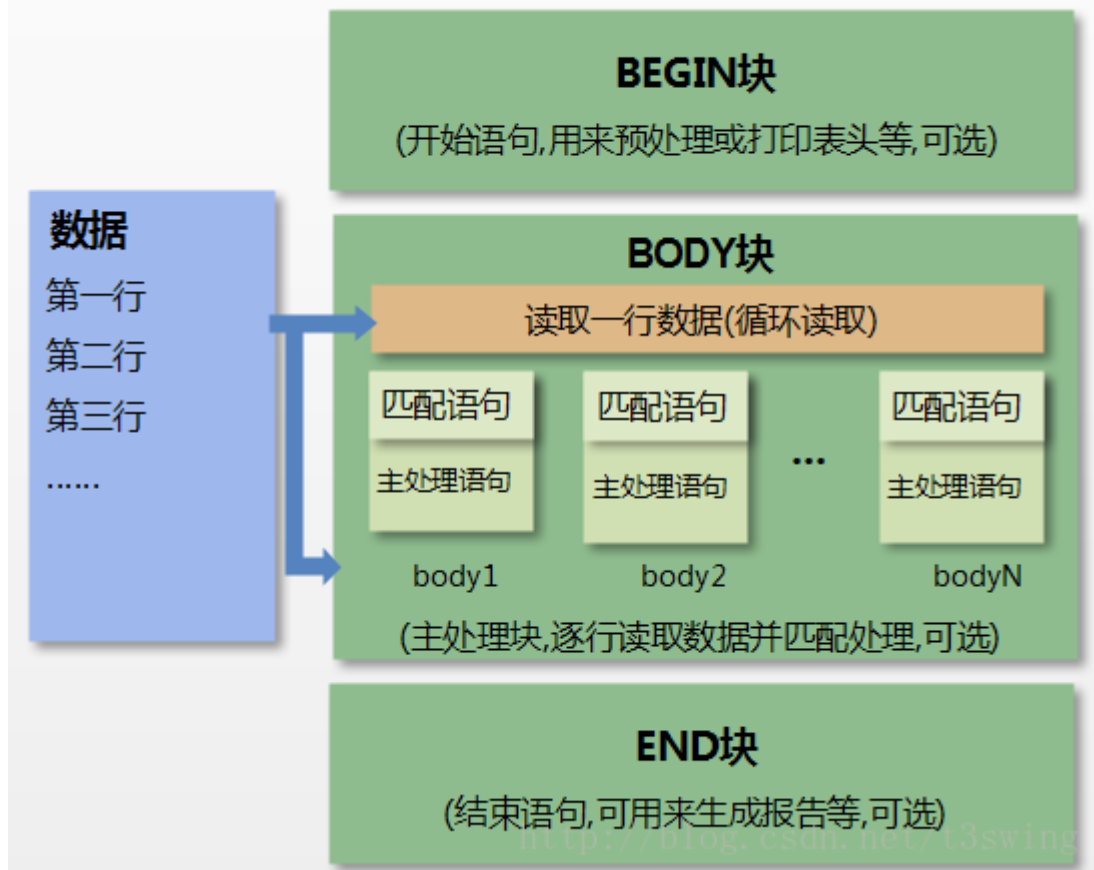
- 读输入文件之前执行的代码段（由BEGIN关键字标识）。
- 主循环执行输入文件的代码段。
- 读输入文件之后的代码段（由END关键字标识）。

命令结构：

```
1 | awk 'BEGIN{ commands } pattern{ commands } END{ commands }'
```

下面的流程图描述出了 AWK 的工作流程：

AWK工作流程



- 1、通过关键字 BEGIN 执行 BEGIN 块的内容，即 BEGIN 后花括号 {} 的内容。
- 2、完成 BEGIN 块的执行，开始执行body块。
- 3、读入有 \n 换行符分割的记录。
- 4、将记录按指定的域分隔符划分域，填充域，\$0 则表示所有域(即一行内容)，\$1 表示第一个域，\$n 表示第 n 个域。
- 5、依次执行各 BODY 块，pattern 部分匹配该行内容成功后，才会执行 awk-commands 的内容。
- 6、循环读取并执行各行直到文件结束，完成body块执行。
- 7、开始 END 块执行，END 块可以输出最终结果。

运算符

运算符	描述
= += -= *= /= %= ^= **=	赋值
?:	C条件表达式
	逻辑或
&&	逻辑与
~ 和 !~	匹配正则表达式和不匹配正则表达式
< <= > >= != ==	关系运算符
空格	连接
+ -	加，减
* / %	乘，除与求余
+ - !	一元加，减和逻辑非
^ ***	求幂
++ --	增加或减少，作为前缀或后缀
\$	字段引用
in	数组成员

过滤第一列大于2的行

```
1 | $ awk '$1>2' log.txt      #命令
2 | 3 Are you like awk
3 | This's a test
4 | 10 There are orange,apple,mongo
```

过滤第一列等于2的行

```
1 | $ awk '$1==2 {print $1,$3}' log.txt      #命令
2 | #输出
3 | 2 is
```

过滤第一列大于2并且第二列等于'Are'的行

```
1 | $ awk '$1>2 && $2=="Are" {print $1,$2,$3}' log.txt      #命令
2 | #输出
3 | 3 Are you
```

内建变量

变量	描述
\$n	当前记录的第n个字段，字段间由FS分隔
\$0	完整的输入记录
ARGC	命令行参数的数目
ARGIND	命令行中当前文件的位置(从0开始算)
ARGV	包含命令行参数的数组
CONVFMT	数字转换格式(默认值为%.6g)ENVIRON环境变量关联数组
ERRNO	最后一个系统错误的描述
FIELDWIDTHS	字段宽度列表(用空格键分隔)
FILENAME	当前文件名
FNR	各文件分别计数的行号
FS	字段分隔符(默认是任何空格)
IGNORECASE	如果为真，则进行忽略大小写的匹配
NF	一条记录的字段的数目
NR	已经读出的记录数，就是行号，从1开始
OFMT	数字的输出格式(默认值是%.6g)
OFS	输出记录分隔符（输出换行符），输出时用指定的符号代替换行符
ORS	输出记录分隔符(默认值是一个换行符)
RLENGTH	由match函数所匹配的字符串的长度
RS	记录分隔符(默认是一个换行符)
RSTART	由match函数所匹配的字符串的第一个位置
SUBSEP	数组下标分隔符(默认值是/034)

格式化变量说明：

- %s 输出字符串
- %i 输出整数
- %f 输出浮点数

%-5s 格式为左对齐且宽度为5的字符串代替（-表示左对齐），不使用则是又对齐。

%-4.2f 格式为左对齐宽度为4，保留两位小数。

```
1 python@ubuntu:~/test$ awk 'BEGIN{printf "%4s %4s %4s %4s %4s %4s %4s %4s\n", "FILENAME", "ARGC", "FNR", "FS", "NF", "NR", "OFS", "ORS", "RS";printf "-----\n"} {printf "%4s %4s %4s %4s %4s %4s %4s %4s\n", FILENAME, ARGC, FNR, FS, NF, NR, OFS, ORS, RS}' log.txt
2 FILENAME ARGC FNR FS NF NR OFS ORS RS
3 -----
4 log.txt 2 1 5 1
5 log.txt 2 2 5 2
```

```

6 | log.txt      2    3          3    3
7 | log.txt      2    4          4    4
8 |
9 | python@ubuntu:~/test$ awk -F: 'BEGIN{printf "%4s %4s %4s %4s %4s %4s %4s %4s\n", "FILENAME", "ARGC", "FNR", "FS", "NF", "NR", "OFS", "ORS", "RS";printf "-----\n"} {printf "%4s %4s %4s %4s %4s %4s %4s\n", FILENAME, ARGC, FNR, FS, NF, NR, OFS, ORS, RS}' log.txt
10 | FILENAME ARGC  FNR   FS    NF    NR   OFS  ORS   RS
11 | -----
12 | log.txt      2    1    :    1    1
13 | log.txt      2    2    :    1    2
14 | log.txt      2    3    :    1    3
15 | log.txt      2    4    :    1    4

```

输出顺序号 NR, 匹配文本行号

```

1 | python@ubuntu:~/test$ awk '{print NR,FNR,$1,$2,$3}' log.txt
2 | 1 1 2 this is
3 | 2 2 3 Are you
4 | 3 3 This's a test
5 | 4 4 10 There are

```

指定输出分割符

```

1 | python@ubuntu:~/test$ cat log.txt
2 | 2 this is a test
3 | 3 Are you like awk
4 | This's a test
5 | 10 There are orange,apple,mongo
6 | python@ubuntu:~/test$ awk '{print $1,$2,$5}' OFS=" $ " log.txt
7 | 2 $ this $ test
8 | 3 $ Are $ awk
9 | This's $ a $
10 | 10 $ There $

```

忽略大小写

```

1 | $ awk 'BEGIN{IGNORECASE=1} /this/' log.txt
2 | -----
3 | 2 this is a test
4 | This's a test

```

正则字符串匹配

~ 表示模式开始。// 中是模式。

输出第二列包含 "th", 并打印第二列与第四列:

```

1 | python@ubuntu:~/test$ awk '$2 ~ /th/ {print $2,$4}' log.txt
2 | this a

```

输出包含 "re" 的行:

```
1 python@ubuntu:~/test$ awk '/re/' log.txt
2 3 Are you like awk
3 10 There are orange,apple,mongo
```

!表示取反

输出第二列不包含 "th", 并打印第二列与第四列:

```
1 python@ubuntu:~/test$ awk '$2 !~ /th/ {print $2,$4}' log.txt
2 Are like
3 a
4 There orange,apple,mongo
```

输出不包含 "re" 的行:

```
1 python@ubuntu:~/test$ awk '!/re/' log.txt
2 2 this is a test
3 This's a test
```

一些实例

计算文件大小

```
1 $ ls -l *.txt | awk '{sum+=$6} END {print sum}'
2 -----
3 666581
```

从文件中找出长度大于80的行

```
1 awk 'length>80' log.txt
```

打印九九乘法表

```
1 seq 9 | sed 'H;g' | awk -v RS=' ' '{for(i=1;i<=NF;i++)printf("%dx%d=%d%s", i, NR, i*NR,
i==NR?"\n":"\t")}'
```

访问日志分析

日志格式

```
1 python@ubuntu:~/test$ head access.log -n1
2 42.236.10.75 "changtou.xiaoxiaoming.xyz" [14/Oct/2019:12:47:18 +0800] "GET
/logo/8@3x.png HTTP/1.1" 200 26053 "https://changtou.xiaoxiaoming.xyz/" "Mozilla/5.0
(Linux; U; Android 8.1.0; zh-CN; EML-AL00 Build/HUAWEIEML-AL00) AppleWebKit/537.36
(KHTML, like Gecko) Version/4.0 Chrome/57.0.2987.108 baidu.sogo.uc.UCBrowser/11.9.4.974
UWS/2.13.1.48 Mobile Safari/537.36 AliApp(DingTalk/4.5.11)
com.alibaba.android.rimet/10487439 Channel/227200 language/zh-CN" "42.236.10.75"
rt="0.000" uct="-" uht="-" urt="-"
```

示例:

```
1 1. 数据清洗
2 awk '($6 ~ /\.html/) && ($8 ~ /200/) {print $0}' access.log > clean.log
3
```

```

4  2.统计PV
5  python@ubuntu:~/test$ awk '{print $0}' clean.log | wc -l
6  700
7  python@ubuntu:~/test$ cut -d ' ' -f 1 clean.log|wc -l
8  700
9
10 3:UV
11 python@ubuntu:~/test$ awk '{print $1}' clean.log |sort|uniq| wc -l
12 155
13 python@ubuntu:~/test$ cut -d ' ' -f 1 clean.log|sort|uniq| wc -l
14 155
15
16 4:获取每天访问网站最多的前10名用户
17 awk '{print $1}' clean.log|sort|uniq -c|sort -k 1nr|head
18 或
19 cut -d ' ' -f 1 clean.log|sort|uniq -c|sort -k 1nr|head

```

awk编程

条件语句IF&ELSE

IF 条件语句语法格式如下:

```

1  if (condition)
2      action

```

也可以使用花括号来执行一组操作:

```

1  if (condition)
2  {
3      action-1
4      action-1
5      .
6      .
7      action-n
8  }

```

判断数字是奇数还是偶数:

```

1  python@ubuntu:~/test$ awk 'BEGIN {num = 10; if (num % 2 == 0) printf "%d 是偶数\n", num}'
2  10 是偶数

```

IF - ELSE 条件语句语法格式如下:

```

1  if (condition)
2      action-1
3  else
4      action-2

```

在条件语句 condition 为 true 时只需 action-1, 否则执行 action-2。

```

1 python@ubuntu:~/test$ awk 'BEGIN {
2   >     num = 11;
3   >     if (num % 2 == 0) printf "%d 是偶数\n", num;
4   >     else printf "%d 是奇数\n", num
5   > }'
6 11 是奇数
7 python@ubuntu:~/test$ awk 'BEGIN {num = 11; if (num % 2 == 0) printf "%d 是偶数\n", num;
8   else printf "%d 是奇数\n", num }'
11 是奇数

```

可以创建多个 IF - ELSE 格式的判断语句来实现多个条件的判断：

```

1 $ awk 'BEGIN {
2   a=30;
3   if (a==10)
4     print "a = 10";
5   else if (a == 20)
6     print "a = 20";
7   else if (a == 30)
8     print "a = 30";
9   }'

```

输出结果：

```

1 python@ubuntu:~/test$ awk 'BEGIN {
2   > a=30;
3   > if (a==10)
4   >   print "a = 10";
5   > else if (a == 20)
6   >   print "a = 20";
7   > else if (a == 30)
8   >   print "a = 30";
9   > }'
10 a = 30

```

循环语句For&While

For 循环的语法如下：

```

1 for (initialisation; condition; increment/decrement)
2   action

```

下面的例子使用 For 循环输出数字 1 至 5：

```

1 python@ubuntu:~/test$ awk 'BEGIN { for (i = 1; i <= 5; ++i) print i }'
2 1
3 2
4 3
5 4
6 5

```

While 循环的语法如下：


```
1 while (condition)
2     action
```

下面是使用 While 循环输出数字 1 到 5 的例子:

```
1 python@ubuntu:~/test$ awk 'BEGIN {i = 1; while (i < 6) { print i; ++i } }'
2 1
3 2
4 3
5 4
6 5
```

在下面的示例中, 当计算的和大于 50 的时候使用 break 结束循环:

```
1 $ awk 'BEGIN {
2     sum = 0; for (i = 0; i < 20; ++i) {
3         sum += i; if (sum > 50) break; else print "Sum =", sum
4     }
5 }'
```

输出结果为:

```
1 python@ubuntu:~/test$ awk 'BEGIN {
2 >     sum = 0; for (i = 0; i < 20; ++i) {
3 >         sum += i; if (sum > 50) break; else print "Sum =", sum
4 >     }
5 > }'
6 Sum = 0
7 Sum = 1
8 Sum = 3
9 Sum = 6
10 Sum = 10
11 Sum = 15
12 Sum = 21
13 Sum = 28
14 Sum = 36
15 Sum = 45
```

Continue 语句用于在循环体内部结束本次循环, 从而直接进入下一次循环迭代。

下面的例子输出 1 到 20 之间的偶数:

```
1 python@ubuntu:~/test$ awk 'BEGIN {for (i = 1; i <= 20; ++i) {if (i % 2 == 0) print i ;
2 else continue} }'
2 2
3 4
4 6
5 8
6 10
7 12
8 14
9 16
10 18
11 20
```

Exit 用于结束脚本程序的执行。

该函数接受一个整数作为参数表示 AWK 进程结束状态。如果没有提供该参数，其默认状态为 0。

下面例子中当和大于 50 时结束 AWK 程序。

```
1 $ awk 'BEGIN {
2     sum = 0; for (i = 0; i < 20; ++i) {
3         sum += i; if (sum > 50) exit(10); else print "Sum =", sum
4     }
5 }'
```

输出结果为：

```
1 python@ubuntu:~/test$ awk 'BEGIN {
2 >     sum = 0; for (i = 0; i < 20; ++i) {
3 >         sum += i; if (sum > 50) exit(10); else print "Sum =", sum
4 >     }
5 > }'
6 Sum = 0
7 Sum = 1
8 Sum = 3
9 Sum = 6
10 Sum = 10
11 Sum = 15
12 Sum = 21
13 Sum = 28
14 Sum = 36
15 Sum = 45
16 python@ubuntu:~/test$ echo $?
17 10
```

awk数组

AWK的数组底层数据结构是散列表，索引可以是数字或字符串。

数组使用的语法格式：

```
1 array_name[index]=value
```

创建数组并访问数组元素：

```
1 $ awk 'BEGIN {
2     sites["taobao"]="www.taobao.com";
3     sites["google"]="www.google.com"
4     print sites["taobao"] "\n" sites["google"]
5 }'
```

删除数组元素语法格式：

```
1 delete array_name[index]
```

下面的例子中，数组中的 google 元素被删除（删除命令没有输出）：

```

1 | $ awk 'BEGIN {
2 |   sites["taobao"]="www.taobao.com";
3 |   sites["google"]="www.google.com"
4 |   delete sites["google"];
5 |   print sites["google"]
6 | }'
```

AWK 本身不支持多维数组，不过我们可以很容易地使用一维数组模拟实现多维数组。

如下示例为一个 3x3 的三维数组：

```

1 | 100 200 300
2 | 400 500 600
3 | 700 800 900
```

以上实例中，`array[0][0]` 存储 100，`array[0][1]` 存储 200，依次类推。为了在 `array[0][0]` 处存储 100，可以使用字符串"0,0"作为索引：`array["0,0"] = 100`。

下面是模拟二维数组的例子：

```

1 | $ awk 'BEGIN {
2 |   array["0,0"] = 100;
3 |   array["0,1"] = 200;
4 |   array["0,2"] = 300;
5 |   array["1,0"] = 400;
6 |   array["1,1"] = 500;
7 |   array["1,2"] = 600;
8 |   # 输出数组元素
9 |   print "array[0,0] = " array["0,0"];
10 |  print "array[0,1] = " array["0,1"];
11 |  print "array[0,2] = " array["0,2"];
12 |  print "array[1,0] = " array["1,0"];
13 |  print "array[1,1] = " array["1,1"];
14 |  print "array[1,2] = " array["1,2"];
15 | }'
```

执行上面的命令可以得到如下结果：

```

1 | array[0,0] = 100
2 | array[0,1] = 200
3 | array[0,2] = 300
4 | array[1,0] = 400
5 | array[1,1] = 500
6 | array[1,2] = 600
```

在数组上可以执行很多操作，比如，使用 `asort` 完成数组元素的排序，或者使用 `asorti` 实现数组索引的排序等等。

AWK 用户自定义函数

自定义函数的语法格式为：

```

1 | function function_name(argument1, argument2, ...)
2 | {
3 |     function body
4 | }
```

以下实例实现了两个简单函数，它们分别返回两个数值中的最小值和最大值。

文件 functions.awk 代码如下：

```
1  # 返回最小值
2  function find_min(num1, num2)
3  {
4      if (num1 < num2)
5          return num1
6      return num2
7  }
8
9  # 返回最大值
10 function find_max(num1, num2)
11 {
12     if (num1 > num2)
13         return num1
14     return num2
15 }
16
17 # 主函数
18 function main(num1, num2)
19 {
20     # 查找最小值
21     result = find_min(10, 20)
22     print "Minimum =", result
23
24     # 查找最大值
25     result = find_max(10, 20)
26     print "Maximum =", result
27 }
28
29 # 脚本从这里开始执行
30 BEGIN {
31     main(10, 20)
32 }
```

执行 functions.awk 文件，可以得到如下的结果：

```
1  $ awk -f functions.awk
2  Minimum = 10
3  Maximum = 20
```

AWK 内置函数

AWK 内置函数主要有以下几种：

- 算数函数
- 字符串函数
- 时间函数
- 位操作函数
- 其它函数

算数函数

函数名	说明	实例
atan2(y, x)	返回 y/x 的反正切。	<pre>\$ awk 'BEGIN { PI = 3.14159265 x = -10 y = 10 result = atan2 (y,x) * 180 / PI; printf "The arc tangent for (x=%f, y=%f) is %f degrees\n", x, y, result }'</pre> 输出结果为: The arc tangent for (x=-10.000000, y=10.000000) is 135.000000 degrees
cos(x)	返回 x 的余弦; x 是弧度。	<pre>\$ awk 'BEGIN { PI = 3.14159265 param = 60 result = cos(param * PI / 180.0); printf "The cosine of %f degrees is %f.\n", param, result }'</pre> 输出结果为: The cosine of 60.000000 degrees is 0.500000.
sin(x)	返回 x 的正弦; x 是弧度。	<pre>\$ awk 'BEGIN { PI = 3.14159265 param = 30.0 result = sin(param * PI /180) printf "The sine of %f degrees is %f.\n", param, result }'</pre> 输出结果为: The sine of 30.000000 degrees is 0.500000.
exp(x)	返回 x 幂函数。	<pre>\$ awk 'BEGIN { param = 5 result = exp(param); printf "The exponential value of %f is %f.\n", param, result }'</pre> 输出结果为: The exponential value of 5.000000 is 148.413159.
log(x)	返回 x 的自然对数。	<pre>\$ awk 'BEGIN { param = 5.5 result = log (param) printf "log(%f) = %f\n", param, result }'</pre> 输出结果为: log(5.500000) = 1.704748
sqrt(x)	返回 x 平方根。	<pre>\$ awk 'BEGIN { param = 1024.0 result = sqrt(param) printf "sqrt(%f) = %f\n", param, result }'</pre> 输出结果为: sqrt(1024.000000) = 32.000000
int(x)	返回 x 的截断至整数的值。	<pre>\$ awk 'BEGIN { param = 5.12345 result = int(param) print "Truncated value =", result }'</pre> 输出结果为: Truncated value = 5
rand()	返回任意数字 n, 其中 0 <= n < 1。	<pre>\$ awk 'BEGIN { print "Random num1 =" , rand() print "Random num2 =" , rand() print "Random num3 =" , rand() }'</pre> 输出结果为: Random num1 = 0.237788 Random num2 = 0.291066 Random num3 = 0.845814
srand([Expr])	将 rand 函数的种子值设置为 Expr 参数的值, 或如果省略 Expr 参数则使用某天的时间。返回先前的种子值。	<pre>\$ awk 'BEGIN { param = 10 printf "srand() = %d\n", srand() printf "srand(%d) = %d\n", param, srand(param) }'</pre> 输出结果为: srand() = 1 srand(10) = 1417959587

字符串函数

函数	说明	实例
gsub(Ere, Repl, [In])	gsub 是全局替换(global substitution) 的缩写。除了正则表达式所有具体值被替代这点，它和 sub 函数完全一样地执行。	<pre>\$ awk 'BEGIN { str = "Hello, world" print "String before replacement = " str gsub("world", "Jerry", str) print "String after replacement = " str }'</pre> 输出结果为: String before replacement = Hello, world String after replacement = Hello, Jerry
sub(regex,sub,string)	sub 函数执行一次子串替换。它将第一次出现的子串用 regex 替换。第三个参数是可选的，默认为 \$0。	<pre>\$ awk 'BEGIN { str = "Hello, world" print "String before replacement = " str sub("world", "Jerry", str) print "String after replacement = " str }'</pre> 输出结果为: String before replacement = Hello, world String after replacement = Hello, Jerry
substr(str, start, l)	substr 函数返回 str 字符串中从第 start 个字符开始长度为 l 的子串。如果没有指定 l 的值，返回 str 从第 start 个字符开始的后缀子串。	<pre>\$ awk 'BEGIN { str = "Hello, world !!!" subs = substr(str, 1, 5) print "Substring = " subs }'</pre> 输出结果为: Substring = Hello
index(String1, String2)	在由 String1 参数指定的字符串（其中有出现 String2 指定的参数）中，返回位置，从 1 开始编号。如果 String2 参数不在 String1 参数中出现，则返回 0（零）。	<pre>\$ awk 'BEGIN { str = "One Two Three" subs = "Two" ret = index(str, subs) printf "Substring \"%s\" found at %d location.\n", subs, ret }'</pre> 输出结果为: Substring "Two" found at 5 location.
length [(String)]	返回 String 参数指定的字符串的长度（字符形式）。如果未给出 String 参数，则返回整个记录的长度（\$0 记录变量）。	<pre>\$ awk 'BEGIN { str = "Hello, world !!!" print "Length = ", length(str) }'</pre> 输出结果为: Substring "Two" found at 5 location.
blength [(String)]	返回 String 参数指定的字符串的长度（以字节为单位）。如果未给出 String 参数，则返回整个记录的长度（\$0 记录变量）。	
substr(String, M, [N])	返回具有 N 参数指定的字符数量子串。子串从 String 参数指定的字符串取得，其字符以 M 参数指定的位置开始。M 参数指定为将 String 参数中的第一个字符作为编号 1。如果未指定 N 参数，则子串的长度将是 M 参数指定的位置到 String 参数的末尾 的长度。	<pre>\$ awk 'BEGIN { str = "Hello, world !!!" subs = substr(str, 1, 5) print "Substring = " subs }'</pre> 输出结果为: Substring = Hello

函数	说明	实例
match(String, Ere)	在 String 参数指定的字符串（Ere 参数指定的扩展正则表达式出现在其中）中返回位置（字符形式），从 1 开始编号，或如果 Ere 参数不出现，则返回 0（零）。RSTART 特殊变量设置为返回值。RLENGTH 特殊变量设置为匹配的字符串的长度，或如果未找到任何匹配，则设置为 -1（负一）。	<pre>\$ awk 'BEGIN { str = "One Two Three" subs = "Two" ret = match(str, subs) printf "Substring \"%s\" found at %d location.\n", subs, ret }'</pre> 输出结果为：Substring "Two" found at 5 location.
split(String, A, [Ere])	将 String 参数指定的参数分割为数组元素 A[1], A[2], ..., A[n]，并返回 n 变量的值。此分隔可以通过 Ere 参数指定的扩展正则表达式进行，或用当前字段分隔符（FS 特殊变量）来进行（如果没有给出 Ere 参数）。除非上下文指明特定的元素还应具有一个数字值，否则 A 数组中的元素用字符串值来创建。	<pre>\$ awk 'BEGIN { str = "One,Two,Three,Four" split(str, arr, ",") print "Array contains following values" for (i in arr) { print arr[i] } }'</pre> 输出结果为：Array contains following values One Two Three Four
tolower(String)	返回 String 参数指定的字符串，字符串中每个大写字符将更改为小写。大写和小写的映射由当前语言环境的 LC_CTYPE 范畴定义。	<pre>\$ awk 'BEGIN { str = "HELLO, WORLD !!!" print "Lowercase string = " tolower(str) }'</pre> 输出结果为：Lowercase string = hello, world !!!
toupper(String)	返回 String 参数指定的字符串，字符串中每个小写字符将更改为大写。大写和小写的映射由当前语言环境的 LC_CTYPE 范畴定义。	<pre>\$ awk 'BEGIN { str = "hello, world !!!" print "Uppercase string = " toupper(str) }'</pre> 输出结果为：Uppercase string = HELLO, WORLD !!!
sprintf(Format, Expr, Expr, ...)	根据 Format 参数指定的 printf 子例程格式字符串来格式化 Expr 参数指定的表达式并返回最后生成的字符串。	<pre>\$ awk 'BEGIN { str = sprintf("%s", "Hello, world !!!") print str }'</pre> 输出结果为：Hello, world !!!
strtonum(str)	strtonum 将字符串 str 转换为数值。如果字符串以 0 开始，则将其当作十进制数；如果字符串以 0x 或 0X 开始，则将其当作十六进制数；否则，将其当作浮点数。	<pre>\$ awk 'BEGIN { print "Decimal num = " strtonum("123") print "Octal num = " strtonum("0123") print "Hexadecimal num = " strtonum("0x123") }'</pre> 输出结果为：Decimal num = 123 Octal num = 83 Hexadecimal num = 291

注：**Ere** 部分可以是正则表达式。

1、gsub、sub 使用

```
1 $ awk 'BEGIN{info="this is a test2012test!";gsub(/[0-9]+/, "",info);print info}'
2 this is a test||test!
```

2、查找字符串 (index 使用)

使用了三元运算符: 表达式 ? 动作1 : 动作2

```
1 $ awk 'BEGIN{info="this is a test2012test!";print index(info,"11111")?"ok":"no found";}'
2 no found
3 $ awk 'BEGIN{info="this is a test2012test!";print index(info,"is")?"ok":"no found";}'
4 ok
5 $ awk 'BEGIN{info="this is a test2012test!";print index(info,"test")?"ok":"no found";}'
6 ok
```

3、正则表达式匹配查找(match 使用)

```
1 $ awk 'BEGIN{info="this is a test2012test!";print match(info,/ [0-9]+/)?"ok":"no
  found";}'
2 ok
```

4、截取字符串(substr使用)

从第 4 个 字符开始，截取 10 个长度字符串。

```
1 $ awk 'BEGIN{info="this is a test2012test!";print substr(info,4,10);}'
2 s is a tes
```

5、字符串分割 (split使用)

```
1 $ awk 'BEGIN{info="this is a test";split(info,tA," ");print length(tA);for(k in tA)
  {print k,tA[k];}}'
2 4
3 2 is
4 3 a
5 4 test
6 1 this
```

分割 info，将 info 字符串使用空格切分为动态数组 tA。注意 awk for ...in 循环，是一个无序的循环。并不是从数组下标 1...n，因此使用时候需要特别注意。

时间函数

函数名	说明	实例
mktime(YYYY MM DD HH MM SS[DST])	生成时间格式	\$ awk 'BEGIN { print "Number of seconds since the Epoch = " mktime("2014 12 14 30 20 10") }' 输出结果 为: Number of seconds since the Epoch = 1418604610
strftime([format , timestamp])	格式化时间输出，将 时间戳转为时间字符 串 具体格式，见下表.	\$ awk 'BEGIN { print strftime("Time = %m/%d/%Y %H:%M:%S", systime()) }' 输出结果为: Time = 12/14/2014 22:08:42
systime()	得到时间戳,返回从 1970年1月1日开始到 当前时间(不计闰年) 的整秒数	awk 'BEGIN{now=systime();print now}' 输出结果为: 1343210982

strftime 日期和时间格式说明符:

序号	描述
%a	星期缩写(Mon-Sun)。
%A	星期全称 (Monday-Sunday) 。
%b	月份缩写 (Jan) 。
%B	月份全称 (January) 。
%c	本地日期与时间。
%C	年份中的世纪部分，其值为年份整除100。
%d	十进制日期(01-31)
%D	等价于 %m/%d/%y.
%e	日期，如果只有一位数字则用空格补齐
%F	等价于 %Y-%m-%d，这也是 ISO 8601 标准日期格式。
%g	ISO8610 标准周所在的年份模除 100 (00-99)。比如，1993 年 1 月 1 日属于 1992 年的第 53 周。所以，虽然它是 1993 年第 1 天，但是其 ISO8601 标准周所在年份却是 1992。同样，尽管 1973 年 12 月 31 日属于 1973 年但是它却属于 1994 年的第一周。所以 1973 年 12 月 31 日的 ISO8610 标准周所在的年是 1974 而不是 1973。
%G	ISO 标准周所在年份的全称。
%h	等价于 %b.
%H	用十进制表示的 24 小时格式的小时(00-23)
%I	用十进制表示的 12 小时格式的小时 (00-12)
%j	一年中的第几天 (001-366)
%m	月份 (01-12)
%M	分钟数 (00-59)
%n	换行符 (ASCII LF)
%p	十二进制表示法 (AM/PM)
%r	十二进制表示法的时间 (等价于 %I:%M:%S %p) 。
%R	等价于 %H:%M。
%S	时间的秒数值 (00-60)
%t	制表符 (tab)
%T	等价于 %H:%M:%S。
%u	以数字表示的星期(1-7),1 表示星期一。
%U	一年中的第几个星期 (第一个星期天作为第一周的开始) , 00-53
%V	一年中的第几个星期 (第一个星期一作为第一周的开始) , 01-53。

序号	描述
%w	以数字表示的星期（0-6），0表示星期日。
%W	十进制表示的一年中的第几个星期（第一个星期一作为第一周的开始），00-53。
%x	本地日期表示
%X	本地时间表示
%y	年份模除 100。
%Y	十进制表示的完整年份。
%z	时区，表示格式为+HHMM（例如，格式要求生成的 RFC 822或者 RFC 1036 时间头）
%Z	时区名称或缩写，如果时区待定则无输出。

位操作函数

函数名	说明	实例
and	位与操作。	<code>\$ awk 'BEGIN { num1 = 10 num2 = 6 printf "(%d AND %d) = %d\n", num1, num2, and(num1, num2) }'</code> 输出结果为: <code>(10 AND 6) = 2</code>
compl	按位求补。	<code>\$ awk 'BEGIN { num1 = 10 printf "compl(%d) = %d\n", num1, compl(num1) }'</code> 输出结果为: <code>compl(10) = 9007199254740981</code>
lshift	左移位操作	<code>\$ awk 'BEGIN { num1 = 10 printf "lshift(%d) by 1 = %d\n", num1, lshift(num1, 1) }'</code> 输出结果为: <code>lshift(10) by 1 = 20</code>
rshift	右移位操作	<code>\$ awk 'BEGIN { num1 = 10 printf "rshift(%d) by 1 = %d\n", num1, rshift(num1, 1) }'</code> 输出结果为: <code>rshift(10) by 1 = 5</code>
or	按位或操作	<code>\$ awk 'BEGIN { num1 = 10 num2 = 6 printf "(%d OR %d) = %d\n", num1, num2, or(num1, num2) }'</code> 输出结果为: <code>(10 OR 6) = 14</code>
xor	按位异或操作	<code>\$ awk 'BEGIN { num1 = 10 num2 = 6 printf "(%d XOR %d) = %d\n", num1, num2, xor(num1, num2) }'</code> 输出结果为: <code>(10 bitwise xor 6) = 12</code>

其他函数

函数名	说明	实例
close(expr)	关闭管道的文件	<pre>\$ awk 'BEGIN { cmd = "tr [a-z] [A-Z]" print "hello, world !!!" & cmd close(cmd, "to") cmd & getline out print out; close(cmd); }'</pre> 输出结果为: HELLO, WORLD !!! 第一条语句 cmd = "tr [a-z] [A-Z]" 在 AWK 中建立了一个双向的通信通道。第二条语句 print 为 tr 命令提供输入。& 表示双向通信。第三条语句 close(cmd, "to") 完成执行后关闭 to 进程。第四条语句 cmd & getline out 使用 getline 函数将输出存储到 out 变量中。接下来的输出语句打印输出的内容, 最后 close 函数关闭 cmd。
delete	用于从数组中删除元素	<pre>\$ awk 'BEGIN { arr[0] = "One" arr[1] = "Two" arr[2] = "Three" arr[3] = "Four" print "Array elements before delete operation:" for (i in arr) { print arr[i] } delete arr[0] delete arr[1] print "Array elements after delete operation:" for (i in arr) { print arr[i] } }'</pre> 输出结果为: Array elements before delete operation: One Two Three Four Array elements after delete operation: Three Four
exit	终止脚本执行, 它可以接受可选的参数 expr 传递 AWK 返回状态。	<pre>\$ awk 'BEGIN { print "Hello, world !!!" exit 10 print "AWK never executes this statement." }'</pre> 输出结果为: Hello, world !!!
flush	刷新打开文件或管道的缓冲区	
getline	读入下一行	使用 getline 从文件 marks.txt 中读入一行并输出: <pre>\$ awk '{getline; print \$0}' marks.txt</pre> , AWK 从文件 marks.txt 中读入一行存储到变量 0 中。在下一条语句中, 我们使用 getline 读入下一行。因此AWK读入第二行并存储到 0 中。最后, AWK 使用 print 输出第二行的内容。这个过程一直到文件结束。
next	停止处理当前记录, 并且进入到下一条记录的处理过程。	当模式串匹配成功后程序并不执行任何操作: <pre>\$ awk '{if (\$0 ~/Shyam/) next; print \$0}' marks.txt</pre>
nextfile	停止处理当前文件, 从下一个文件第一个记录开始处理。	首先创建两个文件。file1.txt 内容如下: file1:str1 file1:str2 file1:str3 file1:str4 文件 file2.txt 内容如下: file2:str1 file2:str2 file2:str3 file2:str4 现在我们来测试 nextfile 函数。 <pre>\$ awk '{ if (\$0 ~ /file1:str2/) nextfile; print \$0 }' file1.txt file2.txt</pre> 输出结果为: file1:str1 file2:str1 file2:str2 file2:str3 file2:str4

函数名	说明	实例
return	从用户自定义的函数中返回值。请注意，如果没有指定返回值，那么的返回值是未定义的。	创建文件 functions.awk，内容如下： <pre>function addition(num1, num2) { result = num1 + num2 return result } BEGIN { res = addition(10, 20) print "10 + 20 = " res }</pre> 执行该文件： <pre>\$ awk -f functions.awk 10 + 20 = 30</pre>
system	执行特定的命令然后返回其退出状态。返回值为 0 表示命令执行成功；非 0 表示命令执行失败。	<pre>\$ awk 'BEGIN { ret = system("date"); print "Return value = " ret }'</pre> 输出结果为： <pre>Sun Dec 21 23:16:07 IST 2014 Return value = 0</pre>

vim全套笔记

VIM快速复习

什么是 vim?

Vim是从 vi 发展出来的一个文本编辑器。代码补完、编译及错误跳转等方便编程的功能特别丰富，在程序员中被广泛使用。vim 的官方网站 (<http://www.vim.org>)

vim 键盘图：

vi / vim 键盘图

Esc
命令
模式

~ 转换大小写	! 外部过滤器	@ 运行宏	# prev ident	\$ 行尾	% 括号匹配	^ "软"行首	& 重复:s	* next ident	(句首	) 下一句首	"soft" bol down	+ 后一行行首
跳转到标注	1	2	3	4	5	6	7	8	9	0 "硬"行首	- 前一行行首	= 自动 ³ 格式化
Q 切换至ex模式	W 下一单词	E 词尾	R 替换模式	T back 'till	Y 拷贝行	U 撤销行内命令	I 到行首插入	O 分段(前)	P 粘贴(前)	{ 段首	}	段尾
q 录制宏	w 下一单词	e 词尾	r 替换字符	t 'till	y 拷贝 ^{1,3}	u 撤销命令	i 插入模式	o 分段(后)	p 粘贴(后)	. 杂项	]	杂项
A 在行尾附加	S 删除行并插入	D 删除至行尾	F 行内字符反向查找	G 文尾/行号	H 屏幕顶行	J 合并两行	K 帮助	L 屏幕底行	: ex 命令	" 寄存器 ¹ 标识	行首/列	
a 附加	s 删除字符并插入	d 删除 ^{1,3}	f 行内字符查找	g 附加 ⁶ 命令	h ←	j ↓	k ↑	l →	; 重复u/T/f/F	' 跳转到标注的行首	\ 未用!	
Z 退出 ⁴	X 退格	C 修改至行末	V 可视行模式	B 前一单词	N 查找上一处	M 屏幕中间行	< 反缩进 ³	> 缩进 ³	? 向前搜索			
z 附加命令 ⁵	x 删除(字符)	c 修改 ^{1,3}	v 可视模式	b 前一单词	n 查找下一处	m 设置标注	, 反向u/T/f/F	. 重复命令	/ 向后搜索			

动作 移动光标, 或者定义操作的范围

命令 直接执行的命令,
红色命令 进入编辑模式

操作 后面跟随表示操作范围的指令

extra 特殊功能,
需要额外的输入

q* 后跟字符参数

w,e,b命令

小写(b): quux(foo, bar, baz);
大写(B): QUUX(FOO, BAR, BAZ);

主要ex命令:

:w (保存), :q (退出), :q! (不保存退出)
:e f (打开文件 f),
:%s/x/y/g ('y' 全局替换 'x'),
:h (帮助 in vim), :new (新建文件 in vim),

其它重要命令:

CTRL-R: 重复 (vim),
CTRL-F/-B: 上翻/下翻,
CTRL-E/-Y: 上滚/下滚,
CTRL-V: 块可视模式 (vim only)

可视模式:

漫游后对选中的区域执行操作 (vim only)

备注:

- (1) 在 拷贝/粘贴/删除 命令前使用 "x (x=a..z,*) 使用命令的寄存器('剪贴板') (如: "ay\$ 拷贝剩余的行内容至寄存器 'a')
- (2) 命令前添加数字 多遍重复操作 (e.g.: 2p, d2w, 5l, d4j)
- (3) 重复本字符在光标所在行执行操作 (dd = 删除本行, >> = 行首缩进)
- (4) ZZ 保存退出, ZQ 不保存退出
- (5) zt: 移动光标所在行至屏幕顶端, zb: 底端, zz: 中间
- (6) gg: 文首 (vim only), gf: 打开光标处的文件名 (vim only)

原图: www.viemu.com 翻译: fdl (linuxsir)

基本上vi可以分为三种状态:

- 命令模式 (command mode)
- 插入模式 (Insert mode)
- 底行模式 (last line mode)

按:冒号即可进入last line mode

```
1 :set nu 列出行号
2 :set nonu 取消行号
3 :#7 跳到文件中的第7行
4 /keyword 查找字符 按n向下
5 ?keyword 查找字符 按N向下
6 :n1,n2/word1/word2/gc 替换指定范围单词, c表示提示
7 :w 保存文件
8 :w filename 以指定的文件名另存
9 :n1,n2 w [filename] 将 n1 到 n2行另存
10 :r [filename] 读入另一个文件加到光标所在行后面
11 :! ls /home 在vi当中察看ls输出信息!
12 :q离开vi
13 :wq 和 :ZZ 和 :x 保存并退出vi
14 !强制执行
15 :% s/^/#/g 来在全部内容的行首添加 # 号注释
16 :1,10 s/^/#/g 在1~10 行首添加 # 号注释
```

从command mode进入Insert mode

按i在当前位置编辑

按a在当前位置的下一个字符编辑

按o插入新行，从行首开始编辑

按R(Replace mode)：R会一直取代光标所在的文字，直到按下ESC为止；(常用)

按ESC键退回command mode

h←j↓k↑→前面加数字移动指定的行数或字符数

1、翻页bu上下整页，ud上下半页

- | | |
|---|---------------|
| 1 | ctrl+b: 上移一页。 |
| 2 | ctrl+f: 下移一页。 |
| 3 | ctrl+u: 上移半页。 |
| 4 | ctrl+d: 下移半页。 |

2、行定位

- | | |
|---|-------------------------------|
| 1 | 7gg或7G: 定位第7行首字符。(可能只在vim中有效) |
| 2 | G: 移动到文章的最后。 |
| 3 | 7H: 当前屏幕的第7行行首 |
| 4 | M: 当前屏幕中间行的行首 |
| 5 | 7L: 当前屏幕的倒数第7行行首 |

3、当前行定位

- | | |
|---|--------------------|
| 1 | \$: 移动到光标所在行的“行尾”。 |
| 2 | 0或^: 移动到光标所在行的“行首” |
| 3 | w: 光标跳到下个单词的开头 |
| 4 | e: 光标跳到下个单词的字尾 |
| 5 | b: 光标回到上个单词的开头 |

4、编辑

- | | |
|----|-------------------------------|
| 1 | x: 剪切当前字符 |
| 2 | 7x: 剪切从当前位置起7个字符 |
| 3 | 大写的X, 表示从前面一个字符开始往前计算 |
| 4 | dd: 剪切光标所在行。 |
| 5 | 7dd: 从光标所在行开始剪切7行 |
| 6 | d7G 删除光标所在到第7行的所有数据 |
| 7 | yw: 复制当前单词 |
| 8 | 7yw: 复制从当前位置起7个单词 |
| 9 | yy: 复制当前行 |
| 10 | 6yy: 从当前行起向下复制6行 |
| 11 | y7G 复制光标所在列到第7列的所有数据 |
| 12 | p: 粘贴 |
| 13 | u: 撤销 |
| 14 | ctrl+r: 取消撤销 |
| 15 | cw: 删除当前单词(从光标位置开始计算)，并进入插入模式 |
| 16 | c7w: 删除7个单词并进入插入模式 |

多行编辑，vim支持，vi不支持

按ctrl+V进入块模式，上下键选中快，按大写G选择到末尾，上下左右键移动选择位置

按大写I进去编辑模式，输入要插入的字符，编辑完成按ESC退出

选中要替换的字符后，按c键全部会删除，然后输入要插入的字符，编辑完成按ESC退出

选中要替删除的字符后，按delete键，则会全部删除

按shift+V可进入行模式，对指定行操作

vim练习

1、创建目录/tmp/test，将/etc/man.config复制到该目录下

```
1 # mkdir -p /tmp/test
2
3 # cp /etc/man.config /tmp/test/
4
5 # cd /tmp/test/
```

2、用vim编辑man.config文件：

```
1 vim man.config
```

3、设置显示行号；移动到第58行，向右移动40个字符，查看双引号内的是什么目录；

```
1 : set nu
2 58G 或58gg
3 40-> 或40空格 目录为: /dir/bin/foo
```

4、移动到第一行，并向下查找“bzip2”这个字符串，它在第几行；

```
1 移动到最后一行，并向上查找该字符串；
2 gg 或1G
3 /bzip 137行 ?bzip2
```

5、将50行到100行之间的man更改为MAN，并且 逐个挑选 是否需要修改；

```
1 若在挑选过程中一直按y，结果会在最后一行出现改变了几个man？
2 :50,100s/man/MAN/gc 25次替换
```

6、修改完后，突然反悔了，要全部复原，有哪些方法？

```
1 一直按u键
2 或者
3 :q! 强制不保存退出后，再重新打开该文件
```

7、复制65到73这9行的内容（含有MANPATH_MAP），并且粘贴到最后一行之后；

```
1 65gg或65G到该行后，9yy,G 移动到最后一行，p粘贴
```

8、21行到42行之间开头为#符号的批注数据不要了，如何删除；

```
1 21G到该行 22dd
```

9、将这个文件另存为man.test.config的文件

```
1 | :w man.test.config
```

10、到第27行，并且删除15个字符，结果出现的第一个字符是什么？

```
1 | 27gg 后15x
```

11、在第一行新增一行，在该行内输入“I am a student ”

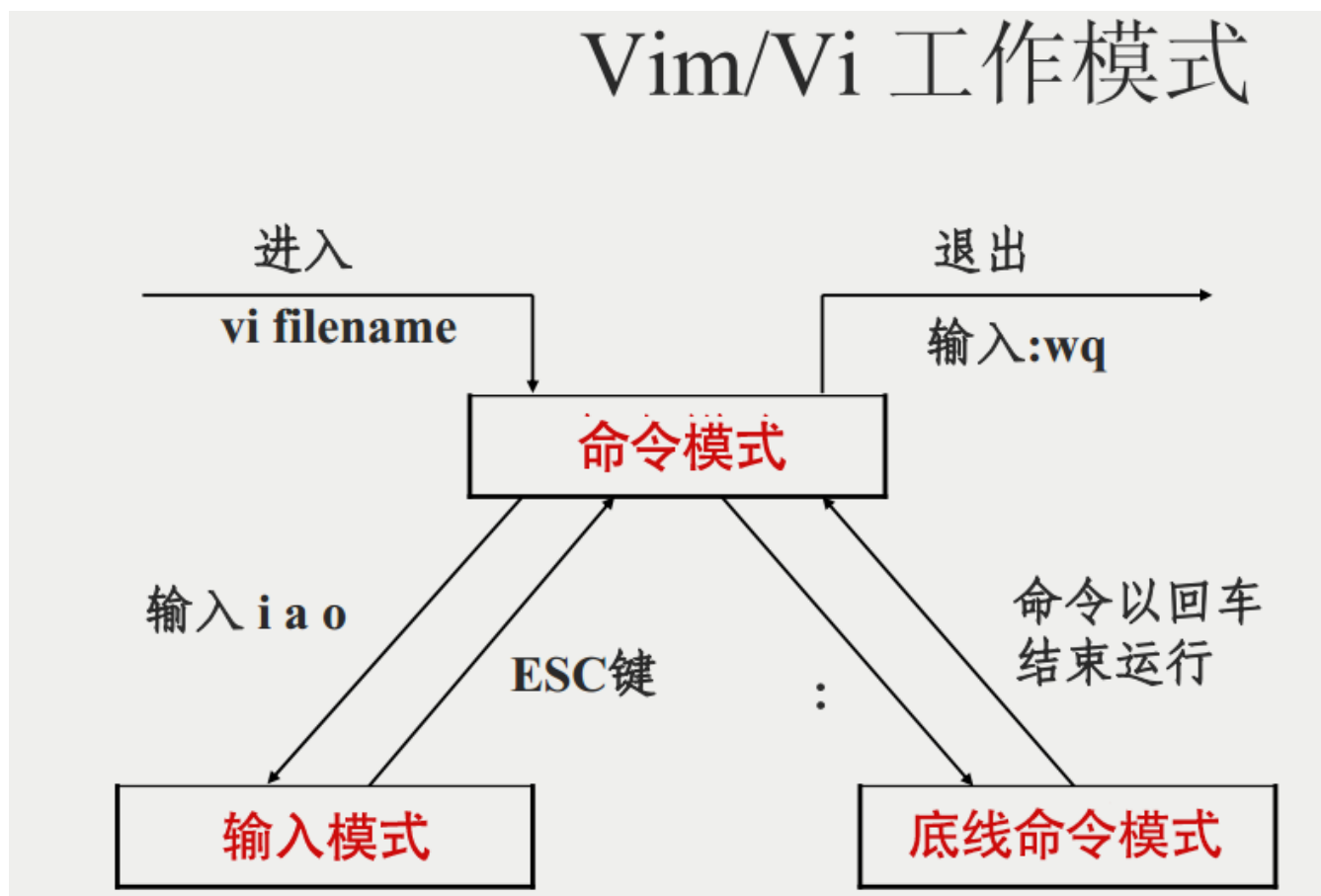
```
1 | gg到第一行 o输入即可 说明：o是在当前行之后插入一行，O是在当前行之前插入一行
```

12、保存并退出

```
1 | :wq
```

vi/vim的三种模式

vi/vim主要分为三种模式，分别是**命令模式 (Command mode)**，**输入模式 (Insert mode)** 和**底线命令模式 (Last line mode)**。



这三种模式的作用分别是：

命令模式

用户刚刚启动 vi/vim，便进入了命令模式。任何时候，不管用户处于何种模式，只要按一下ESC键，即可使Vi进入命令模式；

此状态下敲击键盘动作会被Vim识别为命令，输入`:`可切换到**底线命令模式**，以在最底一行输入命令。

若想要编辑文本：启动Vim，进入了命令模式，按下`i`，切换到输入模式。

输入模式

在命令模式下输入插入命令`i`、附加命令`a`、打开命令`o`、修改命令`c`、取代命令`r`或替换命令`s`都可以进入文本输入模式。在该模式下，用户输入的任何字符都被Vi当做文件内容保存起来，并将其显示在屏幕上。在文本输入过程中，若想回到命令模式下，按键ESC即可。

底行模式

在命令模式下按下`:`（英文冒号）就进入了底行命令模式。

底线命令模式可以输入单个或多个字符的命令，可用的命令非常多。

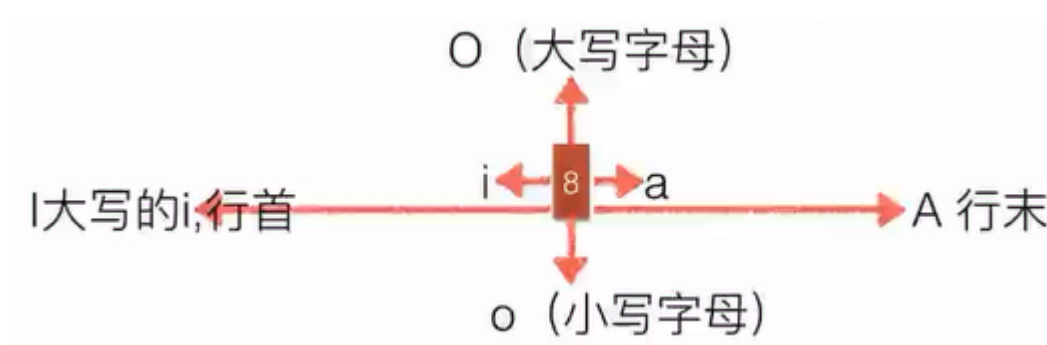
在底线命令模式中，基本的命令有（已经省略了冒号）：

- `q` 退出程序
- `w` 保存文件

按ESC键可随时退出底线命令模式。

vim基础操作

进入输入模式(Insert mode)



- `i`: 插入光标前一个字符
- `I`: 插入行首
- `a`: 插入光标后一个字符
- `A`: 插入行末
- `o`: 向下新开一行,插入行首
- `O`: 向上新开一行,插入行首

在进入输入模式后，vi 画面的左下角处会出现『--INSERT--』的字样

进入替换模式(Replace mode)

- `r`: 只会取代光标所在的那一个字符一次
- `R`: 会一直取代光标所在的文字，直到按下ESC为止

在进入输入模式后，vi 画面的左下角处会出现『--REPLACE--』的字样

命令模式下常用命令

移动光标

移动光标的方法	
h 或 向左箭头键(←)	光标向左移动一个字符
j 或 向下箭头键(↓)	光标向下移动一个字符
k 或 向上箭头键(↑)	光标向上移动一个字符
l 或 向右箭头键(→)	光标向右移动一个字符
向下移动 30 行，可以使用 "30j" 或 "30↓" 的组合按键	
[Ctrl] + [f]	屏幕『向下』移动一页，相当于 [Page Down]按键 (常用)
[Ctrl] + [b]	屏幕『向上』移动一页，相当于 [Page Up] 按键 (常用)
[Ctrl] + [d]	屏幕『向下』移动半页
[Ctrl] + [u]	屏幕『向上』移动半页
+	光标移动到非空格符的下一行
-	光标移动到非空格符的上一行
n	表示空格光标向右移动这一行的 n 个字符。例如 20 则光标会向后面移动 20 个字符距离。
0 或功能键[Home]	这是数字『0』：移动到这一行的最前面字符处 (常用)
\$ 或功能键[End]	移动到这一行的最后面字符处(常用)
H	光标移动到这个屏幕的最上方那一行的第一个字符
M	光标移动到这个屏幕的中央那一行的第一个字符
L	光标移动到这个屏幕的最下方那一行的第一个字符
G	移动到这个文档的最后一行(常用)
nG	n 为数字。移动到这个文件的第 n 行。例如 20G 则会移动到这个文件的第 20 行(可配合 :set nu)
gg	移动到这个文档的第一行，相当于 1G
n	n 为数字。光标向下移动 n 行(常用)

删除操作

删除操作	删除后会添加到剪切板，相当于剪切
x, X	x为向后删除一个字符 (相当于 [del] 按键), X为向前删除一个字符(相当于 [backspace])
nx	n 为数字，连续向后删除 n 个字符。例如10x表示连续删除 10 个字符。
dd	删除光标所在的一整行(常用)
ndd	n 为数字。删除光标所在的向下 n 行，例如 20dd 则是删除 20 行
d1G	删除光标所在行到首行的所有数据
dG	删除光标所在行到最后一行的所有数据
d\$	删除光标所在位置到该行的最后一个字符
d0	删除光标所在位置到该行的最前面一个字符

撤销&复原&重复

撤销&复原	
u	撤销操作，相对于普通编辑器里面的ctrl+z
Ctrl+r	恢复操作，相对于普通编辑器里面的ctrl+y
.	就是小数点！可重复前一个动作

复制&粘贴

复制&粘贴	
yy	复制光标所在行
nyy	n 为数字。复制光标所在的向下 n 行，例如 20yy 则是复制 20 行
y1G	复制光标所在行到第一行的所有数据
yG	复制光标所在行到最后一行的所有数据
y0	复制光标所在的那个字符到该行行首的所有数据
y\$	复制光标所在的那个字符到该行行尾的所有数据
p, P	p 为将已复制的数据在光标下一行贴上，P 则为贴在光标上一行！

合成行

- J: 将光标所在行与下一行的数据结合成同一行

搜索

搜索	
/word	向光标之下寻找一个名称为 word 的字符串。
?word	向光标之上寻找一个字符串名称为 word 的字符串。
n	代表重复前一个搜寻的动作，根据前面输入的/word还是?word向下或向上搜索下一个匹配的字符串。
N	表示反向搜索，与n的搜索方向相反。

替换

替换	
:n1,n2s/word1/word2/g	在第 n1 与 n2 行之间寻找word1并替换为word2! 比如 『:100,200s/vbird/VBIRD/g』表示在100到200行之间将vbird替换为VBIRD
:1,\$s/word1/word2/g 或 :%s/word1/word2/g	\$表示最后一行， %s表示所有行。
:1,\$s/word1/word2/gc 或 :%s/word1/word2/gc	gc中的c表示取代前显示提示字符给用户确认 (confirm) !

底行命令模式的常用操作

底行命令模式	
:w	保存编辑数据
:w!	若文件属性为『只读』时，强制写入该文件。不过，到底能不能写入， 还是跟你对该文件的文件权限有关啊！
:q	离开 vi
:q!	若曾修改过文件， 又不想储存， 使用!为强制离开不储存文件。
惊叹号 (!) 在 vi 当中， 常常具有『强制』的意思 ~	
:wq	储存后离开， 若为 :wq! 则为强制储存后离开
ZZ	若文件没有更动， 则不储存离开， 若文件已经被更动过， 则储存后离开！
:w [filename]	另存为
:r [filename]	将另一个文件『filename』的数据加到光标所在行后面
:n1,n2 w [filename]	将 n1 到 n2 行的内容储存成 filename 这个文件。
:! command	暂时离开 vi 到指令行模式下执行 command 的显示结果！ 例如 『:! ls /home』即可在 vi 当中察看 /home 底下以 ls 输出的文件信息！
:set nu	会在每一行的前缀显示该行的行号
:set nonu	取消行号显示

示例：

```
1 | 将当前路径插入到光标的下一行
2 | :r!pwd
```

可视模式

v 进入字符可视化模式： 文本选择是以字符为单位的。
V 进入行可视化模式： 文本选择是以行为单位的。
Ctrl+v 进入块可视化模式： 选择一个矩形内的文本。

可视模式下可进行如下操作：

可视模式操作	
A	在选定的部分后面插入内容
I	在选定的部分前面插入内容
d	删除选定的部分
c	删除选定的部分并进入插入模式（有批量替换效果）
r	把选定的部分全部替换为指定的单个字符
>>	向右缩进一个单位，更适合行可视化模式
<<	向左缩进一个单位，更适合行可视化模式
gu	选中区域转为小写
gU	选中区域转为大写
g~	大小写互调

可视模式下，选中的区域是由两个端点来界定的（一个在左上角，一个在右下角），在默认情况下只可以控制右下角的端点，而使用  按键则可以在左上角和右下角之间切换控制端点。

Linux系统启动过程

Linux系统的启动过程可以分为5个阶段：

- 内核的引导。
- 运行 init。
- 系统初始化。
- 建立终端。
- 用户登录系统。

加载内核

当计算机打开电源后，首先是BIOS开机自检，按照BIOS中设置的启动设备（通常是硬盘）来启动。

操作系统接管硬件以后，首先读入 /boot 目录下的内核文件。



启动初始化进程init

内核文件加载以后，就开始运行第一个程序 `/sbin/init`，它的作用是初始化系统环境。

init程序首先是需要读取配置文件`/etc/inittab`。

CentOS 各版本init配置文件的位置：

- **SysV:** init, CentOS 5之前, 配置文件： `/etc/inittab`。
- **Upstart:** init, CentOS 6, 配置文件： `/etc/inittab`, `/etc/init/*.conf`。
- **Systemd:** systemd, CentOS 7, 配置文件： `/usr/lib/systemd/system`、`/etc/systemd/system`。



由于init是第一个运行的程序，它的进程编号（pid）就是1。其他所有进程都从它衍生，都是它的子进程。

确定运行级别

许多程序需要开机启动。它们在Windows叫做"服务"（service），在Linux就叫做"守护进程"（daemon）。

init进程的一大任务，就是去运行这些开机启动的程序。

但是，不同的场合需要启动不同的程序，比如用作服务器时，需要启动Apache，用作桌面就不需要。

Linux允许为不同的场合，分配不同的开机启动程序，这就叫做"运行级别"（runlevel）。也就是说，启动时根据"运行级别"，确定要运行哪些程序。



Linux系统有7个运行级别(runlevel)：

- 运行级别0：系统停机状态，系统默认运行级别不能设为0，否则不能正常启动
- 运行级别1：单用户工作状态，root权限，用于系统维护，禁止远程登陆
- 运行级别2：多用户状态(没有NFS)
- 运行级别3：完全的多用户状态(有NFS)，登陆后进入控制台命令行模式
- 运行级别4：系统未使用，保留
- 运行级别5：X11控制台，登陆后进入图形GUI模式
- 运行级别6：系统正常关闭并重启，默认运行级别不能设为6，否则不能正常启动

可以使用运行级别执行关机或重启：

```
1 | init 0  关机
2 | init 6  重启
```

加载开机启动程序

在init的配置文件中有这么一行：`si::sysinit:/etc/rc.d/rc.sysinit` 它调用执行了`/etc/rc.d/rc.sysinit`，而`rc.sysinit`是一个bash shell的脚本，它主要是完成一些系统初始化的工作，`rc.sysinit`是每一个运行级别都要首先运行的重要脚本。

它主要完成的工作有：激活交换分区，检查磁盘，加载硬件模块以及其它一些需要优先执行任务。

```
1 | 15:5:wait:/etc/rc.d/rc 5
```

这一行表示以5为参数运行`/etc/rc.d/rc`，`/etc/rc.d/rc`是一个Shell脚本，它接受5作为参数，去执行`/etc/rc.d/rc5.d/`目录下的所有的rc启动脚本，`/etc/rc.d/rc5.d/`目录中的这些启动脚本实际上都是一些连接文件，而不是真正的rc启动脚本，真正的rc启动脚本实际上都是放在`/etc/rc.d/init.d/`目录下。

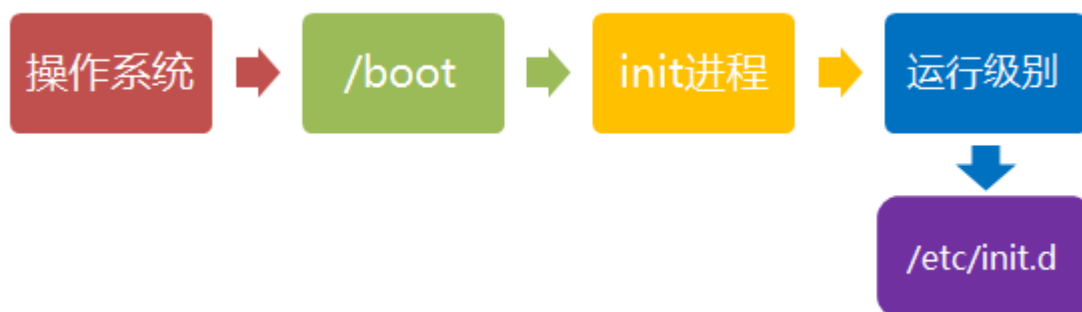
而这些rc启动脚本有着类似的用法，它们一般能接受start、stop、restart、status等参数。

`/etc/rc.d/rc5.d/`中的rc启动脚本通常是K或S开头的连接文件，对于以S开头的启动脚本，将以start参数来运行。

而如果发现存在相应的脚本也存在K打头的连接，而且已经处于运行态了(以`/var/lock/subsys/`下的文件作为标志)，则将首先以stop为参数停止这些已经启动了的守护进程，然后再重新运行。

这样做是为了保证是当init改变运行级别时，所有相关的守护进程都将重启。

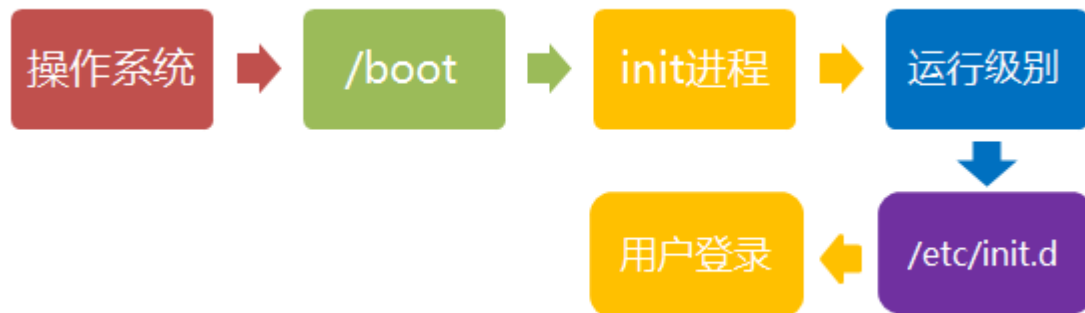
至于在每个运行级中将运行哪些守护进程，用户可以通过chkconfig或setup中的"System Services"来自行设定。



用户登录

一般来说，用户的登录方式有三种：

- (1) 命令行登录
- (2) ssh登录
- (3) 图形界面登录



对于运行级别为5的图形方式用户来说，他们的登录是通过一个图形化的登录界面。登录成功后可以直接进入 KDE、Gnome 等窗口管理器。

而本文主要讲的还是文本方式登录的情况：当我们看到mingetty的登录界面时，我们就可以输入用户名和密码来登录系统了。

Linux 的账号验证程序是login，login会接收mingetty传来的用户名作为用户名参数。

然后login会对用户名进行分析：如果用户名不是root，且存在 /etc/nologin 文件，login 将输出 nologin 文件的内容，然后退出。

这通常用来系统维护时防止非root用户登录。只有/etc/securetty中登记了的终端才允许 root 用户登录，如果不存在这个文件，则root用户可以在任何终端上登录。

/etc/usertty文件用于对用户作出附加访问限制，如果不存在这个文件，则没有其他限制。

图形模式与文字模式的切换方式

Linux预设提供了六个命令窗口终端机让我们来登录。

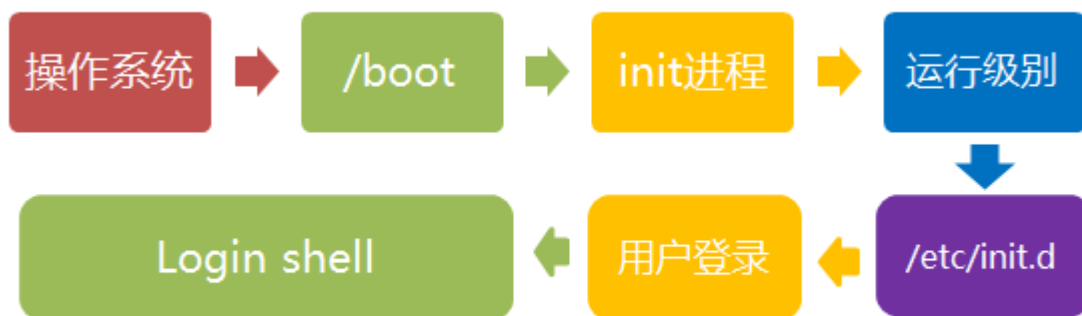
默认我们登录的就是第一个窗口，也就是tty1，这个六个窗口分别为tty1,tty2 ... tty6，你可以按下Ctrl + Alt + F1 ~ F6来切换它们。

如果你安装了图形界面，默认情况下是进入图形界面的，此时你就可以按Ctrl + Alt + F1 ~ F6来进入其中一个命令窗口界面。

当你进入命令窗口界面后再返回图形界面只要按下Ctrl + Alt + F7 就回来了。

如果你用的vmware 虚拟机，命令窗口切换的快捷键为 Alt + Space + F1~F6. 如果你在图形界面下请按Alt + Shift + Ctrl + F1~F6 切换至命令窗口。

login shell



shell，简单说就是命令行界面，让用户可以直接与操作系统对话。用户登录时打开的shell，就叫做login shell。

(1) 命令行登录：首先读入 `/etc/profile`，这是对所有用户都有效的配置；然后依次寻找下面三个文件，这是针对当前用户的配置。

```
1 ~/.bash_profile
2 ~/.bash_login
3 ~/.profile
```

需要注意的是，这三个文件只要有一个存在，就不再读入后面的文件了。比如，要是 `~/.bash_profile` 存在，就不会再读入后面两个文件了。

(2) ssh登录：与第一种情况完全相同。

(3) 图形界面登录：只加载 `/etc/profile` 和 `~/.profile`。也就是说，`~/.bash_profile` 不管有没有，都不会运行。

Linux关机

```
1 sync 将数据由内存同步到硬盘中。
2
3 shutdown 关机指令，你可以man shutdown 来看一下帮助文档。例如你可以运行如下命令关机：
4 shutdown -h 10 'This server will shutdown after 10 mins' 计算机将在10分钟后关机，并且显示信息
   在登陆用户的当前屏幕中。
5 shutdown -h now 立马关机
6 shutdown -h 20:25 系统会在今天20:25关机
7 shutdown -h +10 十分钟后关机
8 shutdown -r now 系统立马重启
9 shutdown -r +10 系统十分钟后重启
10 reboot 就是重启，等同于 shutdown -r now
11 halt 关闭系统，等同于shutdown -h now 和 poweroff
```

最后总结一下，不管是重启系统还是关闭系统，首先要运行**sync**命令，把内存中的数据写到磁盘中。

关机的命令有 **shutdown -h now** **halt** **poweroff** 和 **init 0**，重启系统的命令有 **shutdown -r now** **reboot** **init 6**。

计算机启动的流程



boot是bootstrap（鞋带）的缩写，它来自一句谚语：

"pull oneself up by one's bootstraps"

字面意思是"拽着鞋带把自己拉起来"，这当然是不可能的事情。最早的时候，工程师们用它来比喻，计算机启动是一个很矛盾的过程：必须先运行程序，然后计算机才能启动，但是计算机不启动就无法运行程序！

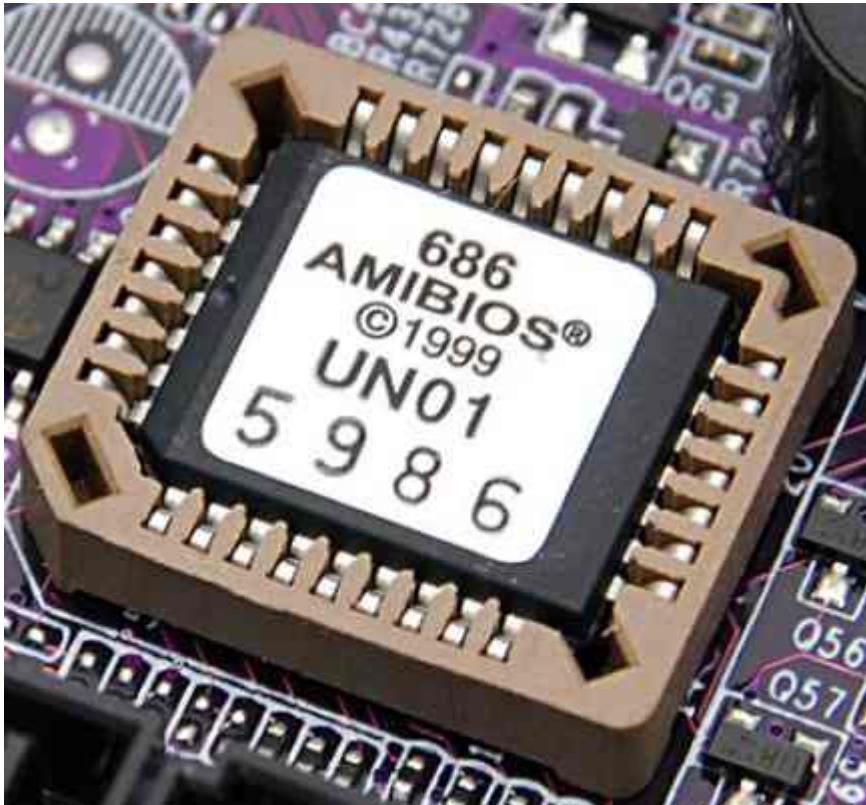
早期真的是这样，必须想尽各种办法，把一小段程序装进内存，然后计算机才能正常运行。所以，工程师们把这个过程叫做"拉鞋带"，久而久之就简称为boot了。

计算机的整个启动过程分成四个阶段。

第一阶段：BIOS

上世纪70年代初, "只读内存" (read-only memory, 缩写为ROM) 发明, 开机程序被刷入ROM芯片, 计算机通电后, 第一件事就是读取它。

BIOS全称是Basic Input/Output System，即基本输入输出系统，即下图芯片里的程序。



硬件自检

BIOS程序首先检查，计算机硬件能否满足运行的基本条件，这叫做"硬件自检"（Power-On Self-Test），缩写为POST。

如果硬件出现问题，主板会发出不同含义的**蜂鸣**，启动中止。如果没有问题，屏幕就会显示出CPU、内存、硬盘等信息。

```

Diskette Drive B : None
Serial Port(s) : 3F0 2F0
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 370
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None

Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled

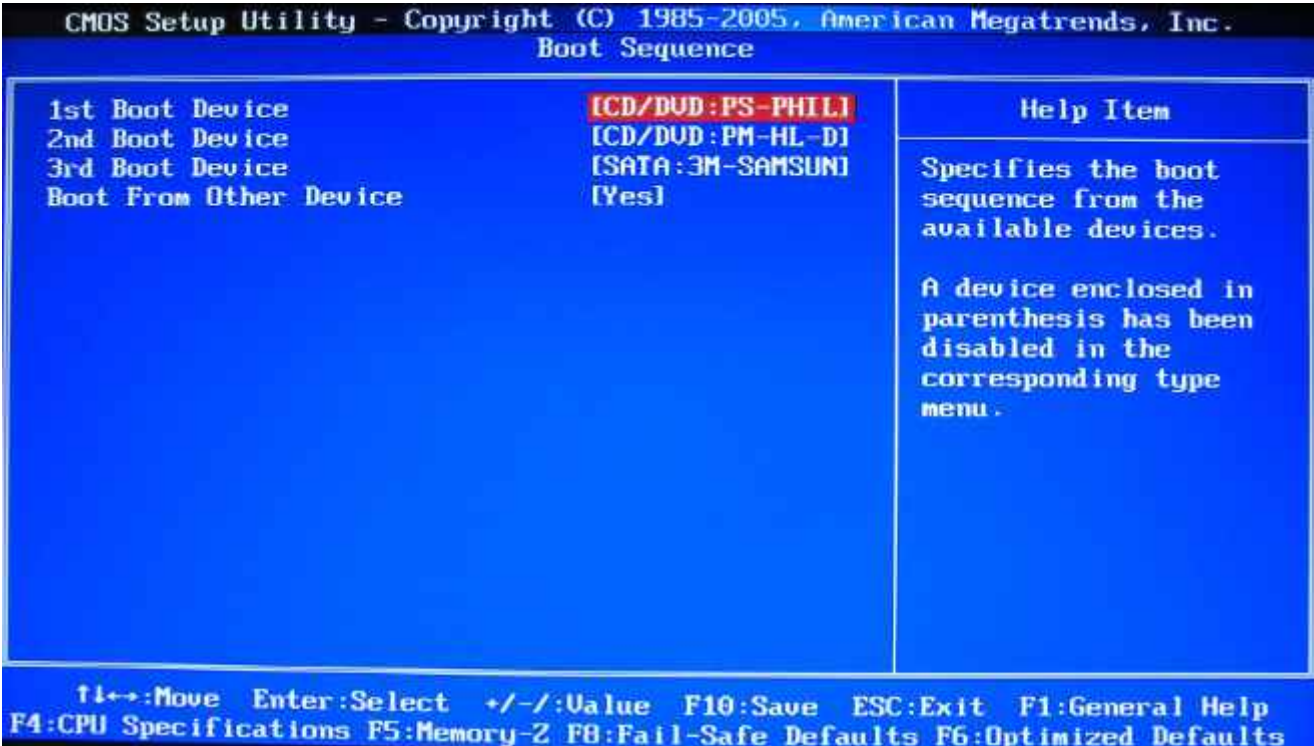
PCI Devices Listing ...
Bus Dev Fun Vendor Device SVID SSID Class Device Class IRQ
0 27 0 8086 2668 1458 A005 0403 Multimedia Device 5
0 29 0 8086 2658 1458 2658 0C03 USB 1.1 Host Cntrlr 9
0 29 1 8086 2659 1458 2659 0C03 USB 1.1 Host Cntrlr 11
0 29 2 8086 265A 1458 265A 0C03 USB 1.1 Host Cntrlr 11
0 29 3 8086 265B 1458 265A 0C03 USB 1.1 Host Cntrlr 5
0 29 7 8086 265C 1458 5006 0C03 USB 1.1 Host Cntrlr 9
0 31 2 8086 2651 1458 2651 0101 IDE Cntrlr 14
0 31 3 8086 266A 1458 266A 0C05 SMBus Cntrlr 11
1 0 0 10DE 0421 10DE 0479 0300 Display Cntrlr 5
2 0 0 1283 8212 0000 0000 0180 Mass Storage Cntrlr 10
2 5 0 11AB 4320 1458 E000 0200 Network Cntrlr 12
ACPI Controller 9

```

启动顺序

硬件自检完成后，BIOS把控制权转交给下一阶段的启动程序。

下一阶段的启动程序根据BIOS设置项Boot Sequence（启动顺序）决定，排在前面的设备就是优先转交控制权的设备。



第二阶段：主引导记录

BIOS按照"启动顺序"，把控制权转交给排在第一位的储存设备。

这时，计算机读取该设备的第一个扇区，也就是读取最前面的512个字节。如果这512个字节的最后两个字节是0x55和0xAA，表明这个设备可以用于启动；如果不是，表明设备不能用于启动，控制权于是被转交给"启动顺序"中的下一个设备。

这最前面的512个字节，就叫做"主引导记录"（Master boot record，缩写为MBR）。

主引导记录组成

主引导记录由三个部分组成：

- （1）第1-446字节：调用操作系统的机器码。
- （2）第447-510字节：分区表（Partition table），共64字节。
- （3）第511-512字节：主引导记录签名（0x55和0xAA）。

其中，第二部分"分区表"的作用，是将硬盘分成若干个区。

分区表

硬盘分区有很多好处。每个分区可以安装不同的操作系统，"主引导记录"因此必须知道将控制权转交给哪个区。

分区表的长度只有64个字节，里面又分成四项，每项16个字节。所以，一个硬盘最多只能分四个一级分区，又叫做"主分区"。

每个主分区的16个字节，由6个部分组成：

- （1）第1个字节：如果为0x80，就表示该主分区是激活分区，控制权要转交给这个分区。四个主分区里面只能有一个是激活的。
- （2）第2-4个字节：主分区第一个扇区的物理位置（柱面、磁头、扇区号等等）。

- (3) 第5个字节：[主分区类型](#)。
- (4) 第6-8个字节：主分区最后一个扇区的物理位置。
- (5) 第9-12字节：该主分区第一个扇区的逻辑地址。
- (6) 第13-16字节：主分区的扇区总数。

最后的四个字节 ("主分区的扇区总数")，决定了这个主分区的长度。也就是说，一个主分区的扇区总数最多不超过2的32次方。

如果每个扇区为512个字节，就意味着单个分区最大不超过2TB。再考虑到扇区的逻辑地址也是32位，所以单个硬盘可利用的空间最大也不超过2TB。如果想使用更大的硬盘，只有2个方法：一是提高每个扇区的字节数，二是[增加扇区总数](#)。

第三阶段：硬盘启动

这时，计算机的控制权就要转交给硬盘的某个分区了，这里又分成三种情况。

情况A：卷引导记录

上一节提到，四个主分区里面，只有一个是激活的。计算机会读取激活分区的第一个扇区，叫做"[卷引导记录](#)" (Volume boot record, 缩写为VBR)。

"卷引导记录"的主要作用是，告诉计算机，操作系统在这个分区里的位置。然后，计算机就会加载操作系统了。

情况B：扩展分区和逻辑分区

随着硬盘越来越大，四个主分区已经不够了，需要更多的分区。但是，分区表只有四项，因此规定有且仅有一个区可以被定义成"扩展分区" (Extended partition)。

所谓"扩展分区"，就是指这个区里面又分成多个区。这种分区里面的分区，就叫做"逻辑分区" (logical partition)。

计算机先读取扩展分区的第一个扇区，叫做"[扩展引导记录](#)" (Extended boot record, 缩写为EBR)。它里面也包含一张64字节的分区表，但是最多只有两项 (也就是两个逻辑分区)。

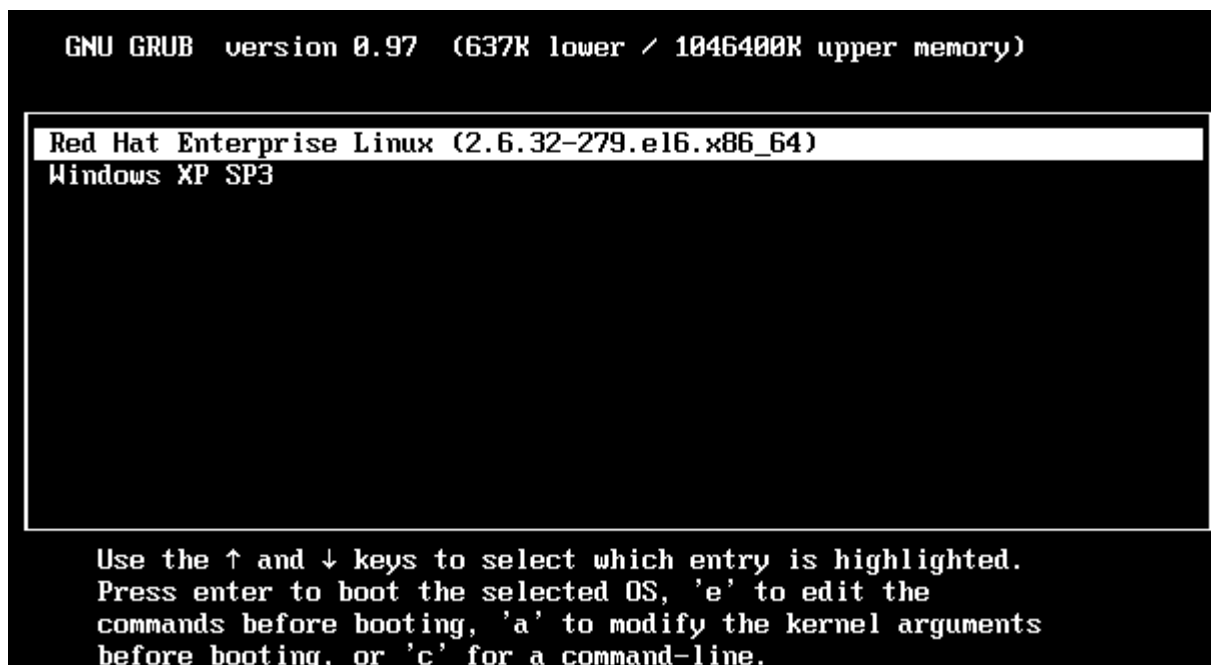
计算机接着读取第二个逻辑分区的第一个扇区，再从里面的分区表中找到第三个逻辑分区的位置，以此类推，直到某个逻辑分区的分区表只包含它自身为止 (即只有一个分区项)。因此，扩展分区可以包含无数个逻辑分区。

但是，似乎很少通过这种方式启动操作系统。如果操作系统确实安装在扩展分区，一般采用下一种方式启动。

情况C：启动管理器

在这种情况下，计算机读取"主引导记录"前面446字节的机器码之后，不再把控制权转交给某一个分区，而是运行事先安装的"[启动管理器](#)" (boot loader)，由用户选择启动哪一个操作系统。

Linux环境中的启动管理器例如[Grub](#)。



第四阶段：操作系统

控制权转交给操作系统后，操作系统的内核首先被载入内存。

以Linux系统为例，先载入/boot目录下面的kernel。内核加载成功后，第一个运行的程序是/sbin/init。它根据配置文件（Debian系统是/etc/initab）产生init进程。这是Linux启动后的第一个进程，pid进程编号为1，其他进程都是它的后代。

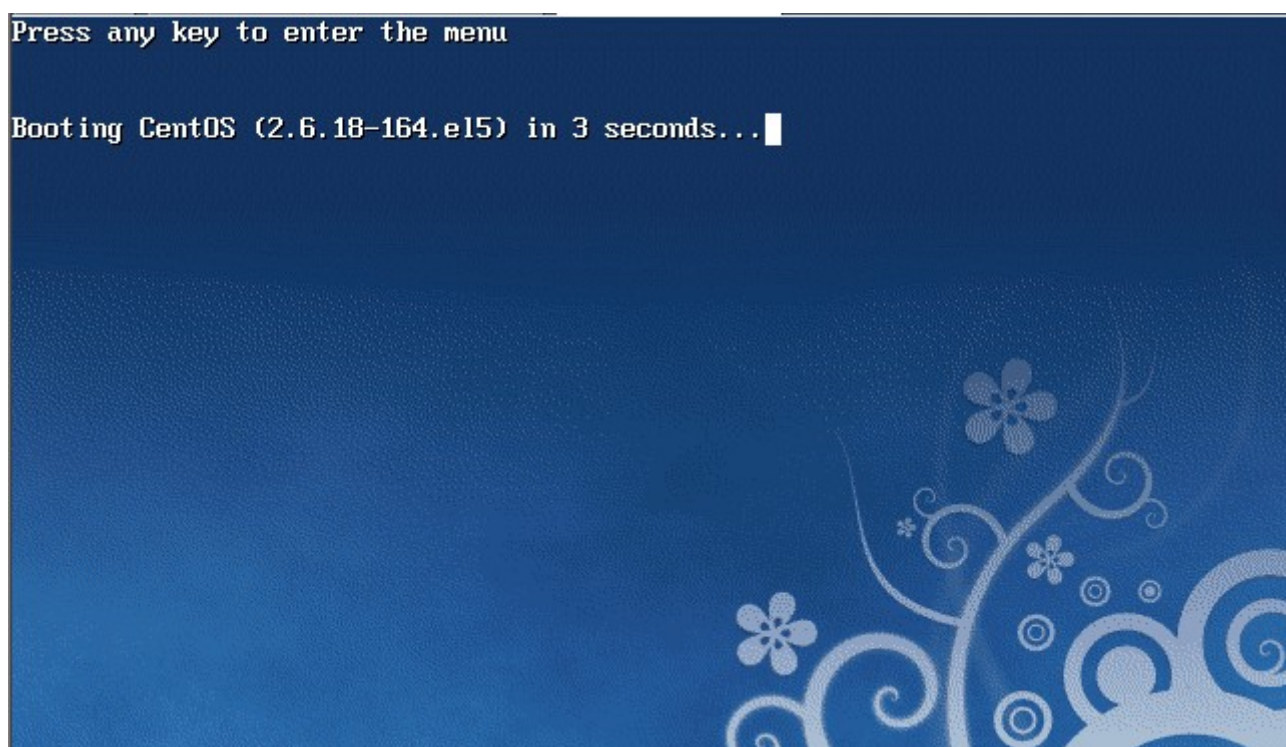
然后，init线程加载系统的各个模块，比如窗口程序和网络程序，直至执行/bin/login程序，跳出登录界面，等待用户输入用户名和密码。

至此，全部启动过程完成。

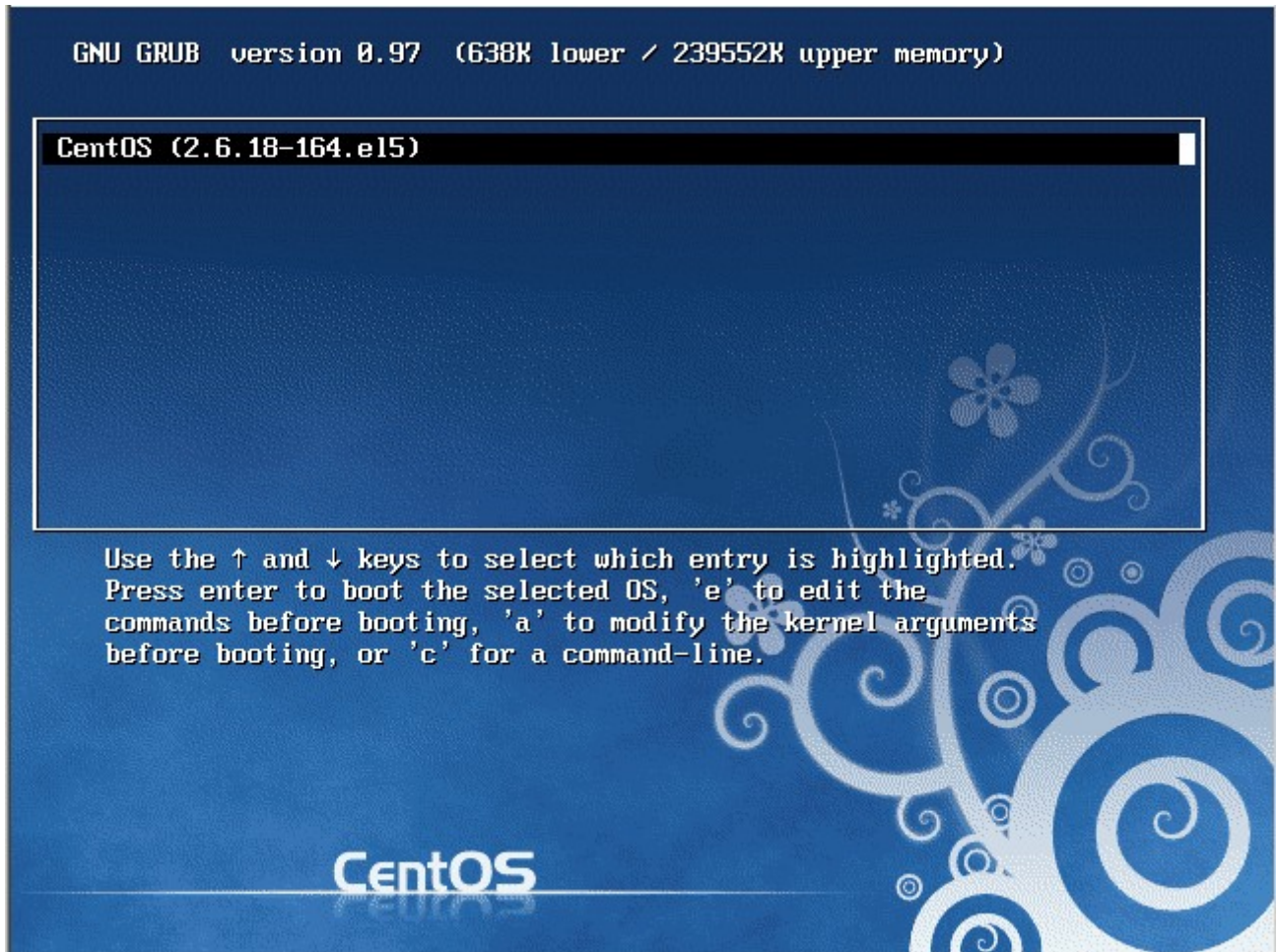
单用户模式修改Centos系统root密码

步骤如下：

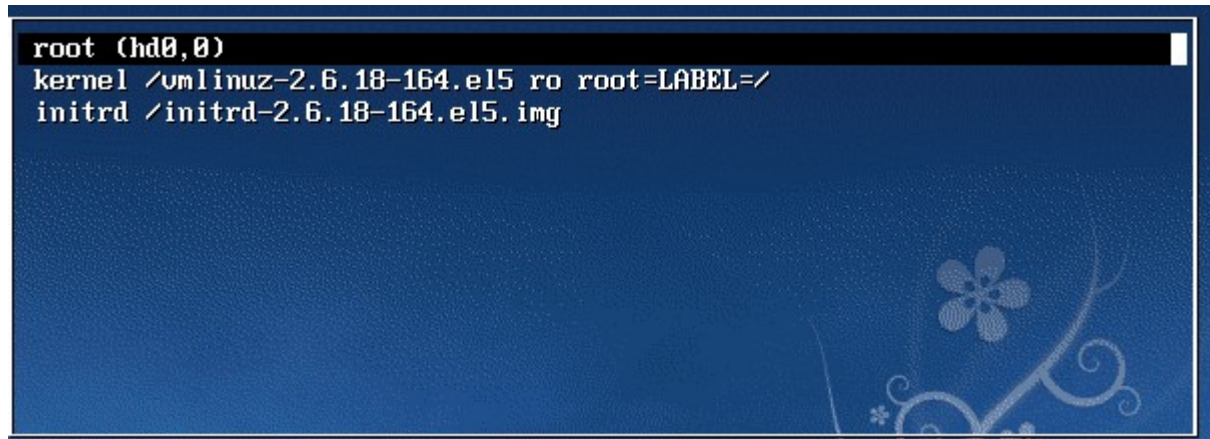
重启linux系统



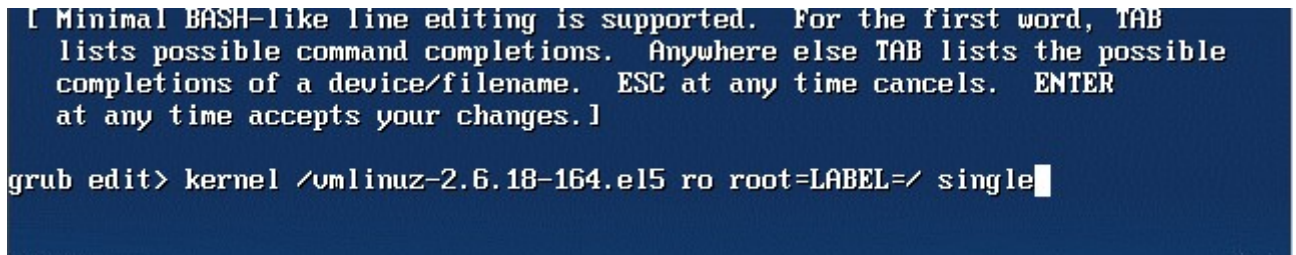
3 秒之内要按一下回车，出现如下界面



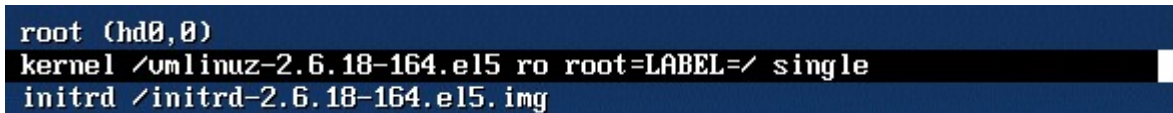
按向下方向键移动到第二行，按"e"进入编辑模式



在 第二行最后边输入 single，用空格与前面内容隔开



回车



最后按"b"启动，启动后就进入了单用户模式了

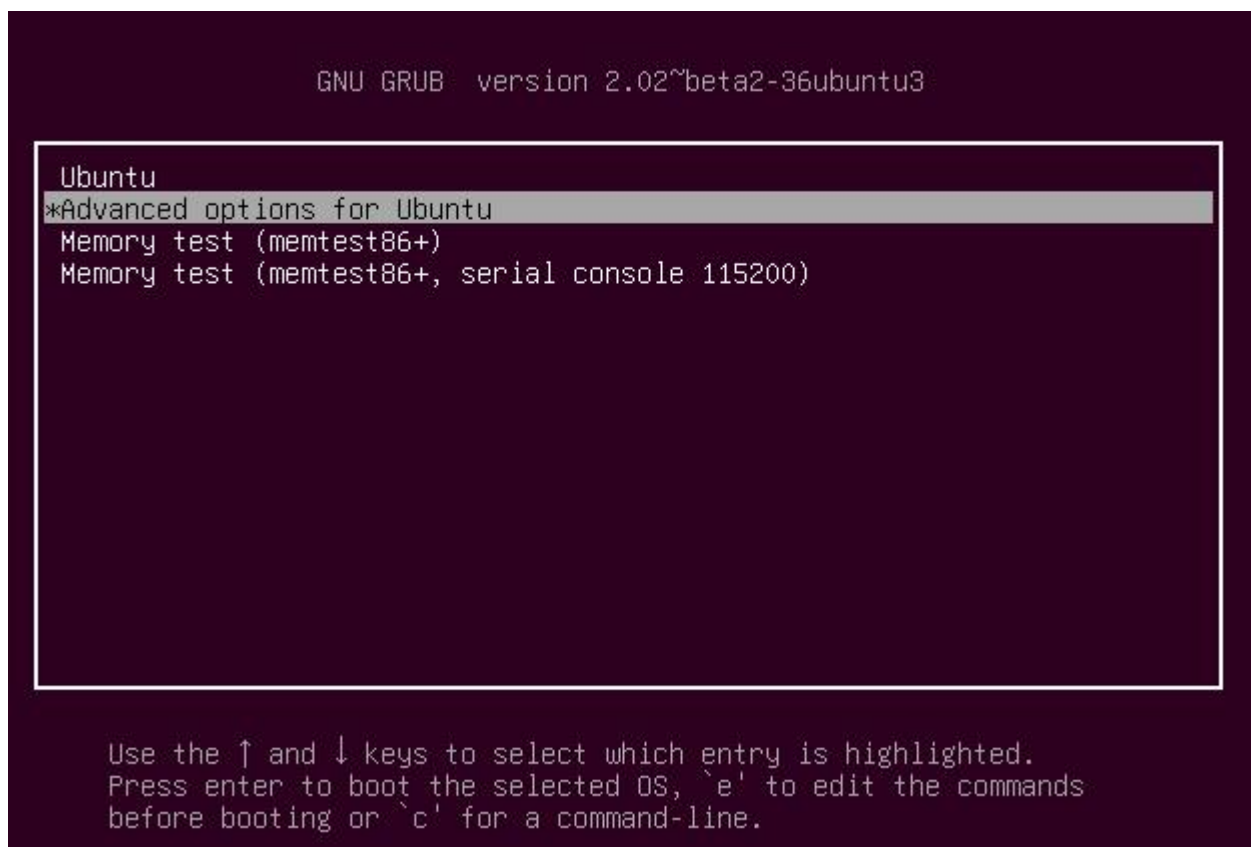
```
no fstab.sys, mounting internal defaults
Switching to new root and running init.
unmounting old /dev
unmounting old /proc
unmounting old /sys
type=1404 audit(1303914022.636:2): enforcing=1 old_enforcing=0 auid=4294967295 s
ses=4294967295
type=1403 audit(1303914023.222:3): policy loaded auid=4294967295 ses=4294967295
INIT: version 2.86 booting
        Welcome to  CentOS release 5.4 (Final)
        Press 'I' to enter interactive startup.
Setting clock  (utc): Wed Apr 27 22:20:50 CST 2011      [  OK  ]
Starting udev:                                         [  OK  ]
Loading default keymap (us):                          [  OK  ]
Setting hostname localhost.localdomain:               [  OK  ]
No devices found
Setting up Logical Volume Management:                 [  OK  ]
Checking filesystems
/: clean, 118508/1969568 files, 713597/1967962 blocks
/boot: clean, 35/26104 files, 14714/104388 blocks
                                                    [  OK  ]
Remounting root filesystem in read-write mode:        [  OK  ]
Mounting local filesystems:                          [  OK  ]
Enabling /etc/fstab swaps:                          [  OK  ]
sh-3.2# _
```

进入到单用户模式后，就可以使用passwd命令任意更改root密码了：

```
sh-3.2# passwd
Changing password for user root.
New UNIX password:
Retype new UNIX password:
passwd: all authentication tokens updated successfully.
```

救援模式修改Ubuntu系统root密码

重启，按住shift键，出现如下界面，选中如下选项



按回车键进入如下界面，然后选中最新的recovery mode选项

GNU GRUB version 2.02~beta2-36ubuntu3

```
Ubuntu, with Linux 4.4.0-24-generic
Ubuntu, with Linux 4.4.0-24-generic (upstart)
*Ubuntu, with Linux 4.4.0-24-generic (recovery mode)
Ubuntu, with Linux 4.4.0-21-generic
Ubuntu, with Linux 4.4.0-21-generic (upstart)
Ubuntu, with Linux 4.4.0-21-generic (recovery mode)
```

Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the commands
before booting or 'c' for a command-line. ESC to return previous
menu.

按e进入如下界面，找到图中红色框的recovery nomodeset并将其删掉，再在这一行的后面输入

```
1 | quiet splash rw init=/bin/bash
```

GNU GRUB version 2.02~beta2-36ubuntu3

```
if [ x$feature_platform_search_hint = xy ]; then
    search --no-floppy --fs-uuid --set=root --hint-bios=hd\
0,msdos1 --hint-efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1 8881dd24-d\
0d5-4cbe-8149-0f5d4145ca6d
else
    search --no-floppy --fs-uuid --set=root 8881dd24-d0d5-\
4cbe-8149-0f5d4145ca6d
fi
echo          'Loading Linux 4.4.0-24-generic ...'
linux         /boot/vmlinuz-4.4.0-24-generic root=UUID=88\
81dd24-d0d5-4cbe-8149-0f5d4145ca6d ro recovery nomodeset find_preseed=/p\
reseed.cfg auto noprompt priority=critical locale=en_US
echo          'Loading initial ramdisk ...'
initrd        /boot/initrd.img-4.4.0-24-generic
```

Minimum Emacs-like screen editing is supported. TAB lists
completions. Press Ctrl-x or F10 to boot, Ctrl-c or F2 for a
command-line or ESC to discard edits and return to the GRUB
menu.

GNU GRUB version 2.02~beta2-36ubuntu3

```
if [ x$feature_platform_search_hint = xy ]; then
    search --no-floppy --fs-uuid --set=root --hint-bios=hd\
0,msdos1 --hint-efi=hd0,msdos1 --hint-baremetal=ahci0,msdos1 8881dd24-d\
0d5-4cbe-8149-0f5d4145ca6d
else
    search --no-floppy --fs-uuid --set=root 8881dd24-d0d5-\
4cbe-8149-0f5d4145ca6d
fi
echo          'Loading Linux 4.4.0-24-generic ...'
linux         /boot/vmlinuz-4.4.0-24-generic root=UUID=88\
81dd24-d0d5-4cbe-8149-0f5d4145ca6d ro find preseed=/preseed.cfg auto no\
prompt priority=critical locale=en_US quiet splash rw init=/bin/bash_
echo          'Loading initial ramdisk ...'
initrd        /boot/initrd.img-4.4.0-24-generic
```

Minimum Emacs-like screen editing is supported. TAB lists completions. Press Ctrl-x or F10 to boot, Ctrl-c or F2 for a command-line or ESC to discard edits and return to the GRUB menu.

接着按F10或者Ctrl+x 后出现如下界面，在命令行内输入passwd后进行修改密码即可

```
[ 1.742917] sd 2:0:0:0: [sda] Assuming drive cache: write through
/dev/sda1: clean, 224796/1245184 files, 1180183/4980480 blocks
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@(none):/# passwd
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@(none):/#
```

修改完之后重启系统。

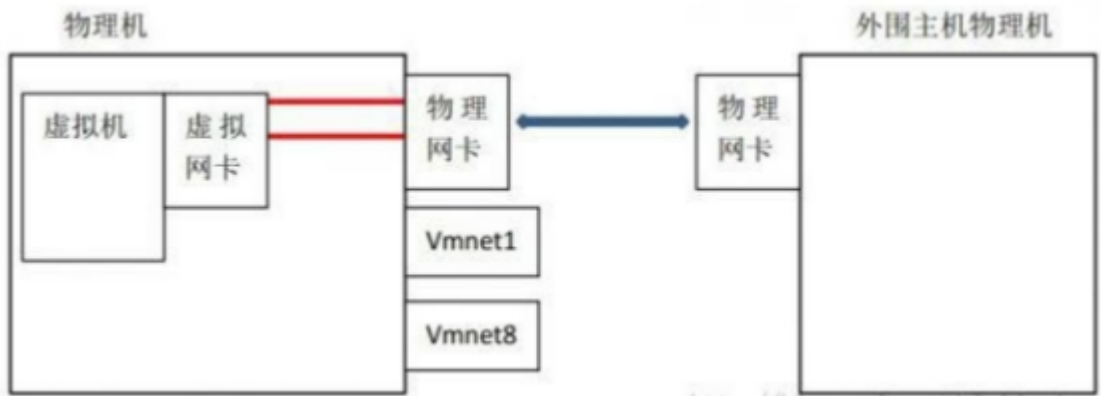
虚拟机

三种网络模式

桥接

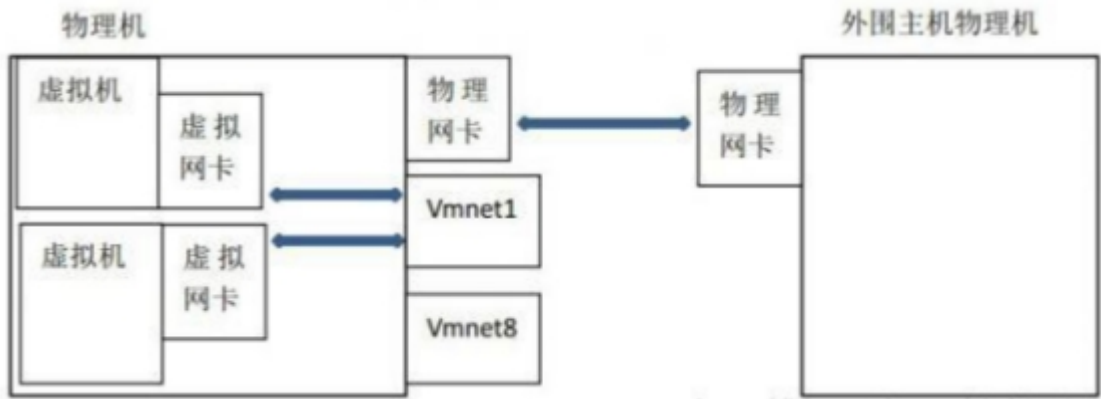
在网络网卡上安装了一个桥接协议，让这块网卡处于混杂模式，可以同时连接多个网络的做法。

桥接下，类似于把物理主机虚拟为一个交换机，所有桥接设置的虚拟机连接到这个交换机的一个接口上，物理主机也同样查在这个交换机当中，所以所有桥接下网卡与网卡都是交换模式的，相互可以访问而不干扰。



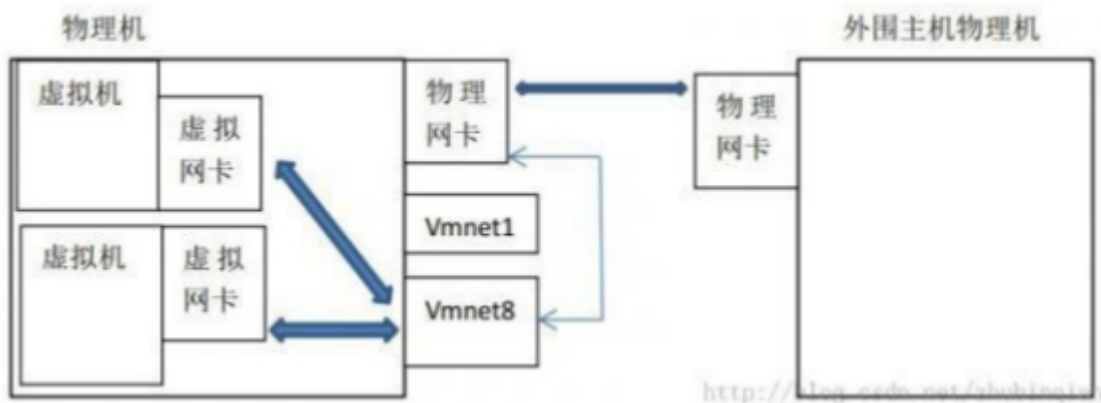
Host-only（仅与主机通信）

虚拟机使用VMnet1网卡与主机单独组网,主机对于虚拟机相当于路由器



NAT

虚拟机使用VMnet8网卡与主机单独组网,主机对于虚拟机相当于路由器，VMnet8网卡通过NAT地址转换协议与物理机网卡通信



	桥接模式	仅主机模式	NAT 模式
使用的网卡	物理网卡	VMnet1	VMnet8
虚拟机 ip	192.168.1.3	192.168.20.128	192.168.62.128
虚拟机能访问其他物理机吗	能	不能	能
其他物理机能访问虚拟机吗	能	不能	不能

常见问题

修改静态地址后发现无法ping外网

```
1  需要设置网关
2  route add default gw 192.168.33.1
3  添加nameserver
4  vi /etc/resolv.conf
5  nameserver 192.168.33.1
```

虚拟机克隆后eth0消失

```
1  直接修改 /etc/sysconfig/network-script/ifcfg-eth0
2  删掉UUID HWADDR
3  配置静态地址
4  然后:
5  rm -rf /etc/udev/rules.d/70-persistent-net.rules
6  然后 reboot
```